

FORMATION PYTHON
CONCEPTS DE BASE,
DATA PREPARATION ET MACHINE LEARNING
JANVIER 2021



MICROPOLÉ

JOURS 1 & 2 :

**INTRODUCTION AU LANGAGE
PYTHON ET MANIPULATION
DE DONNÉES**



01

Les principaux environnements de développement Python

02

Les concepts de base en Python

03

Introduction aux packages NUMPY et SCIPY

04

Import et préparation des données avec Pandas

05

Statistiques descriptives

06

Graphiques avec Matplotlib

LES PRINCIPAUX ENVIRONNEMENTS DE DÉVELOPPEMENT EN PYTHON

INTRODUCTION

Python est un langage open source de haut niveau sorti initialement en 1991.

Il a été développé par **Guido Van Rossum**, en s'inspirant du langage ABC.



Le nom du langage fait référence au Monty Python.

Toute référence à ce groupe est la bienvenue dans la communauté de Python.

Le langage est désormais maintenu par la Python Software Foundation créée en 2001.

<https://www.python.org/psf/>

Le développement du langage exponentiel : il y a actuellement plus de 240 000 packages disponibles sur <https://pypi.python.org/pypi>

LES PRINCIPAUX ENVIRONNEMENTS DE DÉVELOPPEMENT EN PYTHON

INTRODUCTION



- Il existe 2 versions de python : python 2.x et python 3.x
- La v2 est arrêtée à la version 2.7.12 et la dernière version à date de la v3 est la 3.9.1
- La v3 est sortie en décembre 2008 et a été créée dans le but d'améliorer certaines faiblesses du langage, et notamment en renforçant le paradigme objet.



La version 2 n'est plus maintenue à partir de janvier 2020.



Guido van Rossum ✓
@gvanrossum

Abonné



Remember that Python 2 will reach end of life on 1/1/2020. For the impact on *your* project, see python3statement.org

LES PRINCIPAUX ENVIRONNEMENTS DE DÉVELOPPEMENT EN PYTHON

INTRODUCTION



Les avantages de python :

- En comparaison aux différents autres langages de programmation, Python offre de nombreux avantages



Aucune gestion de la mémoire



Syntaxe facile



Pas d'étape de compilation



Type de données de haut niveau nativement



Communauté dynamique

- Python peut être intégré directement avec les librairies les plus populaires grâce à Anaconda
- Chaque librairie additionnelle peut être intégrée grâce aux commandes conda (prompt)
- Les exemples de code présentés dans la suite du support sont écrits en Python 3



LES PRINCIPAUX ENVIRONNEMENTS DE DÉVELOPPEMENT EN PYTHON

INTRODUCTION

Ses principales caractéristiques :

- Langage de haut niveau mais simplifié : on ne s'occupe pas de parties techniques comme l'occupation de mémoire (contrairement à C)
- Open source
- Multiplateforme : le même programme fonctionnera sous Linux, Windows ou Mac OS X
- Python est très complet grâce à un large choix de packages
- C'est à la fois un langage script (c'est-à-dire un langage qui autorise des actions au moment de l'exécution là où d'autres langages ne les autorisent qu'au moment de la compilation : évaluation d'expression dynamiques, fermetures,...) et un langage objet (on peut manipuler des objets dans Python qui héritent des propriétés des objets dont ils découlent)

L'indentation en langage Python :

```
if 5 > 2:  
    print("Cinq est plus grand que deux!")
```



```
if 5 > 2:  
print("Cinq est plus grand que deux!")
```



LES PRINCIPAUX ENVIRONNEMENTS DE DÉVELOPPEMENT EN PYTHON

LE ZEN DE PYTHON

➤ Tout pythonien doit suivre les 19 règles du zen, déterminant un « bon » code.

➤ Dans python, tapez : `import this`

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!

LES PRINCIPAUX ENVIRONNEMENTS DE DÉVELOPPEMENT EN PYTHON

SPYDER



Spyder – c'est quoi ?

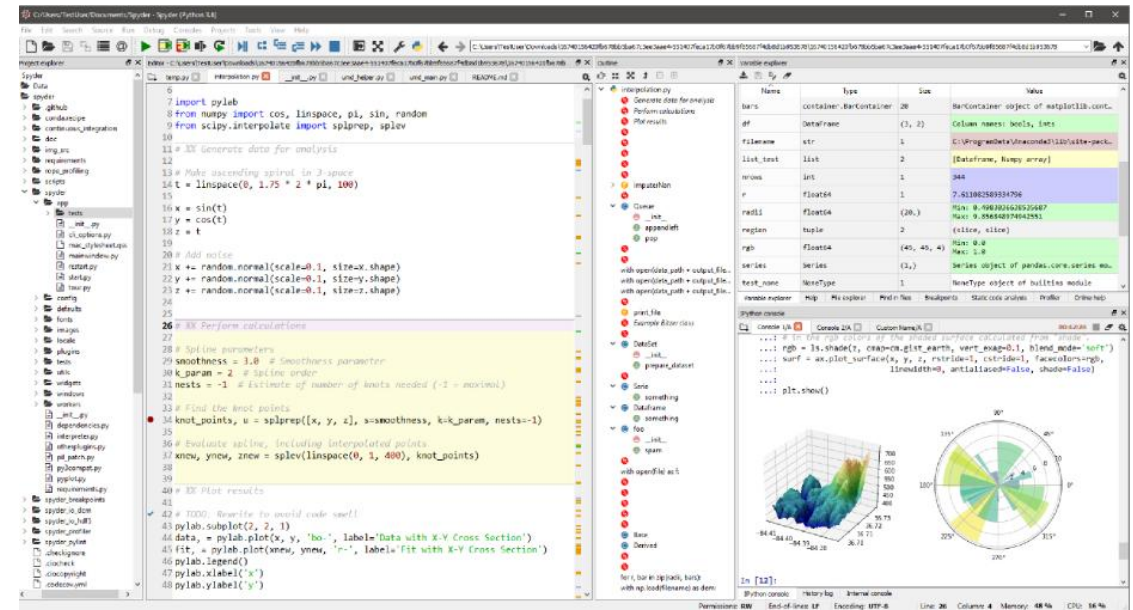
➤ IDE (Environnement de Développement Intégré), Open-Source supportant le langage Python.

➤ Utilisable sur toutes les plateformes (Windows, MacOS, linux et Unix) sous license libre du MIT

➤ Téléchargement principalement depuis Anaconda.

➤ Interface proche de celle de R Studio.

Aperçu de l'interface graphique



- Raccourci clavier pour exécuter le code sélectionné : F9
- Sauvegarde du fichier en format python *.py
- Insertion de commentaires (très important) : « # »

LES PRINCIPAUX ENVIRONNEMENTS DE DÉVELOPPEMENT EN PYTHON

JUPYTER



Jupyter – c'est quoi ?

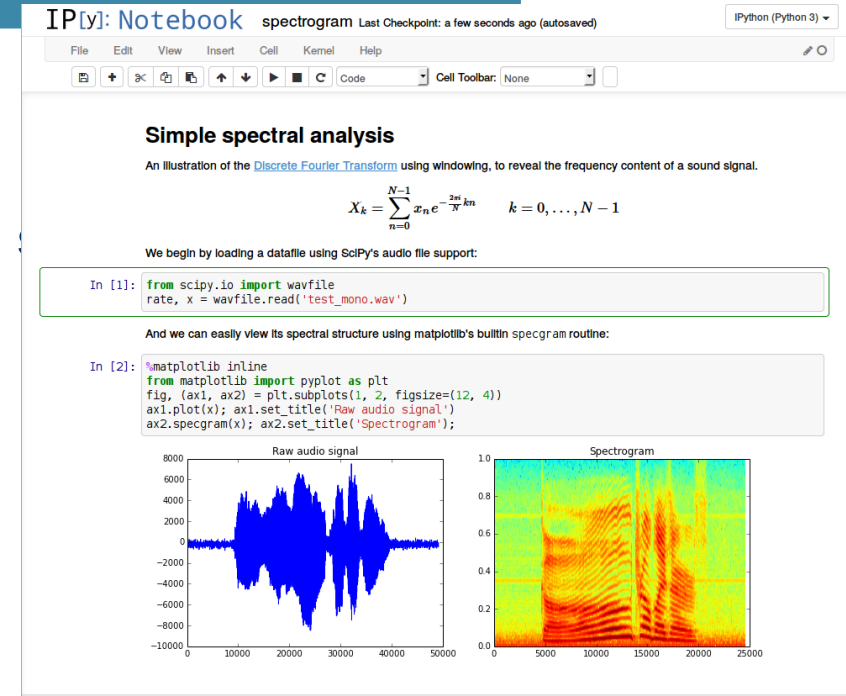
➤ Interface web permettant de coder en plusieurs langages dont Python.

➤ Intégré directement dans Anaconda ou en stand alone

Exécuter la commande « jupyter notebook » dans Anaconda Prompt

➤ Notebook permettant d'allier commentaires et codes

Aperçu de l'interface graphique



- Raccourci clavier pour exécuter un code directement : « Shift + Enter » ou « Ctrl + Enter »
- Sauvegarde du fichier en format Ipython Notebook (ipynb extension)
- Génération du code en format python (.py extension) : File - Download option
- Insertion de commentaires (très important) : « # »

LES PRINCIPAUX ENVIRONNEMENTS DE DÉVELOPPEMENT EN PYTHON

PYCHARM



Pycharm – c'est quoi ?

➤ IDE (Environnement de Développement Intégré, Open-Source supportant le langage Python.

➤ Téléchargement depuis :

<https://www.jetbrains.com/pycharm/>

➤ Interface de développeur.

Aperçu de l'interface graphique

```
20 response = self.client.get(reverse('polls:index'))
21 self.assertEqual(response.status_code, 200)
22 self.assertContains(response, "No polls are available.")
23 self.assertQuerysetEqual(response.context['latest_question_list'], [])
24
25 @test
26 def test_index_view_with_a_future_question(self):
27     """
28     test_index_view_with_a_past_question(self)
29     test_index_view_with_future_question_and_past_question
30     test_index_view_with_no_questions(self)
31     test_index_view_with_two_past_questions(self)
32     """
33     cr = _testMethodDoc
34     re = _testMethodName
35     se = countTestCases(self)
36     defaultTestResult(self)
37
38
39 def test_index_view_with_a_future_question(self):
40     """
41     Questions with a pub_date in the future should not be displayed on
42     the index page.
43     """
44     create_question(question_text="Future question.", days=30)
45     response = self.client.get(reverse('polls:index'))
46     self.assertContains(response, "No polls are available.",
47                         status_code=200)
48     self.assertQuerysetEqual(response.context['latest_question_list'], [])
49
50 def test_index_view_with_future_question_and_past_question(self):
51     """
52     Even if both past and future questions exist, only past questions
53     should be displayed.
54     """
55     create_question(question_text="Past question.", days=-30)
56     create_question(question_text="Future question.", days=30)
57     response = self.client.get(reverse('polls:index'))
58     self.assertQuerysetEqual(
59         response.context['latest_question_list'],
60         ['<Question: Past question.>']
61     )
62
63 def test_index_view_with_two_past_questions(self):
64     """
65     """
```

LES PRINCIPAUX ENVIRONNEMENTS DE DÉVELOPPEMENT EN PYTHON

VISUAL STUDIO



Visual Studio – c'est quoi ?

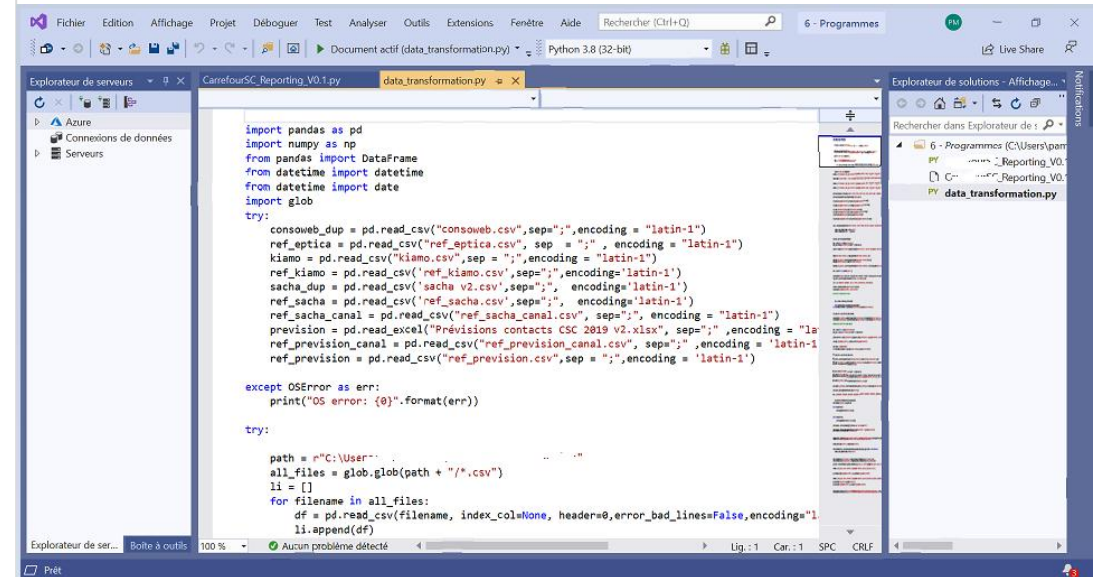
➤ IDE utilisé par les outils de Microsoft.

➤ Téléchargement depuis :

<https://visualstudio.microsoft.com/fr/>

➤ Idéal pour connexion avec un git.

Aperçu de l'interface graphique



VERSIONS DE PYTHON ET DES PACKAGES POUR LA FORMATION

➤ Il est important de fixer les versions utilisées dans Python. Certaines fonctionnalités et/ou modules peuvent évoluer voire être supprimés.

➤ Voici les versions que nous allons utiliser :

➤ Version Python : 3.7.9

➤ Version par package

- matplotlib 3.3.0
- numpy 1.19.0
- pandas 1.2.0
- scikit-learn 0.23.2
- scikit-optimize 0.8.1
- scipy 1.5.4
- seaborn 0.11.0
- statsmodels 0.12.1
- xgboost 1.3.3
- lightgbm 3.1.1

01

Les principaux environnements de développement Python

02

Les concepts de base en Python

03

Introduction aux packages NUMPY et SCIPY

04

Import et préparation des données avec Pandas

05

Statistiques descriptives

06

Graphiques avec Matplotlib

CONCEPTS DE BASE

PREMIÈRES OPÉRATIONS

Affectations – Typage automatique

```
a = 1.2
```

a est une variable, en interne elle a été automatiquement typée en réel « float » parce qu'il y a un point décimal

a est l'identifiant de la variable

= est l'opérateur d'affectation

Calcul

```
b = a + 3
```

b est une variable contenant la valeur 4.2

Forcer le typage d'une variable (sert aussi pour le transtypage)

```
c = float(1)
```

Même sans point décimal, **c** sera considéré comme **float**

(c = 1 → c est considéré comme **int**)

Connaitre le type d'un objet

```
type(nom_de_variable)
```

Afficher le type interne d'une variable

```
type(a) → float
```

Supprimer un objet de la mémoire

```
del nom_de_variable
```

CONCEPTS DE BASE

LES TYPAGES NUMÉRIQUES

➤ `int` (entier) et `float` (double/ décimal)

```
entier = 10
print(type(entier))
```

→ `<class 'int'>`

```
reel = 10.5
print(type(reel))
```

→ `<class 'float'>`

```
piege = 10.0
print(type(piege))
```

→ `???`

➤ Les opérateurs :

`+`, `-`, `*`, `/` (division réelle), `//` (division entière), `%` (modulo), `**` (puissance)

```
nombre = 41
nombre = nombre + 2 # Addition
nombre = nombre - 2 # Soustraction
print('1. Multiplication :', nombre * 2)
print('2. Division :', nombre / 2)
print('3. Exposant :', nombre ** 2)
print('4. Partie entière de la division par 2 :', nombre // 2)
print('5. Partie restante de la division par 2 :', nombre % 2)
```

→

1. Multiplication : 82
2. Division : 20.5
3. Exposant : 1681
4. Partie entière de la division par 2 : 20
5. Partie restante de la division par 2 : 1

CONCEPTS DE BASE

LE TYPAGE BOOLÉEN

➤ Bool

```
vrai = True  
faux = False  
print(type(vrai), ' ', type(faux))
```

→ <class 'bool'> , <class 'bool'>

```
a = bool(0)  
b = bool(1)  
print(type(a), ' ', type(b))
```

→ <class 'bool'> , <class 'bool'>

```
x = (5 > 1)  
y = (5 < 1)  
z = (5 == 1)  
print('x =', x)  
print('y =', y)  
print('z =', z)
```

→
???
???
???

➤ Les opérateurs :

- **not** (négation)
- **and** / **&** (et logique)
- **or** / **|** (ou logique)

CONCEPTS DE BASE

LE TYPAGE CHAÎNE DE CARACTÈRES

➤ Str (string)

```
texte1 = 'Paris, Paris outragé, Paris brisé,'  
texte2 = "Paris martyrisé mais Paris libéré!"  
print(type(texte1))
```

<class 'str'>

#Manipulation de texte

```
print('5 premiers caractères de texte1 :',texte1[0:5])  
print('5 premiers caractères de texte1 :',texte1[:5])  
print('6 derniers caractères de texte1 :',texte1[28:])  
print('6 derniers caractères de texte1 :',texte1[-6:])
```

5 premiers caractères de texte1 : Paris
5 premiers caractères de texte1 : Paris
6 derniers caractères de texte1 : brisé,
6 derniers caractères de texte1 : brisé,

```
print('Extraire le sous-texte "Paris outragé" de texte1 :',???)
```

Paris outragé

#Concaténation de textes

```
print(texte1 + texte2)  
print(texte1,texte2) #ajoute un espace
```

Paris, Paris outragé, Paris brisé,Paris martyrisé mais Paris libéré!
Paris, Paris outragé, Paris brisé, Paris martyrisé mais Paris libéré!

```
print('Concaténer "outragé" et "libéré" de texte1 et texte2 : ', ???)
```

Outragé, libéré

#Multiplication de texte

```
print(2*texte1)
```

Paris, Paris outragé, Paris brisé,Paris, Paris outragé, Paris brisé,

CONCEPTS DE BASE

SLICING SUR CHAÎNES DE CARACTÈRES

Action	Code	Sortie (avec <code>s = "ABCDEFGHIJKL"</code>)
Extraction	<code>s[2:7]</code>	CDEFG
Les 4 premiers	<code>s[:4]</code>	ABCD
Les 4 derniers	<code>s[-4:]</code>	IJKL
Tous sauf les 4 premiers	<code>s[4:]</code>	EFGHIJKL
Tous sauf les 4 derniers	<code>s[:-4]</code>	ABCDEFGH
Partitionner	<code>s[:3], s[3:7], s[7:]</code>	('ABC', 'DEFG', 'HIJKL')
De 3 en 3	<code>s[::3]</code>	ADGJ
De 3 en 3 à partir de la fin	<code>s[::-3]</code>	LIFC
Les indices pairs	<code>s[::2]</code>	ACEGIK
Les indices impairs	<code>s[1::2]</code>	BDFHJL
Copie superficielle	<code>s[:]</code>	ABCDEFGHIJKL
Copie à l'envers	<code>s[::-1]</code>	LKJIHGFEDCBA

CONCEPTS DE BASE

INSTANCIATION ET AFFECTATION

➤ Affectation simple

#typage automatique

```
a = 1
```

Le '#' permet d'ajouter des commentaires au script

#typage explicite

```
a = float(1)
```

Le typage explicite est conseillé

➤ Affectation multiple

#même valeur pour plusieurs variables

```
a = b = 2.5
```

#affectations parallèles

```
a, b = 2.5, 3.2
```

Le plus couramment utilisé : 1 instruction = 1 ligne

```
a = 1
```

```
b = 5
```

```
a = 3 + b
```

CONCEPTS DE BASE

TRANSTYPAGE

➤ Schéma type

Nouveau_type(var)

➤ Exemple

#Conversion numérique

a = '12' # a est une chaîne de caractère

b = float(a) # b = 12.0

print(type(a))

print(type(b))

<class 'str'>
<class 'float'>



#Conversion en booléen

a = bool('False') # a = True

b = bool(0) # b = False

print(type(a))

print(type(b))

<class 'bool'>
<class 'bool'>

#Conversion en chaîne de caractère

a = str(15) # a = '15'

print(type(a))

<class 'str'>

CONCEPTS DE BASE

LES OPÉRATEURS DE COMPARAISON

Objectif : comparer deux éléments **du même type**

- Remarque : `int` et `float` peuvent être comparés)

Résultat : un booléen (`True` / `False`)

Description	Opérateur en Python	Exemples
Egalité	<code>==</code>	<code>1 == 1</code>
Différence	<code>!=</code>	<code>1 != 2</code>
Plus grand	<code>></code>	<code>2 > 1</code>
Plus grand ou égal	<code>>=</code>	<code>1 >= 2</code>
Plus petit	<code><</code>	<code>1 < 2</code>
Plus petit ou égal	<code><=</code>	<code>1 <= 2</code>

CONCEPTS DE BASE

L'INSTRUCTION « IF, ELIF, ELSE »

Schéma :

```
if condition :  
    bloc d'instructions  
elif condition :  
    bloc d'instructions  
else :  
    bloc d'instructions
```

Remarque :

- Les **:** sont primordiaux
- C'est l'**indentation** (le décalage vers la droite) qui délimite le bloc d'instructions
- **else** et **elif** sont facultatifs et ne s'activent que si la condition du **if** n'est pas vérifiée

CONCEPTS DE BASE

L'INSTRUCTION « IF, ELIF, ELSE » : EXEMPLE D'APPLICATION

```
#Saisie à la console du prix produit
pht = int(input('Prix HT produit : '))
#Saisie à la console du code produit
code = int(input('Code produit : '))

#Action conditionnelle
if (code == 1):
    pttc = pht * 1.055
elif (code == 2):
    pttc = pht * 1.1
else:
    pttc = pht * 1.2

#Affichage avec transtypage
print("Prix TTC : " + str(pttc))
```

Questions :

- Que vaut `pttc` si `code = 4` ?
- Que vaut `pttc` si l'on supprime la condition `else` et que `code = 4` ?

CONCEPTS DE BASE

BOUCLE « FOR »

Schéma :

```
for i in sequence :  
    bloc d'instructions
```

« sequence » est une collection de valeurs:

- Une liste, un tuple... (nous verrons cela plus tard)
- Utilisation de la fonction **range()** pour générer une suite d'entiers :
 - Paramétrage : **range(stop)** ou **range(start, stop[, step])**
 - range(3)** → 0 1 2
 - range(1,4)** → 1 2 3
 - range(0,5,2)** → 0 2 4

Remarques :

- Attention à l'indentation et aux :
- On peut stopper la boucle avec l'opérateur **break**
- On peut passer directement à l'itération suivante avec **continue**
- On peut décider de ne rien faire pour une ligne avec **pass**
- Des boucles imbriquées sont possibles **mais attention** au temps de calcul
- Le bloc d'instructions peut contenir des conditions

CONCEPTS DE BASE

BOUCLE « FOR » : EXEMPLE D'APPLICATION

```
#On choisit la valeur limite
n = int(input("n : "))
if n > 10**6 :
    print('Vous avez choisi un nombre trop grand.')
    n = 10**6
    print('n a été fixé à 1000000')

#Initialisation
somme_impair = 0
somme_totale = 0

#Calcul de la somme
for i in range(1, n+1): #n+1 car range(1,n) s'arrête à n-1
    #Somme des valeurs impaires
    if (i % 2 == 1):
        somme_impair = somme_impair + i
    #On quitte le if mais on reste dans la boucle for
    somme_totale = somme_totale + i

#On sort de la boucle for et on affiche les résultats
print("Somme des valeurs impaires : " + str(somme_impair))
print("Somme totale : " + str(somme_totale))
```


CONCEPTS DE BASE

L'INSTRUCTION « BREAK »

- Il est possible de sortir d'une boucle.
- Cette étape passe par la commande `break`.
- L'exemple suivant permet de ne pas sommer si le nombre est supérieur à 10.

```
nb = 100

total = 0
for i in range(nb+1):
    if nb>10:
        break
    else:
        total +=i

print('Le total des nombre entre 1 et nb est : ',total)
```

CONCEPTS DE BASE

L'INSTRUCTION « CONTINUE »

- Dans une boucle, il est possible de passer directement à la suivante itération.
- Cette étape passe par la commande `continue`.
- L'exemple suivant permet de sommer les nombres pairs.

```
nb = 10
total = 0
for i in range(nb+1):
    if i%2 != 0:
        continue
    total +=i

print('Résultat des nombres pairs entre 1 et 10 : ', total)
```

```
Résultat des nombres pairs entre 1 et 10 : 30
```

CONCEPTS DE BASE

L'INSTRUCTION « PASS »

- Dans une boucle, il est possible de ne rien faire sur la ligne.
- Cette étape passe par la commande `pass`.
- L'exemple suivant somme les nombres pairs entre 0 et 10.

```
nb = 10
total = 0
for i in range(nb+1):
    if i%2 != 0:
        pass
    else:
        total +=i

print('Résultat des nombres pairs entre 1 et 10 : ', total)
```

```
Résultat des nombres pairs entre 1 et 10 : 30
```

CONCEPTS DE BASE

BOUCLE « WHILE »

Schéma :

```
while condition :  
    bloc d'instructions
```

Remarques :

- Attention à l'indentation et aux :
- On peut stopper la boucle avec l'opérateur **break**
- Des boucles imbriquées sont possibles **mais attention** au temps de calcul
- Le bloc d'instructions peut contenir des conditions
- **Attention à la boucle infinie !**

CONCEPTS DE BASE

BOUCLE « WHILE » : EXEMPLE D'APPLICATION

```
n = int(input("n : "))

#Initialisation
somme_impaire = 0
somme_totale = 0

#Calcul de la somme et de la sommedes valeurs impaires
i = 0 #il faut initialiser i
while i <= n : # cette fois n car on utilise "<="
    #Somme des valeurs impaires
    if (i % 2 == 1):
        somme_impaire = somme_impaire + i
    #On quitte le if mais on reste dans la boucle while
    somme_totale = somme_totale + i
    i = i + 1

#On sort de la boucle while et on affiche les résultats
print("Somme des valeurs impaires :", str(somme_impaire))
print("Somme totale : ", somme_totale)
```

CONCEPTS DE BASE

BOUCLE « WHILE » AVEC « BREAK »

➤ L'instruction `break` est utilisable dans une boucle `while`

➤ On peut alors spécifier l'instruction « `while True` »

```
nb = input("Entrez un nombre entier entre 1 et 10 : ")
nb = int(nb)

total = 0
while True :
    total += nb
    nb -= 1
    if nb == 0 : break

print('Le total des nombres entre 1 et nb est :',total)
```

CONCEPTS DE BASE

LES TUPLES

- Un n-tuple est un ensemble non modifiable ordonné non dédoublonné d'objets.
- Un tuple contient de 0 à plusieurs objets Python.
- Les () définissent le type tuple et les éléments sont séparés par une virgule

#Création d'un tuple

```
nuplet = ('caractères',12,True,1.25)
```

#Identifier un élément du tuple

```
print('2ème élément de nuplet :',nuplet[2])
```

True

Modification d'un tuple impossible car objet non mutable

```
nuplet[2] = 4
```

TypeError: 'tuple' object does not support item assignment

CONCEPTS DE BASE

OPÉRATIONS SUR TUPLES (1/2)

```
t1 = (2, 3, 4, 5)
```

```
print(t1) # → (2, 3, 4, 5)
```

```
len(t1) # → 4 (correspond à la taille du tuple)
```

```
t1[0] # → 2 (accès aux éléments grâce aux indices, le premier indice est le 0)
```

```
t1[2] = 10 # → Error (les éléments d'un tuple ne sont pas modifiables)
```

```
t1[1:3] # → (3, 4) (récupère de l'indice 1 inclus à l'indice 3 non inclus. (3, 4) est un tuple)
```

```
t1[:2] # → (2, 3) (les 2 premiers éléments)
```

```
t1[-1] # → 5 (le dernier élément, indice négatif)
```

```
t1[-2:] # → (4, 5) (les 2 derniers éléments)
```

```
t2 = (2, 3, 3, 4)
```

```
set(t2) # → {2, 3, 4} avec type({2, 3, 4}) égal à « set »
```


CONCEPTS DE BASE

OPÉRATIONS SUR TUPLES (2/2)

Concaténation :

```
t1 = (1,2,3,4)
t2 = (5,2,4)
t1 + t2 # → (1, 2, 3, 4, 5, 2, 4)
```

Réplication :

```
2 * t2 # → (5, 2, 4, 5, 2, 4)
```

Possibilité d'avoir des objets de type différents dans un même tuple:

```
t3 = ('caractères',12,True,1.25)
```

Tuples de tuples

```
t4= ((2,3,4),(6,7,9),(2,3))
```

Opérations sur les indices :

```
t = (2,3,6,8,6)
```

Rechercher un élément : `present = 3 in t` → `present = True` (car 3 est dans t)

Renvoie l'index de la première occurrence de l'élément : `idx = t.index(6)` → `idx = 2`

Compte les occurrences d'un élément : `nb = t.count(6)` → `nb = 2`

CONCEPTS DE BASE

LES LISTES (1/2)

Définition d'une liste

`list1 = [2,3,4,5]` # → les `[ ]` définissent le type liste et les éléments sont séparés par une virgule

Opérations

- Les opérations utilisées pour les tuples fonctionnent pour les listes
- Comme pour les tuples les listes peuvent contenir des objets de type différents
- Même raisonnement pour les indices

La différence par rapport aux tuples : les listes sont modifiables

- `list1[1] = 10` # → `list1 = [2,10,4,5]`

Opérations spécifiques aux listes :

- Ajout d'un élément : `list1.append(33)` # → `list1 = [2,10,4,5,33]`
- Insertion d'un élément : `list1.insert(1,45)` # → `list1 = [2,45,10,4,5,33]`
- Suppression d'un élément : `del list1[2]` # → `list1 = [2,45,4,5,33]`
- Accès et suppression d'un élément par index : `a = list1.pop(1)` # → `list1 = [2,4,5,33]` ; `a = 45`
- Inversion de la liste : `list1.reverse()` # → `list1 = [33,5,4,2]`
- Tri d'une liste : `list1.sort(reverse = False)`
- Etendre une liste : `list1.extend([6,7])` / `list1 = list1 + [6,7]` # → `list1 = [33,5,4,2,6,7]`
- Somme de listes : `['A','B','C'] + [1,2,3] + [True,False]` # → `['A','B','C', 1, 2, 3, True, False]`
- Retrait de la première occurrence d'une valeur : `list1.remove(6)` # → `list1 = [2,3,8,6]`
- Dédoublonner les éléments d'une liste : `list(set(['A','B','A','A','C','B']))` # → `['A','B','C']`

CONCEPTS DE BASE

LES LISTES (2/2)

Notion importante : référence dans Python (exemple avec les listes)

```
13 = [1,2,3]
#Affectation
14 = 13
#Modification d'une valeur
14[1] = 99
print(14)
print(13)
```

```
[1, 99, 3]
[1, 99, 3]
```

Dans ce cas, on copie la référence mais
l'objet reste stocké au même endroit
→ 2 références pour un même objet

```
13 = [1,2,3]
#Copie de la valeur
14 = 13.copy()
#Modification d'une valeur
14[1] = 99
print(14)
print(13)
```

```
[1, 99, 3]
[1, 2, 3]
```

Dans ce cas, 14 fait référence à une
nouvelle zone mémoire où les données de
13 sont copiées
→ 2 références pour deux objets différents
qui ont les mêmes valeurs

CONCEPTS DE BASE

LES LISTES : SLICING

```
couleurs = ['bleu', 'blanc', 'rouge', 'gris', 'jaune', 'orange']
```

#Extraire des éléments

```
couleurs[1] # 'blanc'
```

```
couleurs[-3] # 'gris'
```

```
couleurs[:3] # ['bleu', 'blanc', 'rouge']
```

```
couleurs[:-2] # ['bleu', 'blanc', 'rouge', 'gris']
```

```
couleurs[2:4] # ['rouge', 'gris']
```

#Remplacer des éléments

```
couleurs[0] = 'vert'
```

```
couleurs[3:] = ['A', 'B', 'C']
```

```
couleurs # ['vert', 'blanc', 'rouge', 'A', 'B', 'C']
```

#Supprimer des éléments

```
del couleurs[3] # ['vert', 'blanc', 'rouge', 'B', 'C']
```

```
del couleurs[-2:] # ['vert', 'blanc', 'rouge']
```

Inverser une liste

```
nums = [10, 20, 30, 40, 50, 60, 70, 80, 90]
```

```
nums[::-1] # [90, 80, 70, 60, 50, 40, 30, 20, 10]
```

CONCEPTS DE BASE

LES LISTES : OPTIMISATION D'ÉCRITURE

Grâce aux listes nous pouvons simplifier notre code en réduisant le nombre de lignes

➤ Exemple : calculer les carrés de chaque élément d'une suite

```
liste = [1,5,8,12,7]
resultat = []
for v in liste:
    resultat.append(v**2)
```

```
liste = [1,5,8,12,7]
resultat = [v**2 for v in liste]
```

➤ Exemple : Actions conditionnelles

```
liste = [1,5,8,12,7]
resultat = []
for v in liste:
    if (v%2==0):
        resultat.append(v**2)
```

```
liste = [1,5,8,12,7]
resultat = [v**2 for v in liste if (v%2==0)]
```

CONCEPTS DE BASE

LES ENSEMBLES

Un ensemble est une collection non ordonnée d'objets uniques.

Il n'existe pas d'ordre et il est impossible de trouver 2 éléments identiques.

Il s'agit du terme mathématique ensemble.

`d = {1,2,3}`

➤ Il existe les ensembles modifiables (set) et les ensembles non modifiables (frozenset).

```
ensemble = set([1,2,3]) # set permet de transformer une liste en un ensemble
ensemble_gele = frozenset([1,2,3])
```

```
print("Set :",ensemble,type(ensemble))
print("Frozen set :",ensemble_gele,type(ensemble_gele))
```

```
Set : {1, 2, 3} <class 'set'>
Frozen set : frozenset({1, 2, 3}) <class 'frozenset'>
```

➤ C'est exactement la différence entre les listes et les tuples.

CONCEPTS DE BASE

OPÉRATIONS ENSEMBLISTES (1/2)

```
s1 = {1,2,3,3}  
s1
```

```
{1, 2, 3}
```

➔ Dédoublonnage automatique

```
3 in s1
```

```
True
```

➔ Un élément est-il dans l'ensemble ?

```
s2 = {5,4,3}  
s1 - s2
```

```
{1, 2}
```

➔ La différence entre 2 ensembles

```
s1 | s2
```

```
{1, 2, 3, 4, 5}
```

➔ Les éléments dans l'ensemble 1 ou l'ensemble 2

```
s1 & s2
```

```
{3}
```

➔ Les éléments dans l'ensemble 1 et l'ensemble 2

```
s1^s2
```

```
{1, 2, 4, 5}
```

➔ Les éléments dans l'ensemble 1 ou dans l'ensemble 2 mais pas dans les 2 (ou exclusif)

Mêmes traitements sur des listes

```
l1 = [1,2,3,3]  
list(set(l1))
```

```
[1, 2, 3]
```

```
3 in l1
```

```
True
```

```
l2 = [5,4,3]  
list(set(l1) - set(l2))
```

```
[1, 2]
```

```
list(set(l1) | set(l2))
```

```
[1, 2, 3, 4, 5]
```

```
list(set(l1) & set(l2))
```

```
[3]
```

```
list(set(l1) ^ set(l2))
```

```
[1, 2, 4, 5]
```

CONCEPTS DE BASE

OPÉRATIONS ENSEMBLISTES (2/2)

```
s1,s2 = {1,2,3}, {5,4,3}
s1 -= s2
s1
```

{1, 2}

➤ Suppression des éléments en commun avec un autre ensemble

```
s1,s2 = {1,2,3}, {5,4,3}
s1 |= s2
s1
```

{1, 2, 3, 4, 5}

➤ Rajout des éléments nouveaux communs avec un autre ensemble

```
s1,s2 = {1,2,3}, {5,4,3}
s1 &= s2
s1
```

{3}

➤ Élément commun avec un autre ensemble

```
s1,s2 = {1,2,3}, {5,4,3}
s1 ^= s2
s1
```

{1, 2, 4, 5}

➤ Ou exclusif : rajout des éléments du second ensemble en supprimant les communs

CONCEPTS DE BASE

COMPARAISON DES ENSEMBLES (1/2)

```
{1, 2, 3} == {3, 2, 1}
```

True

```
{1, 2, 3} == {3, 2, 1, 4}
```

False

```
{1, 2, 3} != {3, 2, 1}
```

False

```
{1, 2, 3} != {3, 2, 1, 4}
```

True

➤ 2 ensembles égaux sont 2 ensembles avec les mêmes objets.

➤ 2 ensembles sont différents si l'un possède au moins une valeur qui n'est pas présente dans l'autre.

CONCEPTS DE BASE

COMPARAISON DES ENSEMBLES (2/2)

```
S1, S2, S3 = {1, 2, 3}, {1}, {5, 4, 3}
print ( S2 < S1 , " et " , S1 > S2 )
```

True et True

```
S2 < S3
```

False

```
print ( S1 >= S2 , " et " , S1 >= S1 )
```

True et True

```
print ( S2 <= S1 , " et " , S1 <= S1 )
```

True et True

```
print ( S1 < S3 , " et " , S1 > S3 )
```

False et False

➤ Un ensemble inférieur à un autre est un ensemble qui est contenu par lui.

➤ Un ensemble supérieur à un ensemble est un ensemble qui le contient.

➤ Un ensemble supérieur ou égal à un autre est un ensemble qui le contient ou qui lui est identique.

➤ Un ensemble inférieur ou égal à un autre est un ensemble qui est contenu par lui ou qui lui est identique.

➤ Un ensemble supérieur ou égal à un autre est un ensemble qui le contient ou qui lui est identique.

CONCEPTS DE BASE

LES DICTIONNAIRES (1/3)

➤ Un dictionnaire est une collection non ordonnée de relations entre clés et valeurs.

```
d1 = {'nom': 'Raoult', 'prénom': 'Didier', 'age': 68}
```

↓ ↓
Clé (key) Valeur (value)

```
print("Clé :", d1.keys())  
print("Valeur :", d1.values())  
print("Item :", d1.items())
```

```
Clé : dict_keys(['nom', 'prénom', 'age'])  
Valeur : dict_values(['Raoult', 'Didier', 68])  
Item : dict_items([('nom', 'Raoult'), ('prénom', 'Didier'), ('age', 68)])
```

➤ Les clés ne sont pas forcément des chaînes de caractères. L'outil est très souple mais, attention, autant de liberté peut être préjudiciable. Il faut être très rigoureux.

```
d2 = {('Pierre', 56): ['Directeur', 1253, True], ('Paul', 44): ['Employé', 100, False]}  
print(d2.keys())  
print(d2.values())
```

```
dict_keys([('Pierre', 56), ('Paul', 44)])  
dict_values([['Directeur', 1253, True], ['Employé', 100, False]])
```

CONCEPTS DE BASE

LES DICTIONNAIRES (2/3)

```
# Création d'un dictionnaire
dic = {'Jean' : 17, 'Paul' : 20, 'Léo' : 16}
print(dic) # ou print(dic.items())

print(len(dic)) # 3

print(dic.keys()) # ['Léo', 'Jean', 'Paul']

print(dic.values()) # [16, 17, 20]

print(dic['Jean']) # 17
# ou print(dic.get('Jean'))

print(dic['Charles']) # Erreur (Charles n'est pas une clé)

#Copier le dictionnaire :
dic2 = dic.copy()

#Vider le dictionnaire :
dic.clear()
print(dic) # {}
```

CONCEPTS DE BASE

LES DICTIONNAIRES (3/3)

```
dic = dic2.copy()
del dic2
#Modification :
dic['Léo'] = 23
print(dic) # {'Léo': 23, 'Jean': 17, 'Paul': 20}

#Ajout d'un élément
dic['Max'] = 45
print(dic) # {'Max': 45, 'Léo': 23, 'Jean': 17, 'Paul': 20}

# Ajout d'un bloc d'élément
dic.update({'Claire':43,'Caroline':39})
print(dic) # {'Caroline': 39, 'Jean': 17, 'Claire': 43, 'Paul': 20, 'Max': 45, 'Léo': 23}

#Détecter la présence d'une clé
test = 'Léo' in dic
print(test) # True

#Suppression d'une clé
del dic['Léo']
print(dic) # {'Caroline': 39, 'Jean': 17, 'Claire': 43, 'Paul': 20, 'Max': 45}
```

CONCEPTS DE BASE

LES VECTEURS (1/6)

- La librairie **numpy** est la référence pour la création et la manipulation de vecteurs et de matrices
- Les vecteurs et les matrices dans numpy s'appellent **array**
- Array est un **object multidimensionnel** qui contient des objects du même type
- Les array sont **indexés** par des **tuples**
- Les lignes sont appelées des **axes** et le nombre d'axes **rank**

```
[[1, 0, 0],  
 [0, 1, 2]]
```

L'array a un rang de 2 (2 lignes)
La dimension du premier axe est 2 (2 lignes)
La dimension du second axe est 3 (3 colonnes)

CONCEPTS DE BASE

LES VECTEURS (2/6)

```
import numpy as np # Import du package
```

```
#Création d'une matrice 3x5 avec des entiers de 0 à 14
```

```
a = np.arange(15).reshape(3,5)
```

```
[[ 0  1  2  3  4]
 [ 5  6  7  8  9]
 [10 11 12 13 14]]
```

```
#Obtenir les dimensions de l'array
```

```
a.shape # (3, 5)
```

```
#Obtenir le rang de la matrice
```

```
a.ndim # 2
```

```
#Le type des éléments présents dans l'array
```

```
a.dtype.name # 'int32'
```

```
#Nombre d'éléments dans l'array
```

```
a.size # 15
```

```
#Création d'un array à l'aide de listes
```

```
b = np.array([[2,3,4],[6,7,8]])
```

```
array([[2, 3, 4],
       [6, 7, 8]])
```

CONCEPTS DE BASE

LES VECTEURS (3/6)

```
#Création d'une matrice 3x4 de 0  
np.zeros((3,4))
```

```
array([[ 0.,  0.,  0.,  0.],  
       [ 0.,  0.,  0.,  0.],  
       [ 0.,  0.,  0.,  0.]])
```

```
#Création d'une matrice avec 2 matrices 3x4 de 1  
a = np.ones((2,3,4))
```

```
array([[[ 1.,  1.,  1.,  1.],  
       [ 1.,  1.,  1.,  1.],  
       [ 1.,  1.,  1.,  1.]],  
      [[ 1.,  1.,  1.,  1.],  
       [ 1.,  1.,  1.,  1.],  
       [ 1.,  1.,  1.,  1.]])
```

```
a[1,2,3] = 50*a[1,2,3] # Modification d'un élément  
(les indices commencent à 0)
```

```
array([[[ 1.,  1.,  1.,  1.],  
       [ 1.,  1.,  1.,  1.],  
       [ 1.,  1.,  1.,  1.]],  
      [[ 1.,  1.,  1.,  1.],  
       [ 1.,  1.,  1.,  1.],  
       [ 1.,  1.,  1.,  50.]])
```


CONCEPTS DE BASE

LES VECTEURS (4/6)

Fonctions références :

```
'''Techniques de création de vecteur'''  
np.arange(10,30,5) #--> array([10, 15, 20, 25])  
np.arange(0,2,0.3) #--> array([ 0. ,  0.3,  0.6,  0.9,  1.2,  1.5,  1.8])  
# 9 valeurs equidistantes de 0 à 2  
np.linspace(0,2,9) # --> array([ 0., 0.25, 0.5, 0.75, 1., 1.25, 1.5 , 1.75, 2.])
```

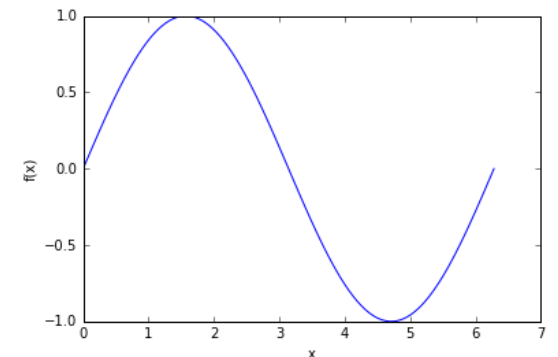
```
'''Appliquer une fonction sur un vecteur'''
```

```
from numpy import pi  
x = np.linspace(0,2*pi,100) # --> 100 points entre 0 et 2*pi  
f = np.sin(x)                # On applique la fonction sin(x) sur les 100 points de x
```

```
'''Tracer un graphique rapidement'''
```

```
import matplotlib.pyplot as plt # Package pour créer des graphiques  
plt.plot(x,f)  
plt.xlabel('x')  
plt.ylabel('f(x)')  
plt.show
```

Schéma
classique
pour la
création de
graphique



CONCEPTS DE BASE

LES VECTEURS (5/6)

Fonctions références pour les opérations:

```
'''Création de vecteurs'''
a = np.array( [20,30,40,50] )
b = np.arange( 4 ) # --> array([0, 1, 2, 3])

'''Opérations entre vecteurs'''
c = a-b #-> array([20, 29, 38, 47])
b**2 # -->array([0, 1, 4, 9])
10*np.sin(a) # --> array([ 9.12945251, -9.88031624,  7.4511316 , -2.62374854])
a<35 #-->array([ True,  True, False, False], dtype=bool)
a*b # --> array([  0,  30,  80, 150])

'''Création de matrices'''
A = np.array( [[1,1], [0,1]] )
B = np.array( [[2,0], [3,4]] )

'''Opération sur les matrices'''
A*B # Produit élément par élément
#array([[2, 0],
#       [0, 4]])

A.dot(B) # Produit matriciel
#array([[5, 4],
#       [3, 4]])
np.dot(A, B) # Autre manière d'écrire le produit matriciel
```

CONCEPTS DE BASE

LES VECTEURS (6/6)

Fonctions références :

```
a = np.random.random((2,3))
#array([[ 0.78492393,  0.93619717,  0.28750659],
#       [ 0.04839579,  0.08941263,  0.96451262]])

'''Somme des éléments'''
a.sum() # --> 3.1109487312709376

'''Min et max des éléments'''
a.min() #--> 0.048395787392701006
a.max() # --> 0.9645126204304354

'''Moyenne médiane quantiles'''
a.mean() # --> 0.51849145521182294
np.median(a) #--> 0.53621526026497179
np.nanpercentile(a, q = 10) # -->
0.068904209822096441
```

Pour les matrices les fonctions sont les mêmes,
il faut juste préciser l'axe en paramètre

```
'''Exemple avec une matrice'''
A = np.array( [[1,4], [0,1]] )
#array([[1, 4],
#       [0, 1]])
A.mean(axis = 0) # axis = 0 --> les
colonnes
#array([ 0.5,  2.5])
A.mean(axis = 1) # axis = 1 --> les lignes
#array([ 2.5,  0.5])
A.mean() # Pas d'axis --> l'ensemble de la
matrice
#1.5
```

CONCEPTS DE BASE

SYNTHÈSE DES PRINCIPAUX TYPES DE VARIABLES

Type	Définition	Exemple
<code>int</code>	Entier. Réservé à une variable dont la valeur est un entier relatif stocké en valeur exacte.	434
<code>float</code>	Un nombre à virgule flottant correspondant à une variable dont la valeur est un nombre réel. Les nombres réels sont stockés selon le format « double précision » (norme IEEE 754).	10.2
<code>str</code>	Chaine de caractère.	'Formation Python'
<code>list</code>	Les listes sont des objets séquentiables modifiables.	[1,2,3, 'je suis une liste',4]
<code>tuple</code>	Les t-uplets sont des objets séquentiels non modifiables.	(1,2,3, 'je suis un tuple',4)
<code>dict</code>	Les dictionnaires sont des tableaux associatifs ; i.e. des collections modifiables d'objets indexés.	{'Jean' : 46, 'Max' : 25, 'mylist': 1,'toto'}}
<code>set</code>	Les ensembles sont des ensembles d'éléments uniques.	{2,4, 'Hello', (1,2,3)}
<code>bool</code>	Booléen (vrai ou faux).	True / False
<code>array</code>	Les vecteurs	<code>np.array([20,30,40,50])</code>

CONCEPTS DE BASE

PROGRAMMATION MODULAIRE

Pourquoi créer des fonctions ?

1. Meilleure organisation du programme → Gain en lisibilité
2. Eviter la redondance dans le code → Pas de copier/coller
3. Possibilité de partager les fonctions
4. **L'objectif majeur** : le programme principal doit être le plus simple possible

Qu'est ce qu'un module Python ?

1. Module = fichier « .py »
2. On peut regrouper dans un module les fonctions traitant des problèmes de même nature ou manipulant le même type d'objet
3. Pour charger les fonctions d'un module dans un autre module ou dans le programme principal on utilise la commande `import nom_module`
4. Les fonctions sont chargées en mémoire.

→ **Attention** si dans deux modules différents deux fonctions portent le même nom, la fonction appelée le plus récemment est en mémoire mais la plus ancienne n'est plus conservée en mémoire

CONCEPTS DE BASE

LES FONCTIONS (1/2)

Définir une fonction :

```
def nom_fonction(p1,p2,...) :  
    bloc d'instructions  
    return (s1,s2,...)
```

Toujours le même schéma :

1. **def** pour dire que l'on définit une fonction
2. Le nom de la fonction
3. Les paramètres
4. Toujours les :
5. L'indentation pour mentionner que le bloc d'instructions est celui de la fonction
6. **return** pour retourner les valeurs désirées
7. Si une fonction n'a pas de **return** on parlera alors de procédure

CONCEPTS DE BASE

LES FONCTIONS (2/2)

Exemple :

Si a est le max retourne a sinon retourne 100

```
def maxi(a,b) :  
    if (a>b):  
        e = a  
    else :  
        e = 100  
    return(e)
```

c = 3

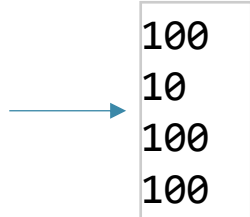
d = 10

print(maxi(c,d))

print(maxi(d,c))

print(maxi(a=c,b=d))

print(maxi(b=d,a=c))



```
100  
10  
100  
100
```

- L'ordre des paramètres est important ou alors il faut les mentionner
- Pour éviter les confusions quand les paramètres sont nombreux il est conseillé d'écrire le nom des paramètres (cf les derniers appels)

CONCEPTS DE BASE

LES FONCTIONS : PARAMÉTRAGE PAR DÉFAUT

Paramètres par défaut :

- Affecter une valeur à un paramètre dès la définition de la fonction
 - Si lors de l'appel de la fonction l'utilisateur ne précise pas de valeur pour le paramètre alors la valeur par défaut est utilisée
 - Si l'utilisateur spécifie une valeur pour le paramètre lors de l'appel de la fonction c'est cette valeur qui est prise en compte
 - Les paramètres avec des valeurs par défaut sont regroupés en dernière positions lors de la définition de la fonction
- Exemple :

#On calcule un prix

```
def f(prix,solde = False, red = 50):  
    if solde == True :  
        prix = prix - prix * red/100  
    return(prix)
```

```
print(f(100))  
print(f(100,True))  
print(f(100,True,75))
```

→

100
50
25

CONCEPTS DE BASE

LES FONCTIONS : MUTABLE / NON MUTABLE

Attention à l'utilisation d'objets mutables ou non mutables :

➤ Les paramètres sont toujours lus comme des références (référence à l'objet appelé), mais ils sont modifiables seulement s'ils sont mutables (dictionnaires, listes, ...) à l'inverse des non mutables (type simples, tuples,...)

➤ Exemple :

```
# Exemple avec des éléments simples
def non_mutable(x,y):
    x = x+3
    y = y-2
    print(x,y)

x = 4
y = 10
non_mutable(x,y)      # 7 8
print(x,y)             # 4 10
```

```
# Exemple avec des listes :
def mutable(l1,l2):
    l1.append(4)
    l2[0]= 0
    print(l1,l2)

l1 = [1,2]
l2 = [2,2,3]
mutable(l1,l2)         # [1,2,4] [0,2,3]
print(l1,l2)           # [1,2,4] [0,2,3]
```

CONCEPTS DE BASE

LES FONCTIONS : RETOURNER PLUSIEURS VALEURS

Une fonction peut retourner plusieurs valeurs. La récupération des résultats passe par une affectation multiple (tuple, liste, dictionnaire)

```
def ordre(a,b) :  
    if (a>b):  
        return a,b  
    else :  
        return b,a  
  
a= 10  
b= 5  
vmax , vmin = ordre(a,b)  
print(vmax,vmin)           # 10 5  
print(ordre(a,b))          # (10,5)  
print(type(ordre(a,b)))    # <class 'tuple'>
```

```
def ordre_dic(a,b):  
    if a > b :  
        return( {'Max' : a, 'Min' : b})  
    else :  
        return({'Max' : b , 'Min' : a})  
  
a = 10  
b = 5  
res = ordre_dic(a,b)  
print(res)                  #{'Max': 10, 'Min': 5}  
print(res['Max'])           #10  
print(type(res))            #<class 'dict'>
```

```
def ordre_liste(a,b) :  
    if (a>b):  
        return [a,b]  
    else :  
        return [b,a]  
  
a= 10  
b= 5  
res = ordre_liste(a,b)  
print(res)                  # [10,5]  
print(res[0])               # 10  
print(type(res))            # <class 'list'>
```

- Les 3 fonctions font le même travail
- Mais les sorties sont différentes
- Et les appels aux valeurs aussi

CONCEPTS DE BASE

LES FONCTIONS : VARIABLES LOCALES ET GLOBALES

Les variables définies à l'intérieur de la fonction sont uniquement visibles dans ces fonctions, on parle de variables locales

Les variables définies hors de la fonction ne sont pas accessibles dans la fonction, on parle alors de variables globales (**SAUF** si on utilise le mot `global`)

➤ `x` est une variable globale

➤ `v` est une variable locale

```
def remplace(v):  
    x = v  
    return(x)  
  
x = 10  
print(remplace(5))    #5  
print(x)              #10
```

```
def remplace_3(v):  
    global x  
    x = x + v  
    return(x)  
  
x = 10  
print(remplace_3(5))  #15  
print(x)              #15
```

Dans ce cas là, la fonction n'a même pas tourné

```
def remplace_2(v):  
    x = x + v  
    return(x)  
  
x = 10  
print(remplace_2(5))  #UnboundLocalError: local variable 'x'  
                      #referenced before assignment  
print(x)              #10
```

CONCEPTS DE BASE

LES FONCTIONS : FONCTIONS LOCALES ET GLOBALES

Même raisonnement que précédemment, il existe des fonctions :

- Globales qui sont valables sur l'ensemble du programme
- Locales qui sont définies dans une fonction et qui ne sont valables que dans cette fonction

➤ Exemple :

```
#On définit la fonction globale
def glob(a):

    # On definit la fonction locale
    def loc(b):
        return(2*b)

    return(3*loc(a))

x = 10
print(glob(x)) #60
print(loc(x)) # NameError : name 'loc' is not defined
```

| CONCEPTS DE BASE

LES FONCTIONS LAMBDA

Une fonction peut être simple à écrire et ne nécessitant pas de la déclarer dans une variable.

Il est alors possible d'écrire une fonction anonyme à l'aide de lambda.

- La fonction n'existe plus après son utilisation.
- Elle n'est pas visible en mémoire.

La fonction **map** applique une fonction à une liste.

```
list(map(lambda x:x**2,range(10)))
```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

CONCEPTS DE BASE

GESTION DES EXCEPTIONS

- Une exception est une erreur détectée durant l'exécution d'une instruction.
- Par exemple, ouvrir un fichier qui n'existe pas génère une erreur.

```
lecture = open("C:/test.xls", 'r')
```

```
-----  
FileNotFoundError                                Traceback (most recent call last)  
<ipython-input-203-618c14e959d9> in <module>()  
----> 1 lecture = open("C:/test.xls", 'r')  
  
FileNotFoundError: [Errno 2] No such file or directory: 'C:/test.xls'
```

CONCEPTS DE BASE

GESTION DES EXCEPTIONS

➤ Il existe de nombreuses exceptions en Python.

➤ Les principales sont :

Exception	Cause
<code>NameError</code>	Utilisation d'un nom (variable, fonction, classe, etc...) qui n'existe pas
<code>IndexError</code>	Tentative d'accéder à une partie indexable (souvent d'une liste ou d'un tuple) qui n'existe pas
<code>KeyError</code>	Tentative d'accéder à une clé d'un mapping (souvent d'un dictionnaire) qui n'existe pas
<code>TypeError</code>	Tentative d'opération incompatible avec ce type
<code>ZeroDivisionError</code>	Division par 0 impossible
<code>MemoryError</code>	Utilisation de trop de mémoire

➤ La liste complète est disponible sur : <https://docs.python.org/fr/3.7/library/exceptions.html>

CONCEPTS DE BASE

LEVÉE D'UNE EXCEPTION

➤ Il est possible de créer ses propres exceptions à l'aide du mot-clé « raise » :

```
def sominf10(a,b):  
    if a+b > 10: raise ValueError("Somme supérieure strictement à 10")  
    else: return(a+b)  
  
sominf10(5,6)
```

```
-----  
ValueError                                Traceback (most recent call last)  
<ipython-input-205-7b91570ffb65> in <module>()  
      3     else: return(a+b)  
      4  
----> 5 sominf10(5,6)  
  
<ipython-input-205-7b91570ffb65> in sominf10(a, b)  
      1 def sominf10(a,b):  
----> 2     if a+b > 10: raise ValueError("Somme supérieure strictement à 10")  
      3     else: return(a+b)  
      4  
      5 sominf10(5,6)  
  
ValueError: Somme supérieure strictement à 10
```


CONCEPTS DE BASE

TRAITEMENT PERSONNALISÉ DES EXCEPTIONS

- Il est possible de capturer différents types d'exceptions et d'effectuer un traitement d'erreur personnalisé pour chaque exception et un pour le cas où il n'est pas capturé.

```
def division(a,b):  
    try:  
        return (a/b)  
    except TypeError: print("Vous n'avez pas rentré des nombres")  
    except: print("Autres erreurs")  
  
division(2,'1')
```

Vous n'avez pas rentré des nombres

```
division (2,0)
```

Autres erreurs

- « try » teste un bloc d'instruction.
- Si une erreur est rencontrée, Python passe aux « except » selon l'erreur produite.
- « try » doit toujours être suivi d'au moins un « except ».

CONCEPTS DE BASE

LES MODULES (OU PACKAGES)

- Un module est un fichier « .py » contenant un ensemble de variables, fonctions et classes que l'on peut importer et utiliser dans le programme principal ou dans d'autres modules.
- Le mot clé pour charger un module : `import`
- C'est un pas supplémentaire vers la modularité : un module maximise la réutilisation et facilite le partage du code
- Des modules standards sont prêts à l'emploi comme `math`, `random`, `os` ...

➤ Exemple :

```
import math, random
# Un nombre aléatoire entre 0 et 1
print(random.random())      # 0.07411960856255406
print(math.log(2))          # 0.6931471805599453
```

- On peut utiliser des alias pour les noms de packages :

```
import math as m
import random as r
print(r.random())           # 0.9557576489937392
print(m.log(2))             # 0.6931471805599453
```

| CONCEPTS DE BASE

LES MODULES (OU PACKAGES)

On peut importer seulement quelques fonctions d'un package :

```
from math import log
from random import random
print(random())      # 0.7510624775356199
print(log(2))        # 0.6931471805599453
```

Ou importer l'ensemble des fonctions d'un package :

```
from math import *
print(log(2))        # 0.6931471805599453
```

Attention : dans ces deux derniers cas, on ne spécifie plus le package lors de l'appel de la fonction. Si deux packages ont des fonctions avec le même nom ça peut être problématique
→ La fonction importée le plus récemment sera appelée.

CONCEPTS DE BASE

CRÉATION D'UN PACKAGE

On crée le fichier « tva.py » :

```
#TVA 10%
def pttc_reduit(p):
    '''Retourne le prix d'un produit toutes charges comprises
    avec une TVA réduite '''
    return(p*1.1)

#TVA 20%
def pttc_normal(p):
    '''Retourne le prix d'un produit toutes charges comprises
    avec la TVA classique '''
    return(p*1.2)
```

Remarque : important de bien annoter son script pour les prochains utilisateurs du code

CONCEPTS DE BASE

IMPORT D'UN PACKAGE CRÉÉ

```
#On se place dans le répertoire où se trouve le package
import os
os.chdir(r'C:\Users\Documents\Slides de formation\Python')

# On importe le package
import tva
help(tva)
#Help on module tva:
#
#NAME
#   tva - # -*- coding: utf-8 -*-
#
#FUNCTIONS
#   pptc_normal(p)
#       Prix toutes charges comprises avec la TVA classique
#
#   pptc_reduit(p)
#       Retourne le prix d'un produit toutes charges comprises
#       avec une TVA réduite
#
#FILE
#   c:\users\documents\slides de formation\python\tva.py

#Utilisation d'une fonction du package importé
print(tva.pptc_normal(100)) #120
```

Plus
d'information
slide suivante

Informations
sur le
package

Utilisation
d'une fonction

CONCEPTS DE BASE

INTÉGRER DES NOUVEAUX PACKAGES

Pour importer des packages plusieurs options :

- Soit le package est dans les dossier listés dans PYTHONPATH (les packages classiques)
 - `import nom_package`
- Soit vous avez sauvegardé votre package dans un répertoire spécifique, il faut alors que le répertoire courant de travail soit celui où est situé le package :
 - Pour connaître le répertoire courant :

```
> import os
> os.getcwd()
```
 - Pour choisir son répertoire de travail :

```
> os.chdir(chemin_repertoire)
```

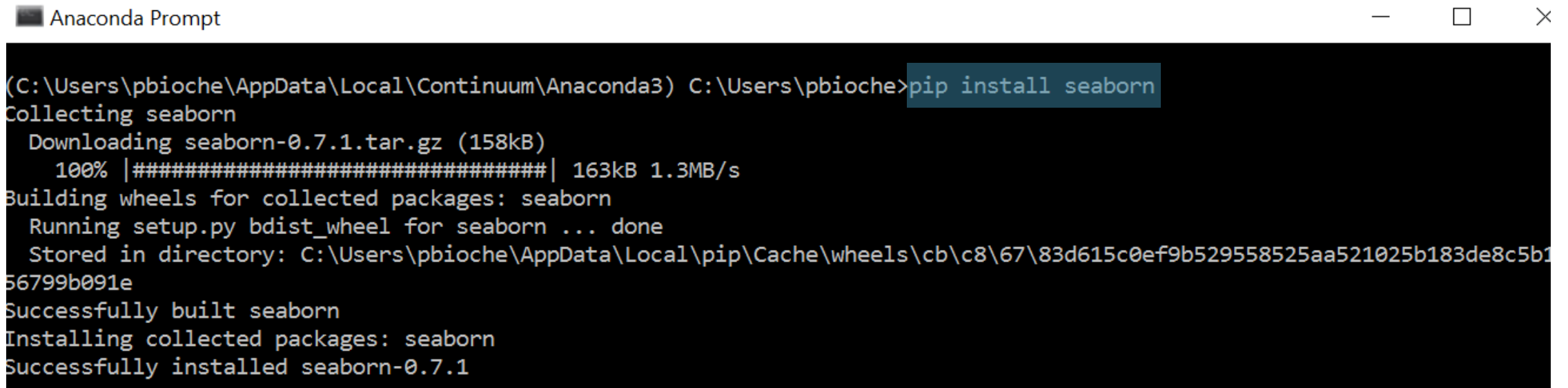
Pour installer un package proposé sur Internet, 3 options dans le prompt anaconda :

- `conda install nom_package` (le package doit être connu par Anaconda)
- `pip install nom_package` (La technique la plus répandue)
- Télécharger le package puis se placer dans le répertoire du package (`cd chemin\package`) puis installer le package (`setup.py`) (L'ancienne technique)

CONCEPTS DE BASE

EXEMPLE D'INSTALLATION DE PACKAGE

Exemple installation package seaborn :



```
Anaconda Prompt
(C:\Users\pbioche\AppData\Local\Continuum\Anaconda3) C:\Users\pbioche>pip install seaborn
Collecting seaborn
  Downloading seaborn-0.7.1.tar.gz (158kB)
    100% |#####| 163kB 1.3MB/s
Building wheels for collected packages: seaborn
  Running setup.py bdist_wheel for seaborn ... done
  Stored in directory: C:\Users\pbioche\AppData\Local\pip\Cache\wheels\cb\c8\67\83d615c0ef9b529558525aa521025b183de8c5b156799b091e
Successfully built seaborn
Installing collected packages: seaborn
Successfully installed seaborn-0.7.1
```

01

Les principaux environnements de développement Python

02

Les concepts de base en Python

03

Introduction aux packages NUMPY et SCIPY

04

Import et préparation des données avec Pandas

05

Statistiques descriptives

06

Graphiques avec Matplotlib

INTRODUCTION AUX PACKAGES NUMPY ET SCIPY

NUMPY : INTRODUCTION



- Le module NumPy est la référence pour le calcul scientifique.
- Il permet l'intégration de code C, C++ et Fortran.
- Il permet la création et la manipulation
 - De vecteurs multidimensionnels ;
 - De matrices ;
 - Des routines pour des applications rapides sur des tableaux :
 - Manipulation mathématique, logique, de forme,
 - Le tri,
 - La sélection,
 - Les entrées / sorties,
 - Les transformations de Fourier,
 - L'algèbre linéaire de base,
 - Les opérations statistiques de bases,
 - Les simulations aléatoires,
 - ...

INTRODUCTION AUX PACKAGES NUMPY ET SCIPY

NUMPY : LES ARRAYS (1/3)

L'objet core de NumPy est ndarray

```
import numpy as np
```

➤ Nous importons le module NumPy.

```
tab = np.array([2,5,8])  
type(tab)
```

➤ Nous créons un tableau.

```
numpy.ndarray
```

➤ Le type est ndarray.

Attention, les crochets sont obligatoires.

➤ Un tableau est un ensemble d'éléments de **même type**

➤ Le tableau est indexé par un tuple d'entiers positifs

```
print('La dimension du tableau est :', tab.ndim)  
print('La forme du tableau est :', tab.shape)  
print('Le type de données enregistrées est', tab.dtype)  
print("Le nombre d'éléments du tableau est", tab.size)  
print("Le buffer des données du tableau est", tab.data)
```

La dimension du tableau est : 1

La forme du tableau est : (3,)

Le type de données enregistrées est int32

Le nombre d'éléments du tableau est 3

Le buffer des données du tableau est <memory at 0x000002682D8B4108>

INTRODUCTION AUX PACKAGES NUMPY ET SCIPY

NUMPY : LES ARRAYS (2/3)

➤ Pour créer un ndarray de plusieurs dimensions, il faut alors insérer une liste de listes.

```
tab2 = np.array([[5,0,8,10],[12,-6,9,56]])  
print(tab2)
```

```
[[ 5  0  8 10]  
 [12 -6  9 56]]
```

```
tab2.shape
```

```
(2, 4)
```

➤ Il est possible de spécifier le type de données.

```
tab3 = np.array([[5,0,8,10],[12,-6,9,56]],  
                 dtype=complex)  
print(tab3)
```

```
[[ 5.+0.j  0.+0.j  8.+0.j 10.+0.j]  
 [12.+0.j -6.+0.j  9.+0.j 56.+0.j]]
```

INTRODUCTION AUX PACKAGES NUMPY ET SCIPY

NUMPY : LES ARRAYS (3/3)

➤ Nous pouvons aussi créer une liste de valeurs en utilisant la fonction `np.arange`

➤ Nous allons de 0 à 10 (exclus) par pas de 2.

```
tab = np.arange(0,10,2)
print(tab)
```

```
[0 2 4 6 8]
```

➤ Nous pouvons ensuite demande de reformater le ndarray en plusieurs dimensions.

➤ Nous souhaitons créer un ndarray avec 3 lignes et 2 colonnes.

```
tab = np.arange(0,12,2).reshape(3,2)
print(tab);print(tab.shape)
```

```
[[ 0  2]
 [ 4  6]
 [ 8 10]]
(3, 2)
```

INTRODUCTION AUX PACKAGES NUMPY ET SCIPY

NUMPY : LES OPÉRATIONS

➤ Somme de 2 ndarrays :

➤ Élément par élément.

```
print(A+B)
```

```
[[ 0.          4.14159265  8.28318531 12.42477796]
 [16.56637061 20.70796327 24.84955592 28.99114858]
 [33.13274123 37.27433388 41.41592654 45.55751919]]
```

➤ Soustraction de 2 ndarrays :

➤ Élément par élément.

```
print(B-A)
```

```
[[ 0.          2.14159265  4.28318531  6.42477796]
 [ 8.56637061 10.70796327 12.84955592 14.99114858]
 [17.13274123 19.27433388 21.41592654 23.55751919]]
```

➤ Multiplication de 2 ndarrays :

➤ Toujours élément par élément.

```
print(A*B)
```

```
[[ 0.          3.14159265 12.56637061 28.27433388]
 [50.26548246 78.53981634 113.09733553 153.93804003]
 [201.06192983 254.46900494 314.15926536 380.13271108]]
```

➤ Le produit matriciel de 2 ndarrays :

➤ @ est valable depuis Python 3.5.

➤ La méthode A.dot(B) donne le même résultat.

➤ Notez l'appel à l'attribut np.pi, i.e. le nombre π .

```
A = np.arange(0,12,1).reshape(3,4)
B = np.arange(0,12*np.pi,np.pi).reshape(4,3)
print(A@B)
```

```
[[131.94689145 150.79644737 169.64600329]
 [358.14156251 427.25660089 496.37163927]
 [584.33623357 703.7167544  823.09727524]]
```

INTRODUCTION AUX PACKAGES NUMPY ET SCIPY

NUMPY : FONCTIONS STATISTIQUES SUR L'ENSEMBLE DES DONNÉES

- Il est possible de réaliser des statistiques pour l'ensemble des données du ndarray
- Construire un ndarray rempli d'éléments tirés aléatoirement peut se faire avec la méthode `random.random`

```
A = np.random.random((2,3))
```

```
[[0.38419078 0.09240464 0.10389585]  
 [0.60182686 0.87034622 0.7452316 ]]
```

méthode	Statistiques	Exemple
<code>A.sum()</code>	Somme des éléments	<code>A.sum()</code> = 2.7978959624132083
<code>A.min()</code>	Minimum des éléments	<code>A.min()</code> = 0.09240464124769654
<code>A.mean()</code>	Moyenne des éléments	<code>A.mean()</code> = 0.46631599373553473
<code>A.max()</code>	Maximum des éléments	<code>A.max()</code> = 0.8703462249587804
<code>A.var()</code>	Variance des éléments	<code>A.var()</code> = 0.0895500245999218
<code>A.std()</code>	Ecart-type des éléments	<code>A.std()</code> = 0.29924910125165255

INTRODUCTION AUX PACKAGES NUMPY ET SCIPY

NUMPY : FONCTIONS STATISTIQUES SUR LES COLONNES

➤ Il est possible de réaliser des statistiques pour les **colonnes** du ndarray en précisant **axis = 0**.

```
A = np.random.random((2,3))
```

```
[[0.38419078 0.09240464 0.10389585]  
 [0.60182686 0.87034622 0.7452316 ]]
```

méthode	Statistiques	Exemple
A.sum(axis=0)	Somme de chaque colonne	[0.98601764, 0.96275087, 0.84912745]
A.min(axis=0)	Minimum de chaque colonne	[0.38419078, 0.09240464, 0.10389585]
A.mean(axis=0)	Moyenne de chaque colonne	[0.49300882, 0.48137543, 0.42456373]
A.max(axis=0)	Maximum de chaque colonne	[0.60182686, 0.87034622, 0.7452316]
A.var(axis=0)	Variance de chaque colonne	[0.01184137, 0.15129828, 0.10282789]
A.std(axis=0)	Ecart-type de chaque colonne	[0.10881804, 0.38897079, 0.32066788]

INTRODUCTION AUX PACKAGES NUMPY ET SCIPY

NUMPY : FONCTIONS STATISTIQUES SUR LES LIGNES

➤ Il est possible de réaliser des statistiques pour les **lignes** du ndarray en précisant **axis = 1**.

```
A = np.random.random((2,3))
```

```
[[0.38419078 0.09240464 0.10389585]  
 [0.60182686 0.87034622 0.7452316 ]]
```

méthode	Statistiques	Exemple
A.sum(axis=1)	Somme de chaque ligne	[0.58049128, 2.21740469]
A.min(axis=1)	Minimum de chaque ligne	[0.09240464, 0.60182686]
A.mean(axis=1)	Moyenne de chaque ligne	[0.19349709, 0.7391349]]
A.max(axis=1)	Maximum de chaque ligne	[0.38419078, 0.87034622]
A.var(axis=1)	Variance de chaque ligne	[0.01820405, 0.01203569]
A.std(axis=1)	Ecart-type de chaque ligne	[0.13492238, 0.10970731]

INTRODUCTION AUX PACKAGES NUMPY ET SCIPY

NUMPY : FONCTIONS MATHÉMATIQUES

➤ NumPy dispose d'un grand nombre de fonctions mathématiques.

```
A = np.random.random((1,3))
```

```
[[0.73371681 0.12216107 0.36223865]]
```

méthode	Statistiques	Exemple
<code>np.exp(A)</code>	Exponentiel des éléments de A	<code>[[2.08280764, 1.12993608, 1.43654173]]</code>
<code>np.sqrt(A)</code>	Racine carrée des éléments de A	<code>[[0.85657271, 0.34951548, 0.60186265]]</code>
<code>np.sin(A)</code>	Sinus des éléments de A	<code>[[0.66963469, 0.12185745, 0.35436849]]</code>
<code>np.cos(A)</code>	Cosinus des éléments de A	<code>[[0.74269063, 0.99254761, 0.93510586]]</code>
<code>np.ceil(A)</code>	Arrondi des éléments de A	<code>[[1., 1., 1.]]</code>

INTRODUCTION AUX PACKAGES NUMPY ET SCIPY

NUMPY : LES CONSTANTES

➤ NumPy dispose de plusieurs constantes

Constantes	Valeur
<code>np.inf</code> , <code>np.Inf</code> , <code>np.Infinity</code> , <code>np.PINF</code> , <code>np.infty</code>	$+\infty$
<code>np.nan</code> , <code>np.NaN</code> , <code>np.NAN</code>	Pas un nombre
<code>np.NINF</code>	$-\infty$
<code>np.NZERO</code>	-0
<code>np.PZERO</code>	$+0$
<code>np.e</code>	Constante d'Euler : $e = 2,7182818 \dots$
<code>np.euler_gamma</code>	Constant d'Euler-Mascheroni : $\gamma = 0,57721 \dots$
<code>np.newaxis</code>	Nouvelle axe dans un array. L'axe est à vide
<code>np.pi</code>	$\pi = 3,141592654$

INTRODUCTION AUX PACKAGES NUMPY ET SCIPY

NUMPY : MANIPULATION DES OBJETS NUMPY

➤ Tout comme les listes en Python, il est possible de manipuler un objet NumPy.

```
import numpy as np
a = np.arange(10)**3
print("Le tableau étudié est :", a, "\n")

#Index
print("Le troisième élément de ce tableau est :", a[2], "\n")

#Slicing
print("Les éléments entre le 3è et le 5è sont ", a[2:5])
a[:6:2] = 1000
print("Nous avons modifié les éléments 0 à 6 par pas de 2 :", a,
"\n")

#Itération
print("Nous itérons sur le tableau créé :")
for i in a: print(i,"puissance 3 vaut ", i**3)
```

➤ Nous avons créé le tableau $\begin{bmatrix} 0 & 1 \\ 8 & 27 & 64 & 125 & 216 & 343 & 512 & 729 \end{bmatrix}$.

➤ Nous pouvons récupérer le 3^e élément.

➤ Nous récupérons les éléments 3 à 5 par slicing. Il est aussi possible de modifier des éléments du tableau.

➤ Il est possible d'itérer sur les éléments d'un tableau.

INTRODUCTION AUX PACKAGES NUMPY ET SCIPY

NUMPY : LES TABLEAUX MULTI-DIMENSION

➤ Les tableaux à multi-dimension peuvent logiquement avoir un indice par axe.

➤ Les indices sont alors donnés à l'aide d'un tuple.

```
def f(x,y) : return 10*x+y  
b = np.fromfunction(f,(5,4), dtype=int)  
print(b)
```

```
[[ 0  1  2  3]  
 [10 11 12 13]  
 [20 21 22 23]  
 [30 31 32 33]  
 [40 41 42 43]]
```

```
print(b[0:5,1])
```

```
[ 1 11 21 31 41]
```

```
print(b[:,1])
```

```
[ 1 11 21 31 41]
```

```
print(b[1:3,:])
```

```
[[10 11 12 13]  
 [20 21 22 23]]
```

```
print(b[-1])
```

```
[40 41 42 43]
```

➤ La fonction fromfunction permet de créer un array en parcourant la fonction entre les bornes fournies

➤ Nous conservons toutes les lignes de la 2nde colonne

➤ Nous conservons toutes les lignes de la 2nde colonne

➤ Nous conservons toutes les colonnes des lignes 2 et 3.

➤ Nous conservons la dernière ligne

INTRODUCTION AUX PACKAGES NUMPY ET SCIPY

NUMPY : QUELQUES FONCTIONS UTILES (1/2)

➤ NumPy dispose d'un grand nombre de fonctions

méthode	Fonctions	Exemple
<code>np.floor(a)</code>	Fonction arrondissant chaque élément réel du tableau à l'entier le plus grand	<pre>a = np.array([-1.7, -1.5, -0.2, 1.5, 1.7, 2.0]) print(np.floor(a))</pre> [-2. -2. -1. 1. 1. 2.]
<code>a.T</code>	T transpose le tableau	<pre>a = np.floor(10*np.random.random((3,4))) print(a)</pre> [[3. 2. 0. 3.] [6. 7. 3. 3.] [3. 7. 8. 7.]] <pre>print(a.T)</pre> [[3. 6. 3.] [2. 7. 7.] [0. 3. 8.] [3. 3. 7.]]
<code>np.sum(a)</code>	Fonction sommant les éléments du tableau	<pre>np.sum([[0, 1], [0, 5]])</pre> 6 <pre>np.sum([[0, 1], [0, 5]], axis=0)</pre> array([0, 6])

INTRODUCTION AUX PACKAGES NUMPY ET SCIPY

NUMPY : QUELQUES FONCTIONS UTILES (2/2)

méthode	Fonctions	Exemple
<code>np.where(a<5, 'Petit', 'Grand')</code>	La fonction where correspond à un if else	<pre>a = np.arange(10) print(a) [0 1 2 3 4 5 6 7 8 9] b = np.where(a<5, 'Petit', 'Grand') print(b) ['Petit' 'Petit' 'Petit' 'Petit' 'Petit' 'Grand' 'Grand' 'Grand' 'Grand' 'Grand']</pre>
<code>np.bincount(a)</code>	La fonction bincount compte le nombre d'occurrences pour chaque valeur. Elle fournit un tableau de 10 valeurs (de 0 à 9) avec le nombre d'occurrences : 1 fois 0, 3 fois 1,..	<pre>a = np.array([1,2,1,0,5,1,9,5]) print(np.bincount(a)) [1 3 1 0 0 2 0 0 0 1]</pre>
<code>np.any(a<0)</code>	La fonction any indique si au moins un valeur vérifie la condition renseignée	<pre>a = np.array([5,3,2,-1,4]) print(np.any(a<0)) True</pre>
<code>np.all(a>0)</code>	La fonction all indique si toutes les valeurs vérifient la condition renseignée	<pre>print(np.all(a>0)) False</pre>

INTRODUCTION AUX PACKAGES NUMPY ET SCIPY

NUMPY : LES TYPES DE DONNÉES (1/3)

- NumPy provient de C : les types de données (dtype) de NumPy proviennent donc de C.
- Elle sont différentes des types de base de Python.

Type NumPy	Description
np.bool_	True ou False sur un bit
np.byte	Caractère signé
np.ubyte	Caractère non signé
np.short	Entier court (dépend de l'OS)
np.ushort	Entier court sans signe (dépend de l'OS)
np.intc	Entier (dépend de l'OS)
np.uintc	Entier sans signe (dépend de l'OS)
np.int_	Entier long (dépend de l'OS)
np.uint	Entier sans signe long (dépend de l'OS)
np.longlong	Entier 2x long (dépend de l'OS)
np.ulonglong	Entier sans signe 2x long (dépend de l'OS)
np.half / np.float16	Réel avec 1 bit pour le signe, 5 bits pour l'exposant et 10 bits pour la mantisse
np.single	Réel avec 1 bit pour le signe, 8 bits pour l'exposant et 23 bits pour la mantisse

Type NumPy	Description
np.double	Réel avec 1 bit pour le signe, 11 bits pour l'exposant et 52 bits pour la mantisse
np.longdouble	Réel avec longueur étendue (dépend de l'OS)
np.csingle	Complexe avec parties réelle et imaginaire en simple
np.cdouble	Complexe avec parties réelle et imaginaire en double long
np.clongdouble	Complexe avec parties réelle et imaginaire en double
np.int8	Entier 8 bits : -128 à 127
np.int16	Entier 16 bits : -32 768 à 32 767
np.int32	Entier 32 bits : -2 147 483 648 à 2 147 483 647
np.int64	Entier 64 bits : -9 223 372 036 854 775 808 à 9 223 372 036 854 775 807

Type NumPy	Description
np.uint8	Entier 8 bits sans signe : 0 à 255
np.uint16	Entier 16 bits sans signe : 0 à 65 535
np.uint32	Entier 32 bits sans signe : 0 à 4 294 967 295
np.uint64	Entier 64 bits
np.intp	Entier utilisé pour indexer (equiv. Au ssize_t de C)
np.uintp	Entier assez grand pour conserver un pointeur
np.float32	Réel sur 32 bits
np.float64	Réel sur 64 bits
np.complex64	Partie réelle et partie imaginaire sur 32 bits
np.complex128	Partie réelle et partie imaginaire sur 64 bits

INTRODUCTION AUX PACKAGES NUMPY ET SCIPY

NUMPY : LES TYPES DE DONNÉES (2/3)

```
import numpy as np
a = np.arange(3, dtype=np.int8)
a

array([0, 1, 2], dtype=int8)
```

➤ Le type de données est assigné avec dtype.

➤ Il est possible de changer le type de données :

```
a.astype(np.int64)

array([0, 1, 2], dtype=int64)
```

```
np.int8(a)

array([0, 1, 2], dtype=int8)
```

- à l'aide la fonction astype ou en appeler directement le type de données NumPy.
- Des types NumPy collent à des types Python standards (int Python correspond à int32).

INTRODUCTION AUX PACKAGES NUMPY ET SCIPY

NUMPY : LES TYPES DE DONNÉES (3/3)

➤ La capacité de stockage est naturellement lié au type de données indiqué.

➤ Par exemple, la fonction power de 100 et 8 réalise $100 \cdot 10^8 = 10\,000\,000\,000\,000\,000$.

```
np.power(100, 8, dtype=np.int64)
```

```
100000000000000000
```

```
np.power(100, 8, dtype=np.int32)
```

```
1874919424
```

➤ Le résultat est bon en utilisant des entiers sur 64 bits.

➤ Le résultat est faux en utilisant des entiers sur 32 bits (maximum atteint).

INTRODUCTION AUX PACKAGES NUMPY ET SCIPY

NUMPY : SYNTHÈSE

- NumPy amène clairement des méthodes spécifiques au calcul de tableaux.
- Pour certaines opérations, NumPy fait les mêmes opérations que les listes basiques de Python.
- Analysez le code ci-contre.
 - Nous réalisons une opération sur un tableau d'une taille de 50 000 000.
- NumPy réalise l'opération en 0,2 secondes contre 11 secondes pour les listes. Soit 61 fois plus vite.
- Pourquoi ? Parce que NumPy est codé en C.
 - Le secret se réside dans la vectorisation.

```
import time; import numpy as np
taille = 50000000

def pur_python_version():
    t1 = time.time()
    X= Y = range(taille)
    Z = [X[i] + Y[i] for i in range(len(X))]
    return time.time() - t1

def numpy_version():
    t1 = time.time()
    X = Y = np.arange(taille)
    Z = X + Y
    return time.time() - t1

t1 = pur_python_version()
t2 = numpy_version()

print("Temps d'opérations est de {:d}".format(taille))
print("Temps pour la liste est de {:.2.3} secondes".format(t1))
print("Temps pour numpy est de {:.2.3} secondes".format(t2))
print("Numpy est {:.2.0f} plus rapide".format(t1/t2))
```

```
Le temps d'opérations est de 50000000
Le temps pour la liste est de 13.0 secondes
Le temps pour numpy est de 0.154 secondes
Numpy est 85 plus rapide
```

INTRODUCTION AUX PACKAGES NUMPY ET SCIPY

SCIPY : INTRODUCTION

- Scipy est une librairie de calcul scientifique
- Le format recommandé est le type array de numpy
- Scipy couvre de nombreux domaines (confère ci-dessous) → Nous nous focaliserons sur les fonctions de statistiques.

- Clustering package (`scipy.cluster`)
- Constants (`scipy.constants`)
- Discrete Fourier transforms (`scipy.fftpack`)
- Integration and ODEs (`scipy.integrate`)
- Interpolation (`scipy.interpolate`)
- Input and output (`scipy.io`)
- Linear algebra (`scipy.linalg`)
- Miscellaneous routines (`scipy.misc`)
- Multi-dimensional image processing (`scipy.ndimage`)
- Orthogonal distance regression (`scipy.odr`)
- Optimization and root finding (`scipy.optimize`)
- Signal processing (`scipy.signal`)
- Sparse matrices (`scipy.sparse`)
- Sparse linear algebra (`scipy.sparse.linalg`)
- Compressed Sparse Graph Routines (`scipy.sparse.csgraph`)
- Spatial algorithms and data structures (`scipy.spatial`)
- Special functions (`scipy.special`)
- Statistical functions (`scipy.stats`)
- Statistical functions for masked arrays (`scipy.stats.mstats`)
- C/C++ integration (`scipy.weave`)

INTRODUCTION AUX PACKAGES NUMPY ET SCIPY

SCIPY : QUELQUES EXEMPLES

```
# Import package
import numpy as np
import scipy.stats as stat } Convention d'importation

'''Soit d un vecteur d'observations'''
d = np.array([0.553,0.57,0.576,0.601,0.606,0.606,0.609,0.611,0.615,
              0.628,0.654,0.662,0.668,0.67,0.672,0.69,0.693,0.749])

'''Statistiques descriptives'''
stat_d = stat.describe(d)
print(stat_d)
#DescribeResult(nobs=18, minmax=(0.55300000000000005, 0.749),
#               mean=0.6351666666666666,
#               variance=0.0025368529411764714,
#               skewness=0.38763289979752136,
#               kurtosis=-0.3587369048751916)

'''Récupérer une valeur en particulier (exemple nombre d'observations) '''
print(stat_d[0]) # --> 18

print(stat_d.nobs) #--> 18

n,mm,moy,var,skw,kt = stat.describe(d)
print(n) #--> 18

'''Médiane et quantile'''
print(np.median(d)) #--> 0.6215
print(np.percentile(d,q = 10)) #--> 0.5742
print(stat.percentileofscore(d,0.6215))
# --> 50.0, la moitié des obs. ont une valeur inf. à 0.6215)
```

INTRODUCTION AUX PACKAGES NUMPY ET SCIPY

SCIPY : LES TESTS STATISTIQUES

➤ Scipy propose de nombreux tests statistiques, notamment des test d'adéquation à la loi Normale → le vecteur suit-il une distribution normale ?

- Test de normalité d'Agostino

```
ag = stat.normaltest(d)
print(ag) #--> NormaltestResult(statistic=0.71439079391858273,
#                               pvalue=0.69963577767413498)
```

- Test de normalité de Shapiro-Wilks

```
sp = stat.shapiro(d)
print(sp) #--> (0.9613385200500488, 0.627672016620636)
# (statistique et p-value)
```

- Test d'adéquation d'Anderson-Darlin

```
ad = stat.anderson(d,dist = 'norm')
print(ad) #AndersonResult(statistic=0.34029632368620355,
#                          critical_values=array([ 0.503,  0.573,  0.687,  0.802,  0.954]),
#                          significance_level=array([ 15. ,  10. ,   5. ,   2.5,   1. ]))
```

- Les distributions possibles (dist =) : normal (norm) , exponentielle (expon) , logistique (logistic), Gumbel (gumbel)
- Les valeurs critiques correspondent aux valeurs maximales par rapport au niveau de confiance en pourcentage, dans notre cas $0.34 < 0.503$ donc $pvalue > 15 \%$

INTRODUCTION AUX PACKAGES NUMPY ET SCIPY

SCIPY : LES DISTRIBUTIONS STATISTIQUES

➤ Scipy propose de nombreuses distributions statistiques :

- Retrouver l'ensemble des lois proposées sur : <https://docs.scipy.org/doc/scipy/reference/stats.html>
- Pour l'ensemble des lois Scipy propose plusieurs fonctions, exemple avec la loi normale et la loi exponentielle :

```
'''Obtenir les valeurs des quantiles et  
des fonctions de repartition'''
```

```
print(stat.norm.ppf(0.95, loc= 0, scale = 1)) # --> CR = 1.64485
```

```
# quantile d'ordre 0.95 d'une loi normale centrée réduite
```

```
print(stat.norm.cdf(1.96, loc = 0, scale = 1)) # --> 0.975
```

```
'''Générer 30 points aléatoirement issus d'une distributions normale'''
```

```
x = stat.norm.rvs(loc = 0, scale = 1, size = 30)
```

```
'''Test de normalité'''
```

```
print(stat.normaltest(x))
```

```
#NormaltestResult(statistic=2.8179964365275616, pvalue=0.2443879839834675)
```

```
'''Générer 30 point d'une loi exponentielle'''
```

```
E = stat.expon.rvs(size = 30)
```

```
'''Test de normalité'''
```

```
print(stat.normaltest(E))
```

```
#NormaltestResult(statistic=29.430225961869137, pvalue=4.0673133463526996e-07)
```

INTRODUCTION AUX PACKAGES NUMPY ET SCIPY

SCIPY : TEST DE STUDENT

➤ Test de Student :

```
'''Test de conformité de la moyenne : TEST DE STUDENT'''
print(stat.ttest_1samp(d,popmean=0.618)) # (1.446, 0.166)
#stat. de test et p-value, p-value < α, rejet de H0

'''Si l'on détaille les calculs'''
#moyenne
m = np.mean(d) # 0.6352
#écart-type - ddof = 1 pour effectuer le calcul : 1/(n-1)
sigma = np.std(d,ddof=1) # 0.0504
#stat. de test t
import math
t = (m - 0.618)/(sigma/math.sqrt(d.size))
print(t) # 1.446, on retrouve bien la bonne valeur de la stat de test
#p-value - c'est un test bilatéral
#t distribution de Student, cdf() : cumulative distribution function
p = 2.0 * (1.0 - stat.t.cdf(math.fabs(t),d.size-1))
print(p) # 0.166, et la bonne p-value
```

<u>Objectif :</u>	$H_0 : \mu = 0.618$
	$H_1 : \mu \neq 0.618$

t est le code pour la distribution de student

INTRODUCTION AUX PACKAGES NUMPY ET SCIPY

SCIPY : EXEMPLES (1/4)

➤ Expérience :

- Une classe suit des activités de lecture en plus pendant 8 jours
- Objectif : savoir si ces activités supplémentaires donnent des résultats
- 2 populations :
 - 21 étudiants qui ont suivi ces activités
 - 23 étudiants qui n'ont pas suivi ces activités
 - Données : résultat au test de lecture

```
'''Statistiques à deux vecteurs '''
'''treated - valeurs pour échantillon des individus ayant suivi le traitement'''
dt = np.array([24,43,58,71,43,49,61,44,67,49,53,56,59,52,62,54,57,33,46,43,57])
'''Echantillon controle'''
dc = np.array([42,43,55,26,62,37,33,41,19,54,20,85,46,10,17,60,53,42,37,42,55,28,48])
'''t-test - comparaison de moyennes - hyp. de variances égales'''
print(stat.ttest_ind(dt,dc)) # (t = 2.2665, p-value = 0.0286)
'''t-test de Welch - comparaison de moyennes - hyp. de variances inégales'''
print(stat.ttest_ind(dt,dc,equal_var=False)) # (2.3109, 0.0264)
'''test de Mann-Whitney - non paramétrique - avec correction de continuité'''
print(stat.mannwhitneyu(dt,dc)) # (stat. U = 135, p-value unilatérale = 0.00634)
'''test de Bartlett - comparaison de paramètres d'échelle (variance)'''
print(stat.bartlett(dt,dc)) # (stat. = 3.8455, p-value = 0.0498)
'''test de Ansari Bradley'''
print(stat.ansari(dt,dc)) # (stat. = 266, p-value = 0.2477)
'''test de Levene'''
print(stat.levene(dt,dc)) # (stat. = 2.342, p-value = 0.1334)
'''test de Kolomogorov-Smirnov - écart entre les fonctions de répartition empiriques'''
print(stat.ks_2samp(dt,dc)) # (stat. = 0.4699, p-value = 0.0099)
```

Création du jeu
de données en
array

Utilisation de
différents test
statistiques

INTRODUCTION AUX PACKAGES NUMPY ET SCIPY

SCIPY : EXEMPLES (2/4)

➤ Expérience :

- Données : taux de travail des femmes dans 19 villes aux Etats-Unis en 1968 et 1972
- Objectif : Comparer les résultats entre les 2 années

➤ Code python :

```
#Comparaison de 2 populations

'''Les données'''
d1968 = np.array([0.42,0.5,0.52,0.45,0.43,0.55,0.45,0.34,0.45,
                  0.54,0.42,0.51,0.49,0.54,0.5,0.58,0.49,0.56,0.63])
d1972 = np.array([0.45,0.5,0.52,0.45,0.46,0.55,0.60,0.49,0.35,
                  0.55,0.52,0.53,0.57,0.53,0.59,0.64,0.5,0.57,0.64])

'''t-test related samples - paramétrique'''
print(stat.ttest_rel(d1968,d1972))# (stat.test = -2.45, p-value = 0.024)
'''test des rangs signés - non paramétrique'''
print(stat.wilcoxon(d1968,d1972)) # (stat = 16, p-value = 0.0122)
```

➤ Conclusion : la proportion des femmes actives est significativement différente en 1972

INTRODUCTION AUX PACKAGES NUMPY ET SCIPY

SCIPY : EXEMPLES (3/4)

➤ Expérience :

- Objectif : Identifier s'il existe une relation entre les dépenses en alcool et les dépenses en tabac au Royaume-Uni

➤ Code python :

```
'''Données pour corrélation et régression (Irlande du Nord non incluse)'''
dalc = np.array([6.47,6.13,6.19,4.89,5.63,4.52,5.89,4.79,5.27,6.08])
dtob = np.array([4.03,3.76,3.77,3.34,3.47,2.92,3.2,2.71,3.53,4.51])
'''Différentes techniques de calcul pour la corrélation
    - Corrélation de Pearson'''
print(stat.pearsonr(dalc,dtob)) # (r = 0.7843, p-value pour test t = 0.0072)
'''    - Corrélation de Spearman - basé sur les rangs'''
print(stat.spearmanr(dalc,dtob)) # (rho = 0.8303, p-value = 0.0029)
'''    - Tau de Kendall - concordance et discordance'''
print(stat.kendalltau(dalc,dtob)) # (tau = 0.6444, p-value = 0.0095)
'''    - Régression linéaire simple'''
print(stat.linregress(dalc,dtob)) # (pente = 0.6115, const = 0.1081, r = 0.7843,
#p-value test signif. pente = 0.0072, sigma err = 0.1710)
```

➤ Conclusion : lien positif entre les dépenses de tabac et les dépenses d'alcool

INTRODUCTION AUX PACKAGES NUMPY ET SCIPY

SCIPY : EXEMPLES (4/4)

➤ Expérience :

- Données : jeu de données du Titanic (informations sur les passagers et si ils ont survécu ou non)
- Objectif : Savoir si la variable 'Sex' est indépendante de la variable 'Survived'

➤ Code python :

```
# Exemple test chi2 tableau de contingence :
import pandas as pd
path_data = 'C:/Formation Python/Datasets/'
df = pd.read_sas(path_data + 'titanic.sas7bdat', encoding = 'latin-1')

cont_T = pd.crosstab(df['Sex'], df['Survived'])
#Survived    0    1
#Sex
#female      81   233
#male       468   109

stat.chi2_contingency(cont_T, correction=False)
#(263.05057407065567,          #test statistic
# 3.711747770113424e-59,      #pvalue
# 1,                          # degrés de liberté
# array([[ 193.47474747,  120.52525253], # fréquence espérée
#        [ 355.52525253,  221.47474747]]))
```

➤ Conclusion : les deux variables ne sont pas indépendantes

01

Les principaux environnements de développement Python

02

Les concepts de base en Python

03

Introduction aux packages NUMPY et SCIPY

04

Import et préparation des données avec Pandas

05

Statistiques descriptives

06

Graphiques avec Matplotlib

IMPORT ET PRÉPARATION DES DONNÉES AVEC PANDAS

INTRODUCTION

➤ Le package Pandas permet de définir et de manipuler des données sous forme de tableaux.

- Nous parlons en Python de data frame

➤ Pandas : Panel Data System.

- Le S est obligatoire

➤ Pandas hérite nativement des packages NumPy (gestion de tableaux), pytz (fuseaux horaires de IANA Time Zone Database) et dateutil (gestion de dates).

➤ Pandas s'est notamment inspiré du package dplyr de R.



PANDAS

NOTIONS DE DATA FRAME, SERIES

➤ Il existe 3 objets Pandas :

- Les **series** : ces objets sont des tableaux de 1 dimension avec un libellé d'axe.
 - Contrairement à NumPy, les éléments peuvent être de nature différente.

```
import pandas as pd
a = pd.Series([1,2,'a',3])
```

- Les **data frame** : ces objets sont des tableaux de 2 dimensions.

```
b = pd.DataFrame({'cle': [1,2,3,5], 'valeur' : [2.8,18,0,'a']})
```

- Les **panels** : ces objets sont des tableaux à 3 dimensions.
 - Les panels sont dépréciés de Pandas depuis la version 0.20.0.
 - Nous n'en ferons plus référence.

```
class pandas.Series(data=None, index=None, dtype=None, name=None, copy=False)
```

Paramètre	Description
data	Contient les données stockées dans la Series
index	Indice des observations
dtype	Format des observations
Name	Nom de la Series
copy	Permet de copier des données

Il existe en fait un autre paramètre fastpath d'ordre technique utilisé par pandas

PANDAS

DATA FRAME

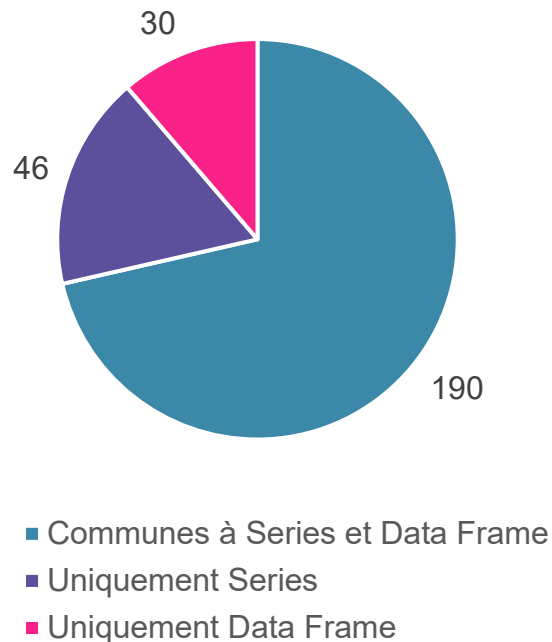
```
class pandas.DataFrame(data=None, index: Optional[Collection] = None,
columns: Optional[Collection] = None, dtype: Union[str, NumPy.dtype,
ExtensionDtype, None] = None, copy: bool = False)
```

Paramètre	Description
data	Contient les données stockées dans le Data Frame
index	Indice des observations
columns	Indice des colonnes
dtype	Format des observations (cela peut être des types NumPy)
copy	permet de copier des données

PANDAS

NOTIONS DE DATA FRAME, SERIES

Nb méthodes et attributs
Pandas v. 0.25.2



- Les Series et Data Frame disposent de nombreuses méthodes et attributs communes (190 en excluant les spéciales).
- Toutefois, les spécificités des Series amènent 46 spécifiques. Nous nous pencherons spécifiquement sur celles-ci.
- Les Data Frames ont 30 méthodes et attributs spécifiques.

```
#Nombre d'attributs et de méthodes dans Series et pas Data Frame
seriesexcl = len([x for x in dir(pd.Series) if x not in
dir(pd.DataFrame) if not x.startswith('_')])

#Nombre d'attributs et de méthodes dans Data Frame et pas dans
Series
dfexcl = len([x for x in dir(pd.DataFrame) if x not in
dir(pd.Series) if not x.startswith('_')])

#Nombre d'attributs et de méthodes en commun
commun = len([x for x in dir(pd.DataFrame) if x in
dir(pd.Series) if not x.startswith('_')])
```

PANDAS

IMPORT DE FICHIERS

➤ La librairie pandas propose plusieurs outils pour importer les données, le schéma type est le suivant :

```
import pandas as pd
df = pd.read_xxx(path_fichier,p1,p2...)
```

➤ Les différentes fonctions d'import :

- read_csv
- read_excel
- read_hdf
- read_json
- read_html
- read_stata
- read_fwf
- read_sas
- read_clipboard
- read_pickle
- read_gbq
- read_table
- read_sql
- read_sql_query

```
df = pd.read_csv(r'C\Users\Documents\Data\data_titanic.csv', sep = ',')
```

Exemple (Import d'un fichier csv)

PANDAS

IMPORT DE FICHIERS

- Import table d'une base de données SQL :

```
import snowflake.connector

ctx = snowflake.connector.connect(
    user='TEST',
    password='1234',
    account='test.west-europe.azure',
    warehouse='WH_MICROPOLE'
)

cs = ctx.cursor()
cs.execute("SELECT current_version()")
myprint('Connexion OK.')

dt_arret_max_client = pd.read_sql_query("SELECT max(DT_ARRET) as DT_ARRET_MAX FROM DTM_PRD.BI.CLIENT", ctx)
```

- Import de plusieurs fichiers :

```
path = "C:/Data_Accidentologie/"
liste_fichiers = ['caracteristiques_2016', 'lieux_2016', 'usagers_2016', 'véhicules-2016']

for f in liste_fichiers :
    fichier = path + f + '.csv'
    varname = 'df_' + f.replace('-', '_').replace('é', 'e')
    vars()[varname] = pd.read_csv(fichier, sep = ';', quotechar = '"', engine = 'python')
```

PANDAS

IMPORT DE FICHIERS TXT, CSV

➤ read_csv : principales options

```
pandas.read_csv(filepath_or_buffer, sep=<object object>, header='infer', nrows=None, names=None, index_col=None, dtype=None, encoding=None, engine=None)
```

Paramètre	Valeurs possibles	Description
filepath or buffer	str, path object or file-like object	Chemin d'accès du fichier à charger
sep	str, default ','	Séparateur de champs
header	int, list of int, default 'infer'	Numéro de la ligne à utiliser comme noms des colonnes
nrows	Int	Nombre de lignes à importer
names	array-like, optional	Noms des colonnes définis manuellement
index_col	int, str, sequence of int / str, or False, default None	Colonne(s) à utiliser comme labels des index
dtype	Type name or dict of column -> type, optional	Type de données des colonnes du dataframe
encoding	str, default is None	Encodage des données textuelles. A choisir parmi 'uft8', 'latin-1'
engine	{'c', 'python'}, optional	Moteur utilisé

PANDAS

IMPORT DE FICHIERS SAS (SAS7BDAT)

➤ read_sas : les options

```
pandas.read_sas(filepath_or_buffer, format=None, index=None, encoding=None, chunksize=None, iterator=False)
```

Paramètre	Valeurs possibles	Description
filepath or buffer	str, path object or file-like object	Chemin d'accès du fichier à charger
format	str {'xport', 'sas7bdat'} or None	Si None, le format de fichier est déduit de l'extension de fichier Si « xport » ou « sas7bdat », utilise le format correspondant
index	identifiant of index column, defaults to None	Identificateur de colonne à utiliser comme index du DataFrame
encoding	str, default is None	Encodage des données textuelles. A choisir parmi 'uft8', 'latin-1', ... Si None, les données de texte sont stockées sous forme d'octets bruts
chunksize	int	Lecture par blocs de lignes (chunksize détermine le nombre de lignes d'un bloc) de façon itérative, renvoie un itérateur
iterator	bool, defaults to False	Si True, renvoie un itérateur pour lire le fichier de manière incrémentale

PANDAS

TYPAGE DES DONNÉES

➤ Créer un data frame et obtenir les types de chaque colonne :

```
df2 = pd.DataFrame({ 'A' : 1.,  
                     'B' : pd.Timestamp('20130102'),  
                     'C' : pd.Series(1,index=list(range(4)),dtype='float32'),  
                     'D' : np.array([3] * 4,dtype='int32'),  
                     'E' : pd.Categorical(["test","train","test","train"]),  
                     'F' : 'foo' })
```

```
print(df2)
```

#	A	B	C	D	E	F
#0	1.0	2013-01-02	1.0	3	test	foo
#1	1.0	2013-01-02	1.0	3	train	foo
#2	1.0	2013-01-02	1.0	3	test	foo
#3	1.0	2013-01-02	1.0	3	train	foo

```
df2.dtypes
```

#A	float64	• Float64 :	numérique décimal
#B	datetime64[ns]	• Datetime64[ns] :	date
#C	float32	• Float 32 :	numérique décimal
#D	int32	• Int32 :	numérique entier
#E	category	• Category :	ne prend que deux valeurs (le type est défini avec la fonction pd.Categorical)
#F	object	• Object :	c'est le type pour les chaînes de caractères dans pandas
#dtype:	object		

PANDAS

FONCTIONS DE BASE

Afficher la structure d'un dataset :



`df.shape`

Renvoie un tuple
(nb lignes, nb colonnes)

Afficher les noms des colonnes :

`df.columns`

Renvoie une liste avec les
noms des colonnes

Afficher les index :

`df.index`

Renvoie une liste avec les
noms des index

Conversion en Array :

`df.values`

Renvoie un Array de la taille (nb
lignes x nb colonnes)

Afficher le type des colonnes :

`df["Var1"].dtype`

Renvoie le type de la
variable Var1

`df.dtypes`

Renvoie le type de chaque
colonne du data frame df

Afficher les 1ères lignes d'un dataset :

`df.head(n)`

Renvoie les n
premières lignes de df

Afficher les principales statistiques descriptives d'un dataset :

`df.describe()`

Renvoie un dataset contenant
les statistiques descriptives
(moyenne, min, max,
percentiles,...) du DataFrame df



Tris à plat :

`df.Var1.value_counts()`

Renvoie un objet Series
contenant les effectifs de chaque
modalité de la variable Var1



Transposée d'un dataset :

`df.T`

Renvoie la transposée de df

Valeurs uniques :

df

Var1
1
1
0



Var1
1
0

`df.Var1.unique()`

PANDAS

FONCTIONS DE BASE

Tableau croisé :



df

Name	Exam	Subject	Result
Alisa	S1	Maths	Pass
Bobby	S1	Maths	Pass
Cathy	S1	Maths	Fail
Alisa	S1	Science	Pass
Bobby	S2	Science	Fail



	Fail	Pass	All
Maths	1	2	3
Science	1	1	2
All	2	3	5



```
pd.crosstab(df.Subject, df.Result, margins = True)
```


PANDAS

FONCTIONS DE BASE

```
df = pd.DataFrame({"Col1": pd.Series([1,2,3,4,5,6]),  
                  "Col2" : pd.Categorical(["Oui","Non","Oui","Oui","Non","Oui"]),  
                  "Col3": pd.Series([23,4,12,2,0,-10])})
```

'''les 2 premières lignes'''

```
df.head(2)
```

#	Col1	Col2	Col3
#0	1	Oui	23
#1	2	Non	4

'''Les 2 dernières lignes'''

```
df.tail(2)
```

#	Col1	Col2	Col3
#4	5	Non	0
#5	6	Oui	-10

'''Toutes les lignes moins les 2 dernières'''

```
df.head(-2)
```

#	Col1	Col2	Col3
#0	1	Oui	23
#1	2	Non	4
#2	3	Oui	12
#3	4	Oui	2

PANDAS

FONCTIONS DE BASE

```
'''Obtenir des informations sur les variables numériques'''
```

```
df.describe()
```

```
#           Col1           Col3
#count  6.000000  6.000000
#mean    3.500000  5.166667
#std     1.870829 11.250185
#min     1.000000 -10.000000
#25%     2.250000  0.500000
#50%     3.500000  3.000000
#75%     4.750000 10.000000
#max     6.000000 23.000000
```

```
'''Transposer une table'''
```

```
df.T
```

```
#           0    1    2    3    4    5
#Col1       1    2    3    4    5    6
#Col2  Oui  Non  Oui  Oui  Non  Oui
#Col3   23    4   12    2    0  -10
```

```
'''Trier une table en fonction d'une colonne'''
```

```
df.sort_values(by= "Col3", ascending = True, inplace = True)
```

```
#    Col1 Col2  Col3
#5      6  Oui  -10
#4      5  Non    0
#3      4  Oui    2
#1      2  Non    4
#2      3  Oui   12
#0      1  Oui   23
```

PANDAS

RENOMMAGE – SUPPRESSION DE COLONNES

RENOMMER - COLONNES

Var1	Var2

 →

Var3	Var2



```
ALTER TABLE df  
RENAME COLUMN Var1 TO Var3
```



```
DATA df;  
SET df (rename=(Var1=Var3));  
RUN;
```

Renommer une colonne d'un Dataframe :
fonction rename (columns = dictionnaire)

- **La clé** : nom actuel de la colonne
- **La valeur** : le nouveau nom



```
df = df.rename(columns = {"Var1" : "Var3"})  
or  
df.rename(columns = {"Var1" : "Var3"}, inplace=True)
```

SUPPRESSION - COLONNES

Var1	Var2	Var3



```
ALTER TABLE df  
DROP COLUMN Var1, Var3
```



```
DATA df (drop = Var1 Var3);  
SET df;  
RUN;
```

Suppression d'une ou plusieurs colonnes d'un Dataframe :
fonction drop (columns = liste)

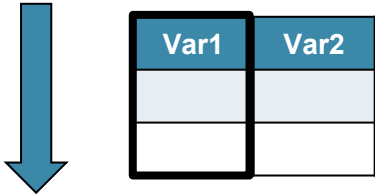


```
df = df.drop(columns = ["Var1", "Var3"], axis=1)  
or  
df.drop(columns = ["Var1", "Var3"], axis=1, inplace=True)
```

PANDAS

TRIER, DEDUPLIQUER UN DATAFRAME

TRIER UN DATAFRAME



```
SELECT * FROM df  
ORDER BY Var1 DESC
```



```
PROC SORT data = df;  
BY DESCENDING Var1;  
RUN;
```

Trier un Dataframe selon une ou plusieurs colonnes d'un Dataframe : fonction `sort_values` (`columns = liste`)



Tri sur 1 variable

```
df = df.sort_values(by=["Var1"],  
                    ascending = True)
```

or

```
df.sort_values(by=["Var1"],  
               ascending = True,  
               inplace = True)
```



Tri sur plusieurs variables

```
df.sort_values(by=["Var1", "Var2"],  
               ascending = [True, False],  
               inplace = True)
```

DÉDUPLIQUER UN DATAFRAME

Var1	Var2	Var3
1	1	1
1	2	1
0	2	3

 →

Var1	Var2	Var3
1	2	1
0	2	3



```
PROC SORT data = df NODUPKEYS;  
BY Var1 Var3;  
RUN;
```



```
df.drop_duplicates(subset = ['Var1', 'Var3'],  
                  keep = 'last',  
                  inplace = True,  
                  ignore_index = False)
```

PANDAS

GESTION DES VALEURS MANQUANTES

VALEURS MANQUANTES

Définition de la valeur manquante en python : `np.nan`

Var1	Var1
1	1
NaN	0
-1	-1
NaN	0

Ex : remplacer les valeurs manquantes par 0



UPDATE df SET Var1 = 0
WHERE Var1 IS NULL



DATA df;
SET df;
IF Var1=. THEN Var1=0;
RUN;



```
df["Var1"] = df.Var1.fillna(0)
```

Options de la fonction fillna :

- Remplacer par la valeur suivante :
`df["Var1"] = df.Var1.fillna(method="pad")`
- Remplacer par la valeur précédente :
`df["Var1"] = df.Var1.fillna(method="bfill")`
- Remplacer par la moyenne :
`df["Var1"] = df.Var1.fillna(df.Var1.mean())`

CONDITIONS SUR LES VALEURS MANQUANTES

La fonctions `pd.isnull()`

Var1	Var2	Var3
1	0	0
2	NaN	0
1	0	NaN



```
df[df["Var3"].isnull()]
```



On ne peut pas écrire une condition du type

`df[df["Var3"] == np.nan]` ❌ FAUX

La fonctions `pd.notnull()`

Var1	Var2	Var3
1	0	0
2	NaN	0
1	0	NaN



```
df[df["Var3"].notnull()]
```

FORMAT DATE

Conversion :

Var1
13/12/2017
26/03/2018



```
Var1 = pd.to_datetime(df.Var1,  
format = "%d/%m/%Y")
```

Extraction éléments :

Var1	D	M	Y
13/12/2017	13	12	2017
26/03/2018	26	03	2018



```
D = df.Var1.dt.day  
M = df.Var1.dt.month  
Y = df.Var1.dt.year
```

Différence entre deux dates :

Var1	Var2	Var3
13/12/2017	25/12/2017	12
26/03/2018	30/03/2017	4



```
Var3 = (df.Var2 - df.Var1).dt.days
```

PANDAS

SÉLECTION DE LIGNES

SÉLECTION À PARTIR D'UNE CONDITION

Condition basée sur 1 variable :

Var1	Var2	Var3
1	0	0
2	1	0
1	0	1

Ex : Sélection basée sur une simple condition



```
SELECT * from df  
WHERE Var1=1
```



```
DATA df;  
SET df;  
IF Var1= 1;  
RUN;
```



```
df[df["Var1"]==1] or df.loc[df.Var1==1]
```

Conditions multiples :

Var1	Var2	Var3
1	0	0
2	1	0
1	0	1

Ex : Sélection basée sur plusieurs condition



```
SELECT * FROM df  
WHERE Var1=1 AND Var3=0
```



```
DATA df;  
SET df;  
IF Var1= 1 AND Var3=0;  
RUN;
```



```
df[(df["Var1"]==1) & (df["Var3"]==0)]
```

AUTRES FONCTIONS

Sélection par numéro de lignes (iloc) :

Var1	Var2
1	2
1	1
0	2
5	1
4	0

Sélection de la deuxième à la quatrième ligne



```
df.iloc[1:4]  
or  
df.iloc[[1,2,3]]
```

Sélection à l'aide de fonctions

Var1	Var2	Var3
1	0	0
2	1	0
0	0	1



Sélection des lignes pour lesquelles Var1 vaut 1 ou 0

```
df[df['Var1'].isin([1,0])]
```



```
df.loc[np.where(df['Var1'] <= 1)]
```

Var1	Var2	Var3
1	0	0
2	1	0



Sélection des lignes pour lesquelles Var1 est différent de 1 et 0

```
df[~(df['Var1'].isin([1,0]))]
```

Var1	Var2	Var3
1	0	NaN
2	NaN	0



Sélection des lignes pour lesquelles Var3 n'est pas manquant

```
df[df["Var3"].notnull()]
```

PANDAS

SÉLECTION DE COLONNES ET SÉLECTION SIMULTANÉE

SÉLECTION - COLONNES

Sélection d'une colonne :

Var1	Var2	Var3



```
SELECT Var1 FROM df
```



```
DATA df (keep = Var1);  
SET df;  
RUN;
```



```
df["Var1"] or df.Var1
```

Sélection de plusieurs colonnes :

Var1	Var2	Var3



```
SELECT Var1,Var2 FROM df
```



```
DATA df (keep = Var1 Var2);  
SET df;  
RUN;
```



```
df[["Var1", "Var3"]]
```

SÉLECTION DE LIGNES ET COLONNES

Index	Var1	Var2	Var3
a	1	2	3
b	4	5	6
c	7	8	9

```
df.iloc[0:2,1:3]
```



or

```
df.loc[['a','b'], ['Var2','Var3']]
```


PANDAS

PRINCIPALES FONCTIONS – STATISTIQUES SIMPLES

Moyenne

Var1
1
0
-1

→ 0



```
SELECT MEAN(Var1)  
FROM df
```



```
df.Var1.mean()
```

Max

Var1
1
0
-1

→ 1



```
SELECT MAX(Var1)  
FROM df
```



```
df.Var1.max()
```

Min

Var1
1
0
-1

→ -1



```
SELECT MIN(Var1)  
FROM df
```



```
df.Var1.min()
```

Écart-type

Var1
1
0
-1

→ 1



```
SELECT STDEV(Var1)  
FROM df
```



```
df.Var1.std()
```

Nombre de valeurs uniques

Var1
1
1
-1

→ 2



```
df.Var1.nunique()
```

Quantiles

Var1
1
1
-1

→

-0.6
1



```
df.Var1.quantile([.1, .5])
```

Kurtosis

Var1
1
0
-1
2

→ -1.2



```
df.Var1.kurtosis()
```

Coefficient d'asymétrie

Var1
1
1
-1

→ -1.2



```
df.Var1.skew()
```

PANDAS

PRINCIPALES FONCTIONS

AGRÉGATIONS AVANCÉES

Group by simple – calcul d'agrégat sur une variable

Ex : Calcul de la moyenne de Var2 pour chaque modalité de Var1



```
SELECT Var1, MEAN(Var2)
FROM df GROUP BY Var1
```



```
df.groupby("Var1")["Var2"].mean()
```

Group by multiple – calcul d'agrégats sur plusieurs variables

Ex : Calcul de la moyenne et de la somme totale de Var2 et Var4 pour chaque modalité de Var1 et de Var3



```
df.groupby(["Var1", "Var3"])["Var2", "Var4"]
    .agg(['mean', 'sum'])
```

Group by multiple – calcul d'agrégats sur une variable

Ex : Calcul de la moyenne et de la somme totale de Var2 pour chaque modalité de Var1 et de Var3



```
SELECT Var1, Var3 MEAN(Var2)
FROM df GROUP BY Var1
```



```
df.groupby(["Var1", "Var3"])["Var2"]
    .agg(['mean', 'sum'])
```

PANDAS

PRINCIPALES FONCTIONS

EXEMPLES

```
df = pd.DataFrame({'ID_maison': [0,1,2,3,4,5],  
                  'date_vente': ['15/01/2021', '10/01/2021', '13/01/2021', '15/01/2021', '15/01/2021', '10/01/2021'],  
                  'ville': ['Paris', 'Paris', 'Lyon', 'Paris', 'Lyon', 'Paris'],  
                  'Prix_m2': [15000, 9000, 4000, 8000, 6000, 13000],  
                  'Surface' : [30, 21, 42, 15, 63, 40]})
```

Group by – clé simple – calcul d'un agrégat pour une variable quantitative

Ex : Calcul de la moyenne du Prix_m2 pour chaque modalité de ville

ID_maison	date_vente	ville	Prix_m2	Surface
0	15/01/2021	Paris	15 000	30
1	10/01/2021	Paris	9 000	21
2	13/01/2021	Lyon	4 000	42
3	15/01/2021	Paris	8 000	15
4	15/01/2021	Lyon	6 000	63
5	10/01/2021	Paris	13 000	40



ville	Prix_m2
Lyon	5 000
Paris	11 250

Le résultat obtenu est une serie qui prend ville pour index. Pour conserver un dataframe en sortie avec la ville non pas comme index mais comme variable, il faut ajouter `.reset_index()` après l'instruction ci-dessous à gauche.



```
df.groupby(['ville'])['Prix_m2'].mean()
```

PANDAS

PRINCIPALES FONCTIONS

EXEMPLES

Group by – clé multiple – calcul de plusieurs agrégats pour une variable quantitative

Ex : Calcul de la moyenne et du maximum du Prix_m2 pour chaque modalité de ville et de date_vente

ID_maison	date_vente	ville	Prix_m2	Surface
0	15/01/2021	Paris	15 000	30
1	10/01/2021	Paris	9 000	21
2	13/01/2021	Lyon	4 000	42
3	15/01/2021	Paris	8 000	15
4	15/01/2021	Lyon	6 000	63
5	10/01/2021	Paris	13 000	40



ville	date_vente	mean	max
Lyon	13/01/2021	4 000	4 000
	15/01/2021	6 000	6 000
Paris	10/01/2021	11 000	13 000
	15/01/2021	11 500	15 000

Le résultat obtenu est une dataframe contenant 2 colonnes et un index à deux niveaux.

Pour conserver un dataframe en sortie avec 4 colonnes et un index numéroté, il faut ajouter `.reset_index()` après l'instruction ci-dessous à gauche.



```
df.groupby(['ville', 'date_vente'])['Prix_m2'].agg(['mean', 'max'])
```

PANDAS

PRINCIPALES FONCTIONS

EXEMPLES

Group by – clé multiple – calcul de plusieurs agrégats pour plusieurs variables quantitatives

Ex : Calcul de la moyenne et du maximum du Prix_m2 et de Surface pour chaque modalité de ville et de date_vente

ID_maison	date_vente	ville	Prix_m2	Surface
0	15/01/2021	Paris	15 000	30
1	10/01/2021	Paris	9 000	21
2	13/01/2021	Lyon	4 000	42
3	15/01/2021	Paris	8 000	15
4	15/01/2021	Lyon	6 000	63
5	10/01/2021	Paris	13 000	40



ville	date_vente	Prix_m2		Surface	
		mean	max	mean	max
Lyon	13/01/2021	4 000	4 000	42	42
	15/01/2021	6 000	6 000	63	63
Paris	10/01/2021	11 000	13 000	30.5	40
	15/01/2021	11 500	15 000	22.5	30

Le résultat obtenu est une dataframe contenant 4 colonnes et un index à deux niveaux. Les libellés de colonnes sont structurés sur 2 niveaux également.



```
df.groupby(['ville', 'date_vente'])[['Prix_m2', 'Surface']].agg(['mean', 'max'])
```

Remarque : Pour obtenir un dataframe contenant 6 colonnes ('ville', 'date_vente', 'Prix_m2_mean', 'Prix_m2_max', 'Surface_mean', 'Surface_max') et un index numéroté, il faut utiliser les instructions suivantes :

```
df_agg = df.groupby(['ville', 'date_vente'])[['Prix_m2', 'Surface']].agg(['mean', 'max']).reset_index()
df_agg.columns = ['_'.join(x) if len(x[1])>0 else x[0] for x in df_agg.columns]
```

PANDAS

CONCATÉNATION / EMPILEMENT (1/3)

FONCTION CONCAT

Ex : Concaténation de deux DataFrames df1 et df2 en df

Paramètres par défaut

df1	index	Var1
	0	a
	1	b

df2	index	Var1
	0	c
	1	d

→ df

index	Var1
0	a
1	b
0	c
1	d



```
CREATE TABLE df AS  
(SELECT Var1 FROM df1) UNION  
(SELECT Var1 FROM df2)
```



```
DATA df;  
SET df1 df2;  
RUN;
```



```
df = pd.concat([df1,df2],axis=0)
```

Paramètre `ignore_index`

df1	index	Var1
	0	a
	1	b

df2	index	Var1
	0	c
	1	d

→ df

index	Var1
0	a
1	b
2	c
3	d



```
df = pd.concat([df1, df2], ignore_index = False)
```

PANDAS

CONCATENATION / EMPILEMENT (2/3)

FONCTION CONCAT

Ex : Paramètres par défaut

df1	index	Letter	Num
	0	a	1
	1	b	2

df2	index	Letter	Num	Pet
	0	c	3	Cat
	1	d	4	Dog

→ df

index	Letter	Num	Pet
0	a	1	NaN
1	b	2	NaN
0	c	3	Cat
1	d	4	Dog

 `df = pd.concat([df1, df2])`

Ex : Paramètre join

df1	index	Letter	Num
	0	a	1
	1	b	2

df2	index	Letter	Num	Pet
	0	c	3	Cat
	1	d	4	Dog

→ df

index	Letter	Num
0	a	1
1	b	2
0	c	3
1	d	4

 `df = pd.concat([df1, df2], join="inner")`

join = « inner » permet de ne conserver que les colonnes présentes dans les dataframes concaténés
join = « outer » permet de conserver toutes les colonnes des différents dataframes concaténés.
(Paramètre par défaut)

Note : il n'y a pas de limite sur le nombre de dataframes à concaténer.

PANDAS

CONCATÉINATION / EMPILEMENT (3/3)

FONCTION APPEND

Ex : Ajout du DataFrame df2 au DataFrame df1

df1

index	Var1	Var2
0	1	2
1	3	4

df2

index	Var1	Var2
0	5	6
1	7	8
2	9	10

→ df

index	Var1	Var2
0	1	2
1	3	4
2	5	6
3	7	8
4	9	10



```
df = df1.append(df2, ignore_index = True)
```


PANDAS

JOINTURE PAR LES INDEX (CONCAT)



FONCTION CONCAT (axis = 1)

Ex : Jointure de df1 et df2 par l'index

df1	index	Var1	df2	index	Var2
	0	0		0	1
	1	1		1	2
	2	0		2	4

↓

index	Var1	Var2
0	0	1
1	1	2
2	0	4



```
df = pd.concat([df1, df2], axis = 1)
```

JOINTURE DE PLUSIEURS DATAFRAMES

df1	index	Var1	df2	index	Var2	df3	index	Var3
	0	0		0	1		1	15
	1	1		1	2		2	9
	2	0		4	4			

↓

index	Var1	Var2	Var3
0	0	1	NaN
1	1	2	15
2	0	NaN	9
4	NaN	4	NaN



```
df = pd.concat([df1, df2, df3], axis = 1)
```

PANDAS

JOINTURES DE TYPE SQL (1/2)



INNER

Ex : Jointure de df1 et df2 selon la clé ID

df1	Var1	ID
	0	1
	1	2
	0	3
	1	4

df2	ID	Var2
	1	1
	2	2
	3	4
	5	6

↓

df	ID	Var1	Var2
	1	0	1
	2	1	2
	3	0	4



```
CREATE TABLE df AS  
SELECT * FROM df1  
INNER JOIN df2 ON df1.ID=df2.ID
```



```
DATA df;  
MERGE df1 (in=a) df2 (in=b);  
BY ID;  
IF a and b;  
RUN;
```

! df1 et df2 doivent être triés



```
df = pd.merge(df1, df2, on="ID", how="inner")
```



La clause where ne peut être utilisée qu'après la jointure

FULL

Ex : Jointure de df1 et df2 selon les clés ID_A et ID_B

df1	Var1	ID_A
	0	1
	1	2
	0	3
	1	4

df2	ID_B	Var2
	1	1
	2	2
	3	4
	5	6

↓

df	ID_A	ID_B	Var1	Var2
	1	1	0	1
	2	2	1	2
	3	3	0	4
	4	4	1	NaN
	5	5	NaN	6



```
CREATE TABLE df AS  
SELECT * FROM df1  
FULL JOIN df2 ON df1.ID_A=df2.ID_B
```



```
DATA df;  
MERGE df1 (in=a) df2 (in=b);  
BY ID;  
RUN;
```

! df1 et df2 doivent être triés



```
df = pd.merge(df1, df2, left_on = 'ID_A', right_on = 'ID_B', how = 'outer')
```

PANDAS

JOINTURES DE TYPE SQL (2/2)



LEFT

Ex : Jointure de df1 et df2 selon la clé ID

df1	Var1	ID
	0	1
	1	2
	0	3
	1	4

ID	Var2
1	1
2	2
3	4
5	6

df2

↓

ID	Var1	Var2
1	0	1
2	1	2
3	0	4
4	1	NaN

df



```
CREATE TABLE df AS  
SELECT * FROM df1  
LEFT JOIN df2 ON df1.ID=df2.ID
```



```
DATA df;  
MERGE df1 (in=a) df2 (in=b);  
BY ID;  
IF a;  
RUN;
```

! df1 et df2 doivent être triés



```
df = pd.merge(df1, df2, on = 'ID', how = 'left')
```

RIGHT

Ex : Jointure de df1 et df2 selon la clé ID

df1	Var1	ID
	0	1
	1	2
	0	3
	1	4

ID	Var2
1	1
2	2
3	4
5	6

df2

↓

ID	Var1	Var2
1	0	1
2	1	2
3	0	4
5	NaN	6

df



```
CREATE TABLE df AS  
SELECT * FROM df1  
RIGHT JOIN df2 ON df1.ID=df2.ID
```



```
DATA df;  
MERGE df1 (in=a) df2 (in=b);  
BY ID;  
IF b;  
RUN;
```

! df1 et df2 doivent être triés



```
df = pd.merge(df1, df2, on = 'ID', how = 'right')
```

PANDAS

COMPLÉMENT SUR LES JOINTURES



➤ Les clés multiples dans la fonction merge

Il est possible de faire des jointures avec la fonction merge sur une clé multiple

Exemple :  `pd.merge(df1, df2, on = ['Var1', 'Var2'], how = 'inner')`

➤ Clés avec des noms différents dans la fonction merge

Dans le cas de jointure sur des clés avec des noms différents, on utilise les paramètres `left_on` et `right_on` de la fonction merge.

Exemple :  `pd.merge(df1, df2, left_on = 'Var1', right_on = 'Var2', how = 'inner')`

Remarque : On peut aussi spécifier des listes de clés de part et d'autre.

➤ La clause WHERE en SQL ne peut se faire qu'après la jointure dans Pandas

Si l'on souhaite faire un filtre, celui-ci doit s'effectuer après la jointure

Exemple :  `df = pd.merge(df1, df2, left_on = 'Var1', right_on = 'Var2', how = 'inner')`
`df = df[df['date_df1'] >= df['date_df2']]`

➤ Mêmes noms de variables dans les deux tables à fusionner

Dans ce cas, vous pouvez utiliser le paramètre `suffixes = ['_left', '_right']` qui ajoutera les chaînes spécifiées à la fin des noms de variables

➤ Si des clés sont dupliquées dans l'une ou les deux tables, il en résulte un produit cartésien

➤ Pour obtenir le produit cartésien de 2 DataFrame, on utilise la fonction merge avec le paramètre `how = 'cross'`

➤ Les jointures qui peuvent être faites sur les index des lignes doivent être réalisées avec la fonction `concat` plutôt qu'avec la fonction `merge` car beaucoup plus rapide

PANDAS

CRÉATION DE NOUVEAUX INDICATEURS (1/4)

```
df = pd.DataFrame({'Col1': pd.Series([1,2,3,4,5,6]),  
                  'Col2': pd.Series([23,4,12,2,0,-10])})
```

```
'''Ajouter une colonne constante'''
```

```
df['Col4'] = "constant"
```

```
'''Ajouter une colonne'''
```

```
df['Col5'] = [7,9,6,6,19,44] # Attention le nombre d'éléments de la liste  
                           # doit être égal au nombre de lignes du dataframe
```

```
'''Supprimer une colonne'''
```

```
del df['Col1']
```

```
print(df)
```

#	Col2	Col3	Col4	Col5
#5	Oui	-10	constant	7
#4	Non	0	constant	9
#3	Oui	2	constant	6
#1	Non	4	constant	6
#2	Oui	12	constant	19
#0	Oui	23	constant	44

➤ Fonctions sur les dataframe :

1. Création de la fonction f

2. Application de la fonction :

- Sur l'ensemble de la table : `df.applymap(lambda x : f(x))` → ici x représente une cellule du Dataframe
- Sur l'ensemble des colonnes : `df.apply(lambda x : f(x))` → ici x représente une colonne
- Sur l'ensemble des lignes : `df.apply(lambda x : f(x), axis = 1)` → ici x représente une ligne
- Sur une Serie : `df['col'].map(lambda x : f(x))` → ici x représente une valeur de la Serie

➤ Exemple :

```
df = pd.DataFrame({'Col1': pd.Series([1,2,3,4,5,6]),
                  'Col2': pd.Series([23,4,12,2,0,-10])})
```

```
df.applymap(lambda x : x+3)
```

```
#   Col1  Col2
#0     4    26
#1     5     7
#2     6    15
#3     7     5
#4     8     3
#5     9    -7
```

```
df.apply(lambda x : x.mean())
```

```
#Col1    3.500000
#Col2    5.166667
#dtype: float64
```

```
df['Col1'].map(lambda x : x +3)
```

```
#0     4
#1     5
#2     6
#3     7
#4     8
#5     9
```

```
#Name: Col1, dtype: int64
```

➤ Fonctions sur les dataframe :

Il est également possible d'utiliser la fonction `map` pour appliquer un dictionnaire de transformation

➤ Exemple :

- Création du dictionnaire de transformation à partir du fichier CSV :



```
import ssl; import pandas as pd; import numpy as np
ssl._create_default_https_context = ssl._create_unverified_context
url_depts = 'https://www.insee.fr/fr/statistiques/fichier/3720946/departement2019-csv.zip'
DEPTS = pd.read_csv(url_depts, dtype = 'str')
```

```
dic_DEPTS = DEPTS.set_index('dep').to_dict()['libelle']
```

RQ : Pour mettre une colonne en index, on utilise la fonction `set_index`

- Application du dictionnaire de transformation :

DEPT		DEPT	LIBELLE_DEPT
10	→	10	Aube
NaN		NaN	NaN
976		976	Mayotte
2A		2A	Corse-du-Sud
78		78	Yvelines
33		33	Gironde
31		31	Haute-Garonne



```
df = pd.DataFrame({'DEPT' : ['10', np.nan, '976', '2A', '78', '33', '31']})
```

```
df['LIBELLE_DEPT'] = df.DEPT.map(dic_DEPTS)
```

or

```
df['LIBELLE_DEPT'] = df.applymap(dic_DEPTS.get)
```



Avec cette méthode, ce n'est pas `NaN` qui est retourné en sortie mais `None` sur la ligne 2

CRÉATION DE NOUVEAUX INDICATEURS (4/4)

```
df = pd.DataFrame({'Ville':['Paris','Londres','Manchester','Paris'],
                  'Statut':['Actif','Actif','Etudiant','Retraité'],
                  'Age':[45,56,19,78], 'Sexe':['H','F','F','H'],
                  'Nb_annees_travail' : [20,29,0,43]})
```

```
'''np.where'''
```

```
df['Travailleur'] = pd.DataFrame(np.where(df['Statut']=='Actif',1,0),
                                index = df.index)
```

```
df['Ville_statut'] = df['Ville'] + "_" + df['Statut']
```

```
df['Temps restant à travailler'] = 43 - df['Nb_annees_travail']
```

```
df['Proportion de temps passé au travail'] = df['Nb_annees_travail']/df['Age']
```

Indice	Ville	Statut	Age	Sexe	Nb_annees_travail	Travailleur	Ville_statut	Temps restant à travailler	Proportion de temps passé au travail
0	Paris	Actif	45	H	20	1	Paris_Actif	23	0.444444
1	Londres	Actif	56	F	29	1	Londres_Actif	14	0.517857
2	Manchester	Etudiant	19	F	0	0	Manchester_Etudiant	43	0
3	Paris	Retraité	78	H	43	0	Paris_Retraité	0	0.551282

PANDAS

FONCTIONS DE DATA MANAGEMENT

VARIABLES CATÉGORIELLES TRANSFORMATION

Binarization / Dummisation :

Var1	
A	
B	
C	

 →

Var1A	Var1B	Var1C
1	0	0
0	1	0
0	0	1

```
df_dummies = pd.get_dummies(df.Var1)
```

Label Encoding :

Var1	
A	
B	
C	

 →

Var1
1
2
3

Cette méthode encode chaque catégorie comme un entier dans l'ordre croissant

```
df_label = pd.factorize(df.Var1)
```

VARIABLES NUMÉRIQUES TRANSFORMATION

Fonction cut :

index	Var1
a	24
b	7
c	2
d	25
e	22
f	29

 →

index	Var1
a	(20.0, 29.0]
b	(1.973, 11.0]
c	(1.973, 11.0]
d	(20.0, 29.0]
e	(20.0, 29.0]
f	(20.0, 29.0]

```
pd.cut(df['Var1'], 3)
```

La commande `cut` crée des classes amplitudes égales mais de fréquences inégales

Fonction qcut :

index	Var1
a	24
b	7
c	2
d	25
e	22
f	29

 →

index	Var1
a	(17.0, 24.333]
b	(1.999, 17.0]
c	(1.999, 17.0]
d	(24.333, 29.0]
e	(17.0, 24.333]
f	(24.333, 29.0]

```
pd.qcut(s['Var1'], 3)
```

La commande `qcut` crée des classes fréquences égales mais d'amplitudes inégales

AGRÉGATS CALCULÉS SUR DES FENÊTRES TEMPORELLES GLISSANTES

ID_maison	date_vente	ville	Prix_m2
0	15/01/2021	Paris	15 000
1	10/01/2021	Paris	9 000
2	13/01/2021	Lyon	4 000
3	01/01/2021	Paris	8 000
4	15/01/2021	Lyon	6 000
5	12/01/2021	Paris	13 000



ID_maison	date_vente	ville	Prix_m2
2	13/01/2021	Lyon	4 000
4	15/01/2021	Lyon	6 000
3	01/01/2021	Paris	8 000
1	10/01/2021	Paris	9 000
5	12/01/2021	Paris	13 000
0	15/01/2021	Paris	15 000



Il est important de trier au préalable le DataFrame, au moins sur les dates



```
df['date_vente'] = pd.to_datetime(df['date_vente'], format = '%d/%m/%Y')
df.sort_values(by = ['ville', 'date_vente'], inplace = True)
```

ID_maison	ville	Prix_m2
2	Lyon	4 000
4	Lyon	5 000
3	Paris	8 000
1	Paris	8 500
5	Paris	11 000
0	Paris	12 333

1	10/01/2021	Paris	9 000
5	12/01/2021	Paris	13 000
0	15/01/2021	Paris	15 000



ID_maison	ville	Prix_m2
2	Lyon	NaN
4	Lyon	4 000
3	Paris	NaN
1	Paris	8 000
5	Paris	9 000
0	Paris	11 000

1	10/01/2021	Paris	9 000
5	12/01/2021	Paris	13 000



```
df_var_ind = df.groupby('ville').rolling('10d', on = 'date_vente',
closed = 'both').mean()['Prix_m2'].reset_index()
.rename(columns = {'level_1': 'ID_maison'})
```



```
df_var_ind = df.groupby('ville').rolling('10d', on = 'date_vente',
closed = 'left').mean()['Prix_m2'].reset_index()
.rename(columns = {'level_1': 'ID_maison'})
```

AGRÉGATS CALCULÉS SUR LES N DERNIÈRES VALEURS DISPONIBLES

ID_maison	date_vente	ville	Prix_m2
0	15/01/2021	Paris	15 000
1	10/01/2021	Paris	9 000
2	13/01/2021	Lyon	4 000
3	01/01/2021	Paris	8 000
4	15/01/2021	Lyon	6 000
5	12/01/2021	Paris	13 000



ID_maison	date_vente	ville	Prix_m2
2	13/01/2021	Lyon	4 000
4	15/01/2021	Lyon	6 000
3	01/01/2021	Paris	8 000
1	10/01/2021	Paris	9 000
5	12/01/2021	Paris	13 000
0	15/01/2021	Paris	15 000



Il est important de trier au préalable le DataFrame, au moins sur les dates



```
df['date_vente'] = pd.to_datetime(df['date_vente'], format = '%d/%m/%Y')
df = df.sort_values(by = ['date_vente', 'ville'])
```



Le paramètre `closed` de la fonction `rolling` n'est disponible pour les fenêtres fixes qu'à partir de la version 1.2.0 de Pandas

ID_maison	ville	Prix_m2
2	Lyon	NaN
4	Lyon	5 000
3	Paris	NaN
1	Paris	8 500
5	Paris	11 000
0	Paris	14 000

5	12/01/2021	Paris	13 000
0	15/01/2021	Paris	15 000



ID_maison	ville	Prix_m2
2	Lyon	NaN
4	Lyon	NaN
3	Paris	NaN
1	Paris	NaN
5	Paris	8 500
0	Paris	11 000

1	10/01/2021	Paris	9 000
5	12/01/2021	Paris	13 000



```
df_var_ind = df.groupby('ville').rolling(2, on = None,
closed = None).mean()['Prix_m2'].reset_index()
.rename(columns = {'level_1': 'ID_maison'})
```



```
df_var_ind = df.groupby('ville').rolling(2, on = None,
closed = left).mean()['Prix_m2'].reset_index()
.rename(columns = {'level_1': 'ID_maison'})
```

01

Les principaux environnements de développement Python

02

Les concepts de base en Python

03

Introduction aux packages NUMPY et SCIPY

04

Import et préparation des données avec Pandas

05

Statistiques descriptives

06

Graphiques avec Matplotlib

STATISTIQUES DESCRIPTIVES

#Générer le dataframe

```
import numpy as np; import pandas as pd
```

```
np.random.seed(123) # reproductivité des résultats
```

```
df_f = pd.DataFrame({'SEXE':['F']*45, 'AGE':np.random.randint(18,105,45), 'TAILLE':np.random.randint(145,185,45)})
```

```
df_h = pd.DataFrame({'SEXE':['H']*45, 'AGE':np.random.randint(18,95,45), 'TAILLE':np.random.randint(165,205,45)})
```

```
df_aut = pd.DataFrame({'SEXE':[None]*10, 'AGE':np.random.randint(18,105,10), 'TAILLE':np.random.randint(145,205,10)})
```

```
df = pd.concat([df_f,df_h,df_aut],axis=0,ignore_index=True) # concaténation des trois dataframes
```

```
df['CAT_TAILLE'] = np.where(df['TAILLE']>=175, 'GRAND', 'PETIT')
```

#Aperçu du dataframe

```
df.head(5) # 5 premières lignes du dataframe
```

	SEXE	AGE	TAILLE	CAT_TAILLE
0	F	84	175	GRAND
1	F	35	151	PETIT
2	F	101	171	PETIT

Etude du dataframe dans son ensemble

```
len(df) # nombre total d'observations
```

100

```
df.isnull().sum() # nombre de valeurs manquantes par colonne
```

SEXE	10
AGE	0
TAILLE	0
CAT_TAILLE	0
dtype:	int64

STATISTIQUES DESCRIPTIVES

VARIABLES NUMÉRIQUES

➤ Par variable :

`df.describe()` #présente les principales stats pour chaque champ numérique

	AGE	TAILLE
count	100.000000	100.000000
mean	61.820000	172.920000
std	25.629205	15.751017
min	18.000000	146.000000
25%	42.500000	161.750000
50%	64.000000	171.500000
75%	84.000000	184.000000
max	104.000000	203.000000

`df.AGE.mean()` # identique à `df['AGE'].mean()`
`df.AGE.min()`
`df.AGE.max()`
`.var()` pour variance, `.median()`, `.sum()` pour la somme

61.82
18
104

➤ Croisement de variables :

`df.groupby('SEXE')['TAILLE'].mean()` # moyenne par SEXE pour POIDS

SEXE	
F	162.955556
H	182.977778

Name: TAILLE, dtype: float64

`df.groupby('SEXE').mean()` #moyenne pour toutes les var numériques
#(sauf celles dans le GROUP BY)

SEXE	AGE	TAILLE
F	68.488889	162.955556
H	57.177778	182.977778

`df.groupby(['SEXE', 'CAT_TAILLE'])['TAILLE'].mean()`

SEXE	CAT_TAILLE	
F	GRAND	180.500000
F	PETIT	157.942857
H	GRAND	189.032258
H	PETIT	169.571429

Name: TAILLE, dtype: float64

STATISTIQUES DESCRIPTIVES

VARIABLES CATÉGORIELLES

➤ Par variable :

```
df.SEXE.value_counts() # tri à plat pour la variable SEXE
```

```
H    45
F    45
Name: SEXE, dtype: int64
```

```
df.SEXE.value_counts(dropna=False) # dropna = False permet d'intégrer
# les valeurs manquantes
```

```
H    45
F    45
NaN   10
Name: SEXE, dtype: int64
```

```
df.SEXE.str.len() # calcule le nombre de caractères pour
# chaque observation pour la variable SEXE
```

```
0    1.0
1    1.0
...
98   NaN
99   NaN
Name: SEXE, Length: 100, dtype: float64
```

➤ Croisement de variables :

```
pd.crosstab(df.SEXE,df.CAT_TAILLE) # tableau croisé
```

CAT_TAILLE	GRAND	PETIT
SEXE		
F	10	35
H	31	14

```
pd.crosstab(df.SEXE.fillna('Non Renseigné'), df.CAT_TAILLE)
# .fillna('Non Renseigné') remplace les valeurs manquantes
# par la modalité 'Non Renseigné'
```

CAT_TAILLE	GRAND	PETIT
SEXE		
F	10	35
H	31	14
Non Renseigné	4	6

```
pd.crosstab(df.SEXE.fillna('Non Renseigné'),df.CAT_TAILLE, margins=True)
# margins = True pour intégrer les totaux
```

CAT_TAILLE	GRAND	PETIT	All
SEXE			
F	10	35	45
H	31	14	45
Non Renseigné	4	6	10
All	45	55	100

01

Les principaux environnements de développement Python

02

Les concepts de base en Python

03

Introduction aux packages NUMPY et SCIPY

04

Import et préparation des données avec Pandas

05

Statistiques descriptives

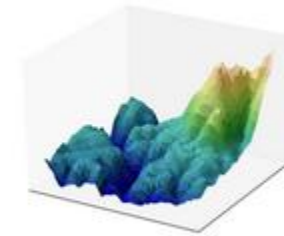
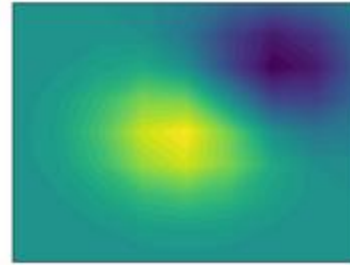
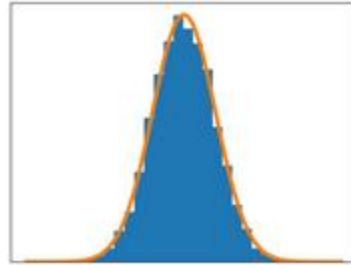
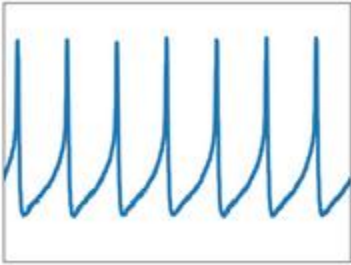
06

Graphiques avec Matplotlib

GRAPHIQUES AVEC MATPLOTLIB

PRÉSENTATION DU MODULE

➤ Ce package permet de réaliser des graphiques avec Python



➤ Un grand nombre de types de graphiques sont disponibles

➤ Consultez <https://matplotlib.org/3.1.0/gallery/index.html> pour avoir la liste exhaustive

GRAPHIQUES AVEC MATPLOTLIB

APPROCHE FONCTIONNELLE VS APPROCHE ORIENTÉE OBJET

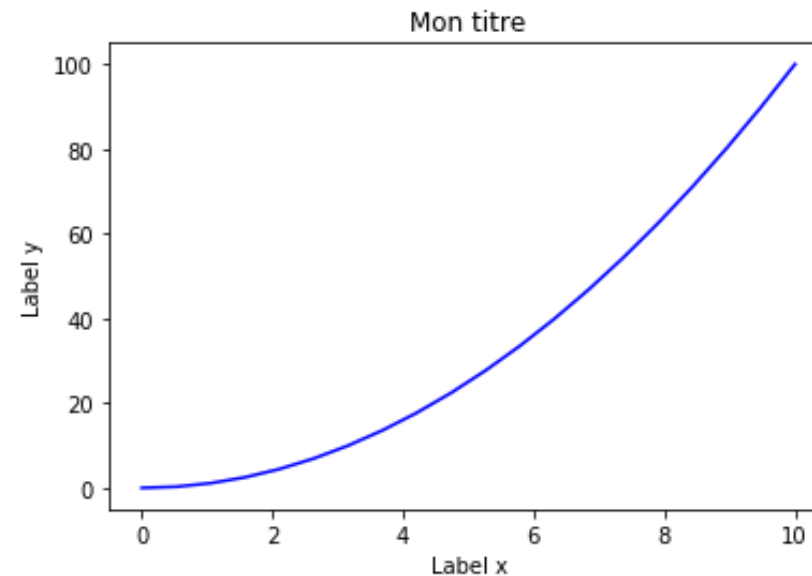
```
import numpy as np
import matplotlib.pyplot as plt
x= np.linspace(0,10,20) # génère 20 points entre 0 et 10
y = x**2
```

➤ Approche fonctionnelle :

```
plt.title('Mon titre') # ajouter un titre
plt.xlabel('Label x') # ajouter un label à l'axe x
plt.ylabel('Label y') # ajouter un label à l'axe y
plt.plot(x,y,'blue') #tracer la courbe (en bleu)
```

➤ Approche orientée objet :

```
fig, ax = plt.subplots() # génère le graphique vide
ax.set_title('Mon titre') # ajouter un titre
ax.set_xlabel('Label x') # ajouter un label à l'axe x
ax.set_ylabel('Label y') # ajouter un label à l'axe y
ax.plot(x,y,'blue') #tracer la courbe (en bleu)
```



➤ L'approche orientée objet est à privilégier pour réaliser des graphiques complexes.

GRAPHIQUES AVEC MATPLOTLIB

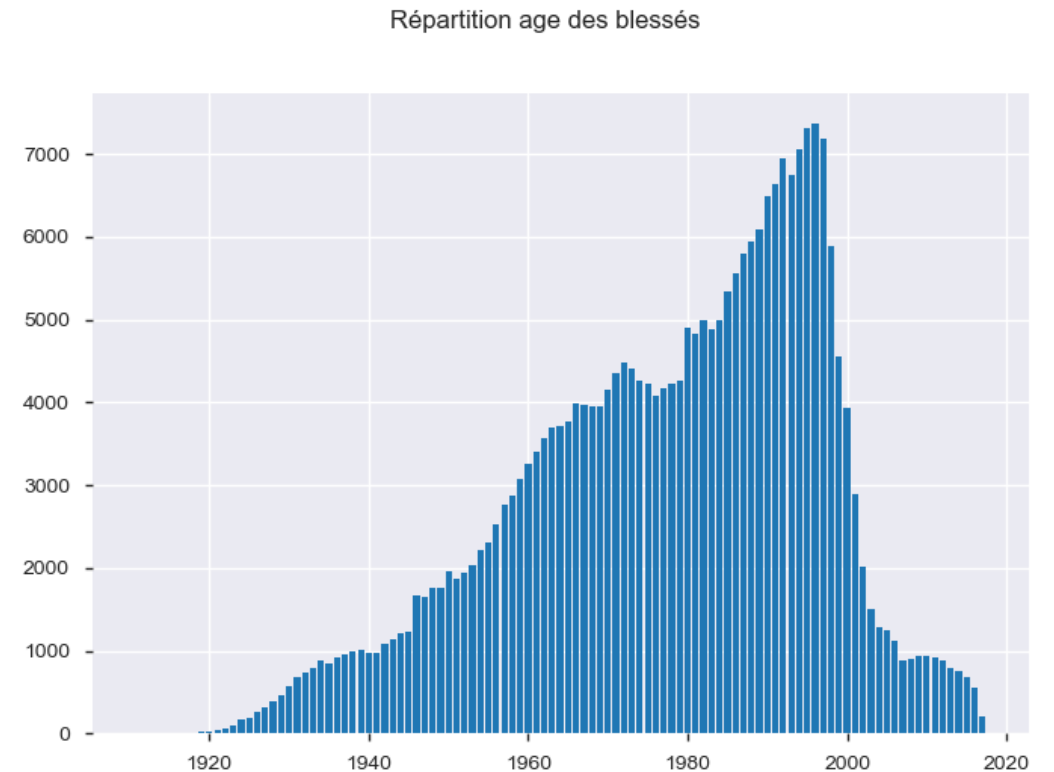
EXEMPLE

- Nous avons d'abord construit un Data frame comptant le nombre de blessés par tranche d'âge.
- La fonction « bar » permet de tracer des graphiques en barre.
- La fonction « subplots » permet de tracer plusieurs graphiques.

```
import matplotlib
import matplotlib.pyplot as plt

# Comptage du nombre de blessé par âge
an = usagers.groupby('an_nais').size().reset_index()
an.columns = ["an_nais" , "nb_usagers"]

#Tracé du graphique
fig, axs = plt.subplots()
axs.bar(an["an_nais"],an["nb_usagers"])
fig.suptitle('Répartition age des blessés')
```

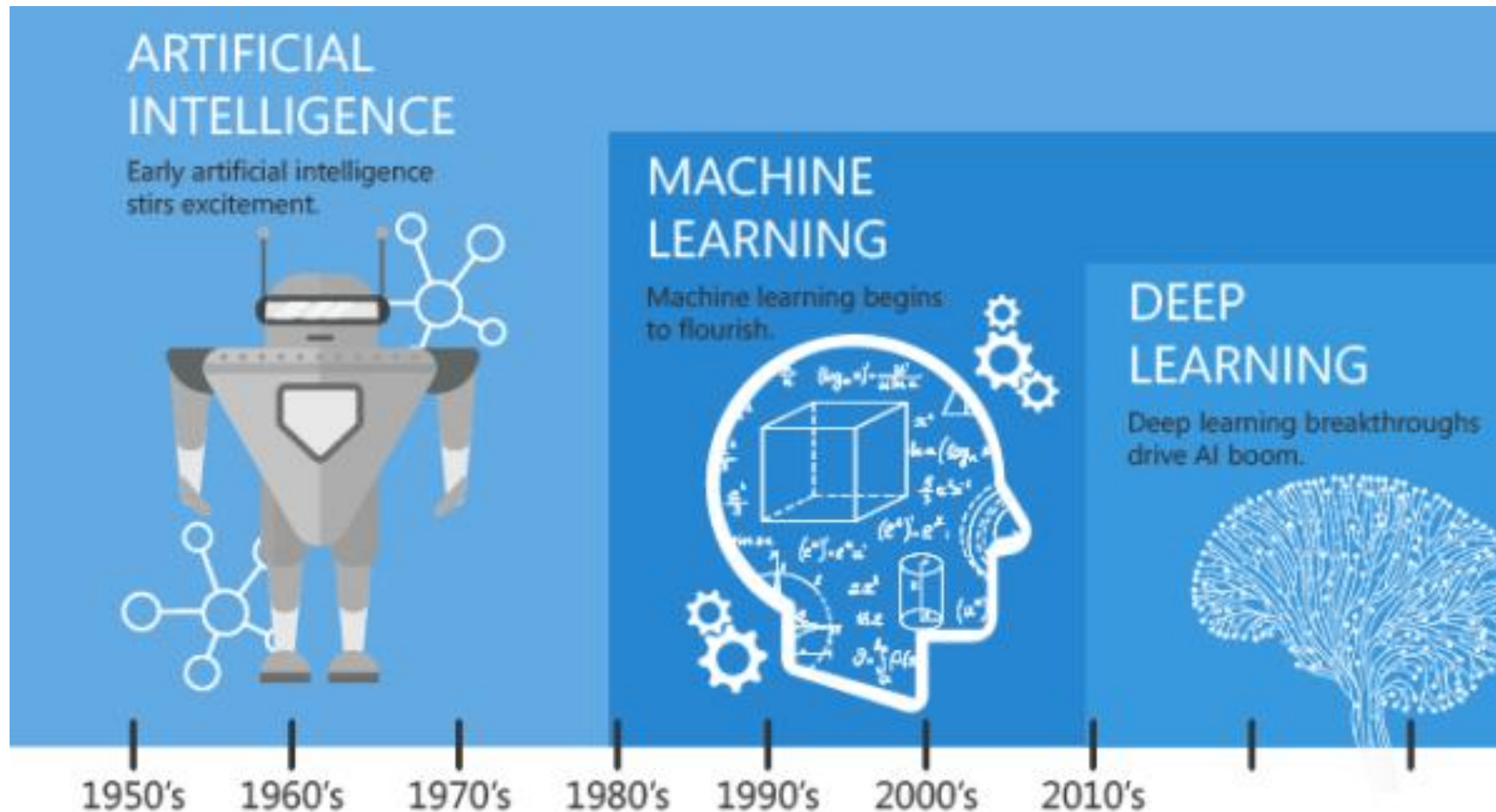


JOURS 3 & 4 :

***DATA PREPROCESSING &
MACHINE LEARNING***



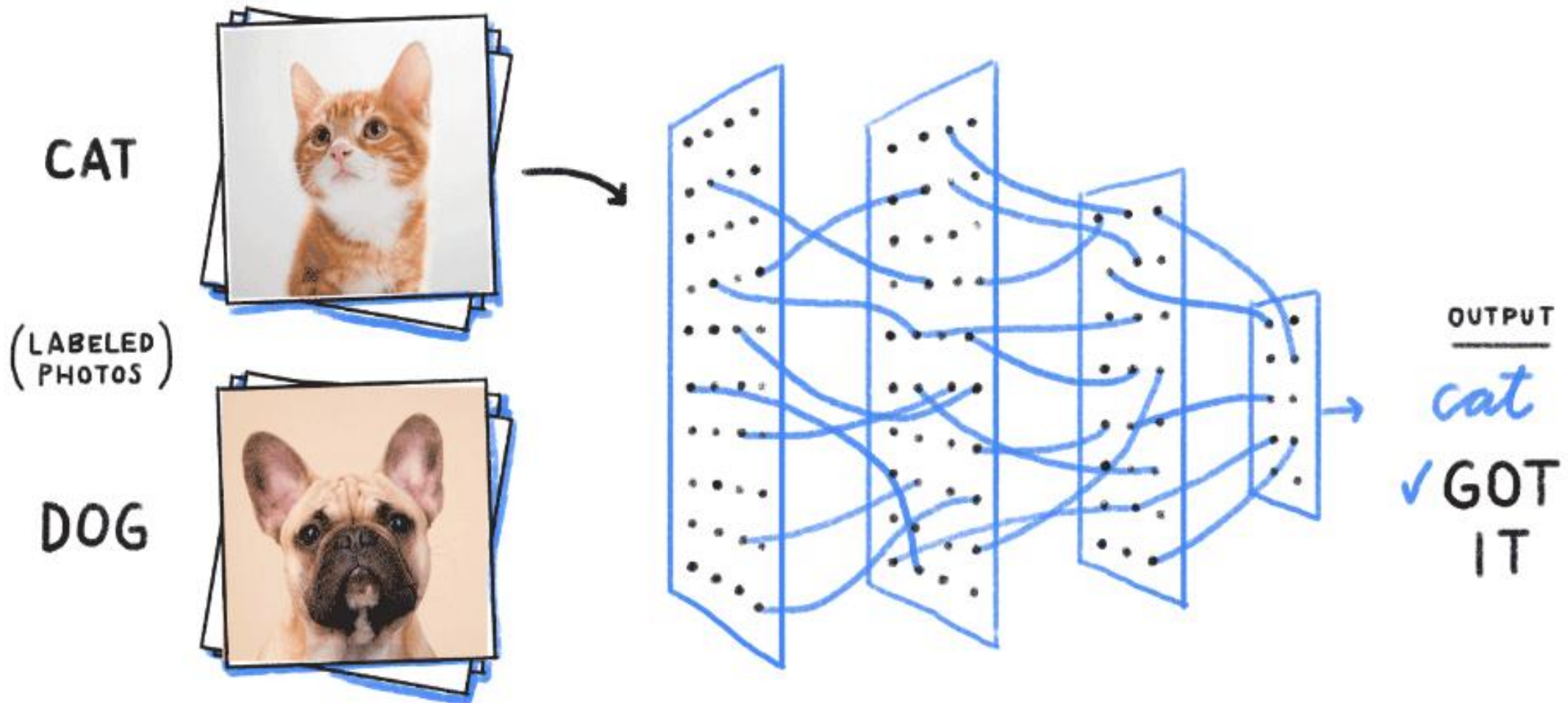
FOCUS SUR L'IA & LE MACHINE LEARNING



L'IA s'intéresse à l'ensemble des facultés cognitives humaines :
perception, apprentissage, raisonnement, expression, action/décision

MACHINE LEARNING

SOUS-DISCIPLINE DE L'IA PRINCIPALEMENT UTILISÉE POUR L'APPRENTISSAGE



MACHINE LEARNING

SOUS-DISCIPLINE DE L'IA PRINCIPALEMENT UTILISÉE POUR L'APPRENTISSAGE

Apprentissage : acquérir des connaissances à partir d'exemples

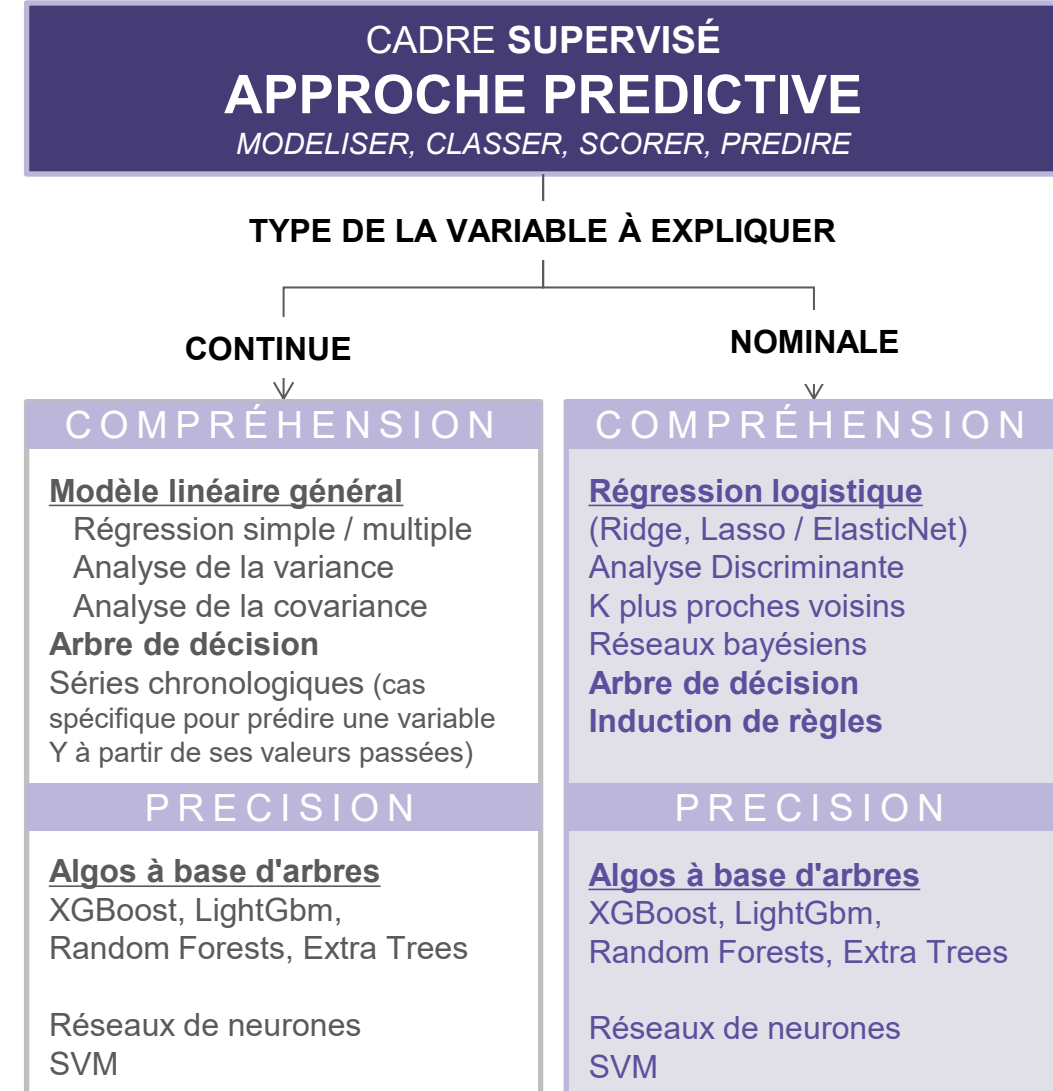
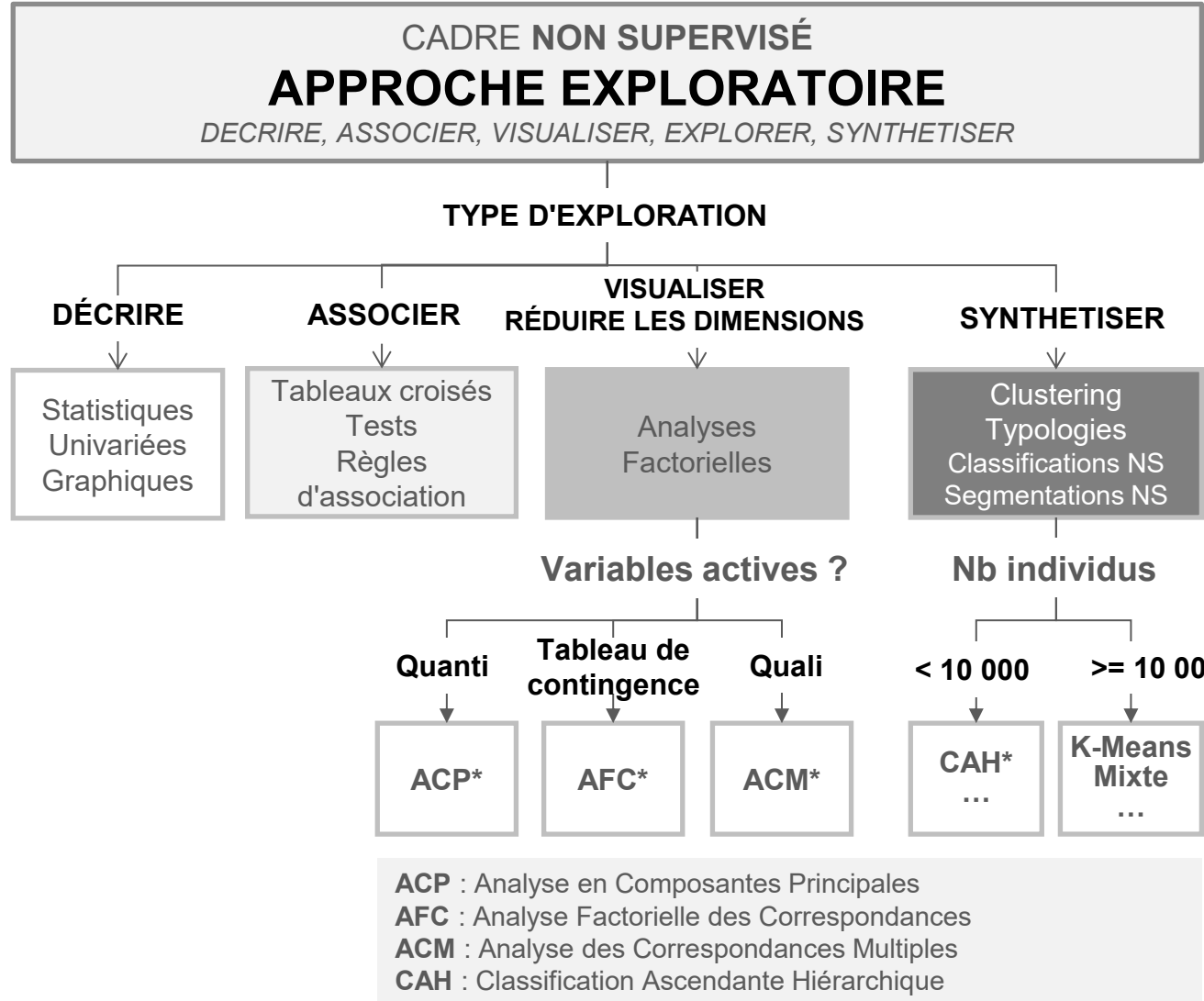
- apprendre à reconnaître des visages à partir de photos étiquetées
- apprendre à identifier des dossiers frauduleux à partir d'un historique de dossiers étiquetés (Fraude / Non Fraude)
- capacité à apprendre à partir de données (du passé) pour :
 - identifier des formes, des ressemblances / corrélations,
 - Identifier des groupes aux comportements / profils similaires
 - faire et/ou automatiser des diagnostics d'experts
 - classifier
 - prédire

Déduction - Prédiction :

- Stocker les connaissances acquises lors de l'apprentissage pour pouvoir les appliquer à de nouvelles données (images, dossiers, clients, transactions...) et donc prédire

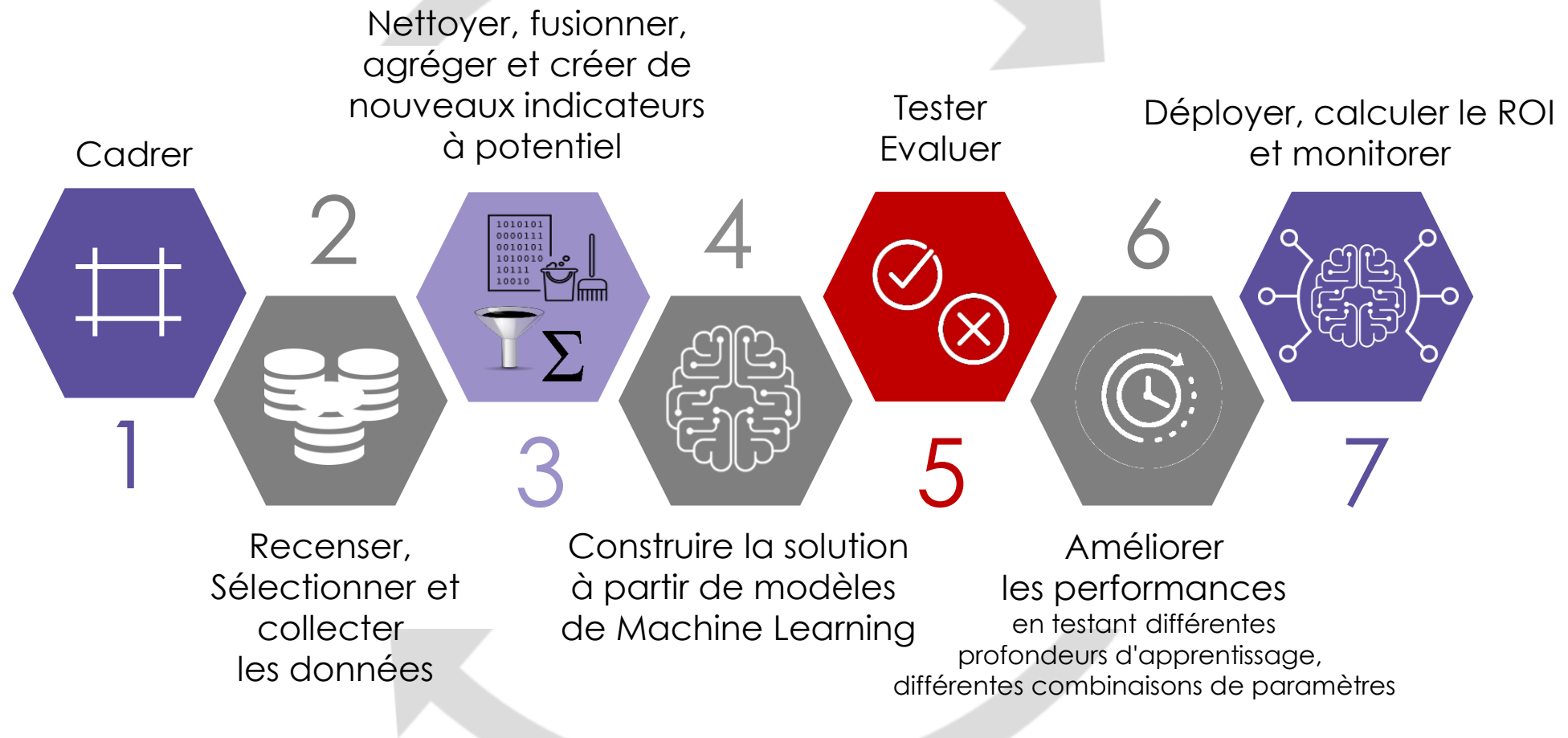
MACHINE LEARNING

DEUX APPROCHES COMPLÉMENTAIRES



MACHINE LEARNING

UNE DÉMARCHE MINUTIEUSE ET ITÉRATIVE

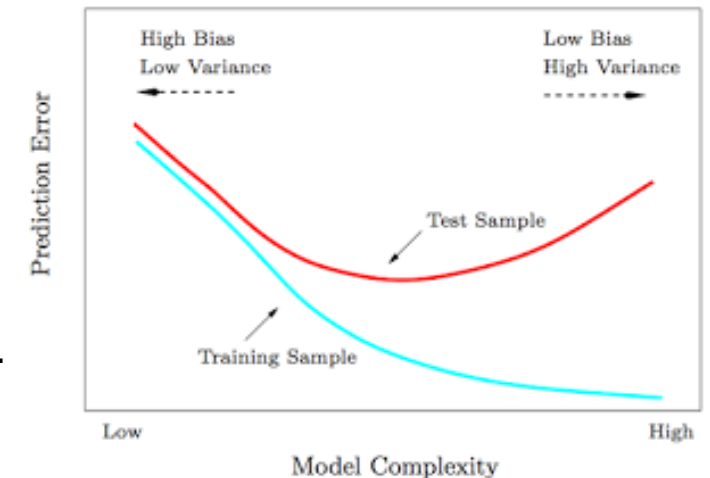


ANALYSE PRÉDICTIVE

LE DILEMME BIAIS-VARIANCE

En statistique et en apprentissage automatique, le dilemme (ou compromis) biais–variance est le problème qui consiste à **minimiser simultanément deux sources d'erreurs qui empêchent les algorithmes d'apprentissage supervisé de généraliser au-delà de leur échantillon d'apprentissage** :

- **Le biais est l'erreur de prédiction.** Un biais élevé peut être lié à un algorithme qui manque de relations pertinentes entre les données en entrée et les sorties prévues (sous-apprentissage).
- **La variance est l'erreur due à la sensibilité aux petites fluctuations de l'échantillon d'apprentissage.**
- Les modèles simples présentent un biais fort mais une faible variance.
- Les modèles complexes présentent un biais faible mais une variance forte.



La décomposition biais-variance est une façon d'analyser l'espérance de l'erreur de prédiction d'un algorithme d'apprentissage d'un problème particulier comme une somme de trois termes :

- le biais, la variance et une quantité, appelée erreur irréductible, résultant du bruit dans le problème lui-même.

SCORING

DÉFINITION

Cadre : modélisation d'un évènement binaire

- **Fraude** (Oui / Non)
- **Défaut** (Oui / Non)

Qu'est ce qu'un score ?

- **Une note** donnée à un client, un dossier, une entreprise
- **Sur une échelle** de 0 à 1 (ou 0 à 1000)
- Cette note **reflète la probabilité de survenance de l'évènement que l'on modélise**

2 types d'utilisation :

- **Ciblage : sélection** des dossiers / entreprises présentant les scores les plus élevés pour action spécifique (Contrôle, demande de justificatifs)
- **Classement / Décision** : selon la note de score obtenue, classement dans une catégorie (sans risque / risque moyen / risque élevé) et décision adaptée selon la catégorie (dossier accepté / dossier à compléter / dossier refusé)

La Prédiction

- On applique le score à de nouvelles données déclaratives pour estimer la probabilité de l'évènement modélisé. On classe ou on hiérarchise pour le ciblage.

SCORING – EXEMPLE SUR LA LUTTE CONTRE LA FRAUDE FISCALE

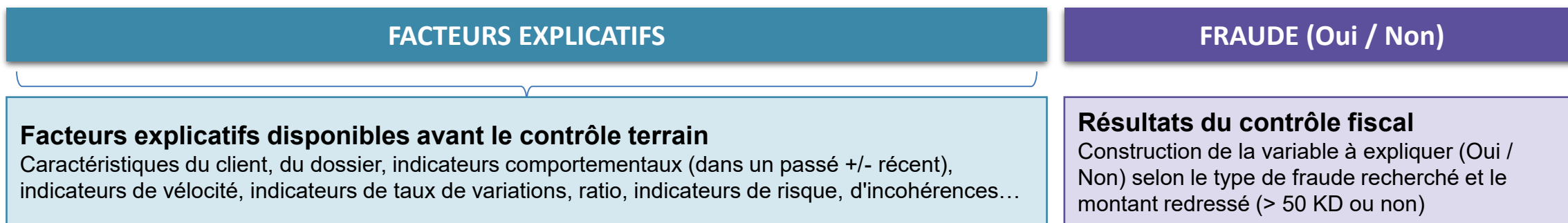
OBJECTIF ET MÉTHODOLOGIE

Objectif : améliorer le ciblage des dossiers pour la programmation

- Concentrer ses efforts sur les dossiers dont la probabilité de fraude est maximale

On apprend à partir des données des contrôles passés pour :

- identifier de futurs cas suspects
- mais aussi revenir sur des cas suspects qui n'ont pas fait l'objet de contrôles et qu'il est encore temps de contrôler



- **Démarche**

- Cadrage - Définition du périmètre de la fraude étudiée et des entreprises éligibles
- Prise en charge des données, data management, analyse descriptive
- Préparation des données pour la modélisation – Recherche et construction de variables prédictives
- Modélisations
- Evaluation statistique de la capacité de généralisation des modèles et choix du modèle « final »
- Scoring des dossiers non contrôlés et sélection d'une liste pour évaluation quali du bureau par un expert métier
- Bilan et décision d'utiliser le modèle pour la programmation
- Déploiement, Monitoring et documentation

01

Introduction à Scikit-Learn

02

Principaux concepts de Scikit-Learn

03

Partitionnement des données

04

Préprocessing

05

Fonctions avancées pour la gestion des variables catégorielles

06

Régressions logistiques (ridge, lasso, elasticnet)

07

Algorithmes à base d'arbres

08

Éléments pour l'interprétation

09

Sélection de variables explicatives

10

Optimisation des hyperparamètres

11

Métriques d'évaluation de la performance des modèles

INTRODUCTION À SCIKIT-LEARN



- Créée en 2007, Scikit-Learn est l'une des librairies open-source les plus populaires pour le Machine Learning / Data Science en Python
- Développée et contrôlée par l'équipe Parietal rattachée à l'INRIA (Institut National de Recherche en Informatique et en Automatique)
- En plus de l'INRIA, beaucoup de chercheurs/étudiants en thèse participent à la construction de l'outil dans le monde entier
- La librairie est construite à partir de Numpy / Matplotlib / Scipy et est compatible avec des dataframes de la librairie Pandas et des matrices sparse (de numpy)
- Scikit-Learn rend accessible facilement tous les principaux algorithmes de ML :
 - Classification supervisée
 - Clustering
 - Régression

Mais contient également de nombreux modules pour :

- Réduction de dimensions
- Préprocessing des données
- Evaluation des modèles

INTRODUCTION À SCIKIT-LEARN

SCIKIT-LEARN



- Solide boîte à outils pour les statistiques et le Machine Learning
- Très grand panel d'algorithmes de Machine Learning
- Développement régulier et actif du package

Principaux outils :



PREPROCESSING

Extraction des variables, normalisation, dummisation



DIMENSIONALITY REDUCTION

Réduction du nombre de variables



CLUSTERING

Regroupement des observations similaires



CLASSIFICATION

Prédiction d'une variable cible catégorielle



REGRESSION

Prédiction d'une variable quantitative

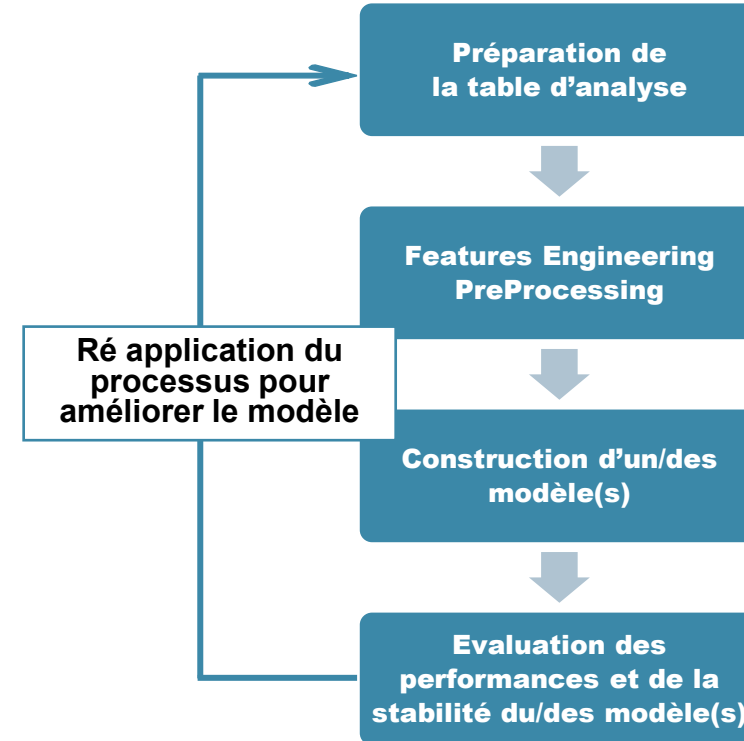


MODEL SELECTION

Validation des performances du modèle et sa robustesse

MACHINE LEARNING WORKFLOW

Pour construire un modèle efficace de Machine Learning, le Data Scientist doit contrôler les différentes étapes du processus de Machine Learning.



01

Introduction à Scikit-Learn

02

Principaux concepts de Scikit-Learn

03

Partitionnement des données

04

Préprocessing

05

Fonctions avancées pour la gestion
des variables catégorielles

06

Régressions logistiques (ridge, lasso,
elasticnet)

07

Algorithmes à base d'arbres

08

Éléments pour l'interprétation

09

Sélection de variables explicatives

10

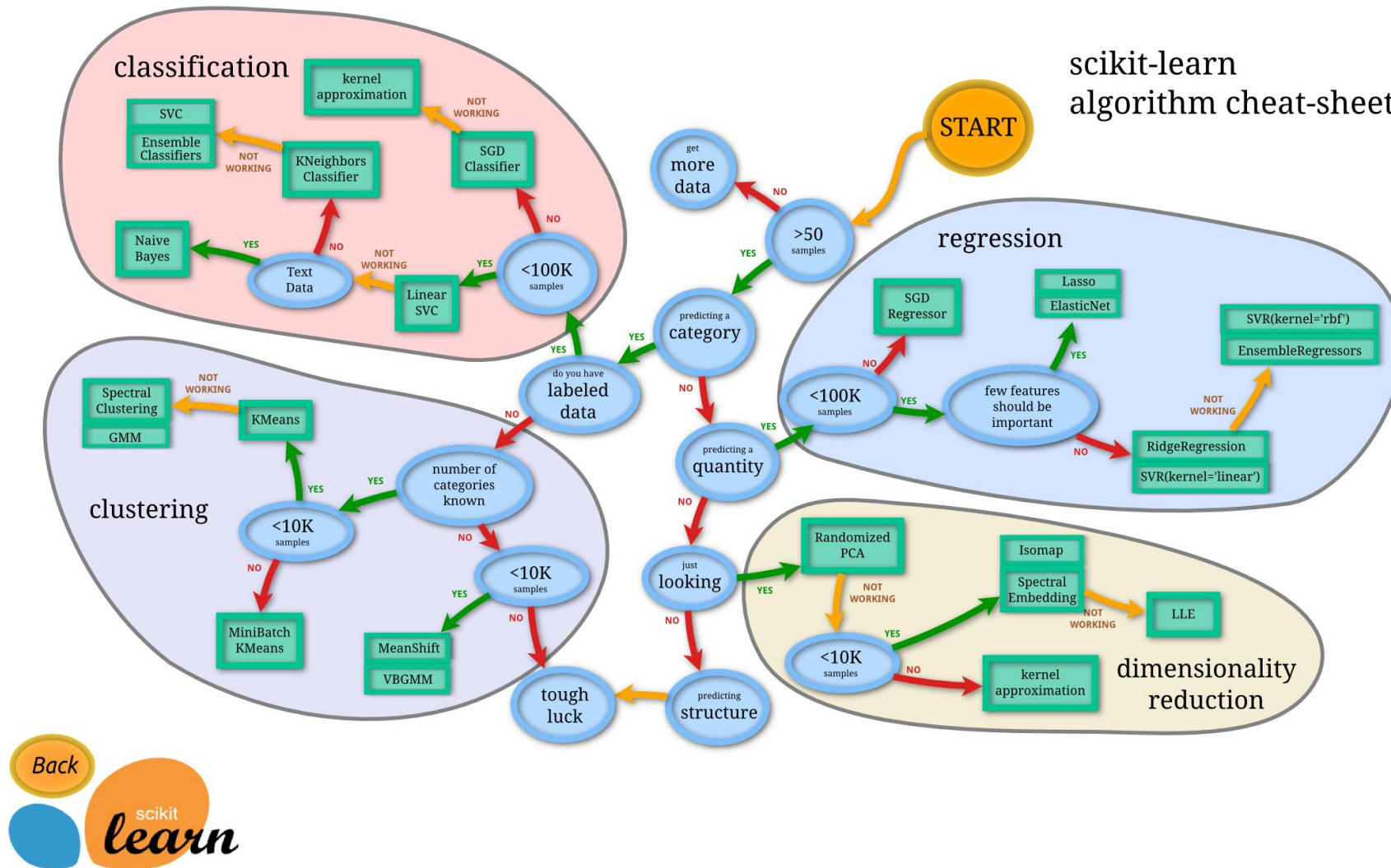
Optimisation des hyperparamètres

11

Métriques d'évaluation de la
performance des modèles

PRINCIPAUX CONCEPTS DE SCIKIT-LEARN

CHEAT SHEET ML



PRINCIPAUX CONCEPTS DE SCIKIT-LEARN

INTERFACE UNIFORME

➤ Scikit-Learn s'efforce d'avoir une interface uniforme pour tous les algorithmes

➤ La plupart des opérations vont être réalisées à l'aide d'objets de type estimator
(par exemple : `model = LinearRegression(normalize=True)` est un estimator `model`)

➤ Etant donné un objet estimator `model`, les méthodes suivantes peuvent être utilisées :

- Pour tous les estimators :
 - `model.fit()` : lance le mécanisme d'apprentissage
 - `model.fit(X, y)` pour les algorithmes supervisés (X : variables explicatives, Y : variable cible)
 - `model.fit(X)` pour les algorithmes non-supervisés
- Les estimators supervisés
 - `model.predict(X_new)` : prédit et retourne les labels d'un nouveau jeu de données `X_new`
 - `model.predict_proba(X_new)` : pour les algorithmes de classification, prédit et retourne les probabilités d'appartenance à une classe d'un nouveau jeu de données `X_new`
 - `model.score(X_test, y_test)` : Retourne un score de performance du modèle entre 0 (mauvais) et 1 (idéal mais suspicieux (R^2 pour la régression par exemple))
- Les estimators non-supervisés
 - `model.transform(X_new)` : transforme un jeu de données selon le modèle
 - `model.fit_transform(X)` : apprend ses paramètres grâce à un jeu de données et transforme celui-ci.

01

Introduction à Scikit-Learn

02

Principaux concepts de Scikit-Learn

03

Partitionnement des données

04

Préprocessing

05

Fonctions avancées pour la gestion des variables catégorielles

06

Régressions logistiques (ridge, lasso, elasticnet)

07

Algorithmes à base d'arbres

08

Éléments pour l'interprétation

09

Sélection de variables explicatives

10

Optimisation des hyperparamètres

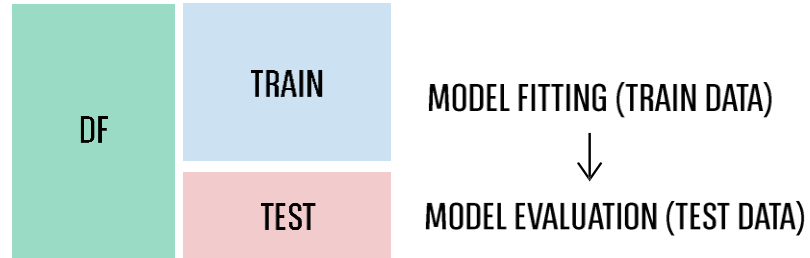
11

Métriques d'évaluation de la performance des modèles

PARTITIONNEMENT DES DONNÉES

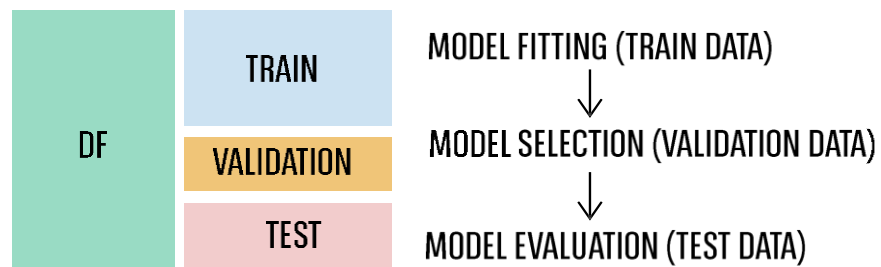
VALIDATION CROISEE (SPLIT SIMPLE)

- Le dataset doit être découpé en 2 parties :



- Le Test est utilisé uniquement pour évaluer la capacité du modèle à prédire correctement sur de nouvelles données

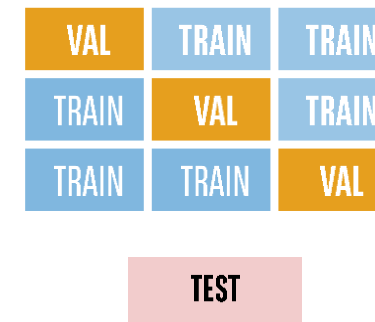
VALIDATION CROISEE (SPLIT EN 3 ÉCHANTILLONS)



- Dans le cas précédent, il est fréquent d'utiliser l'échantillon Test de façon trop intensive (revenant à faire du sur-apprentissage sur cet échantillon également). Dans ce cas, il est préférable d'utiliser un échantillon de validation et de n'appliquer le modèle à l'échantillon test qu'à la toute fin des traitements

K-FOLD CV

3-FOLD CV



- Le Dataset est divisé en 3 parts égales
- A chaque étape, le modèle est entraîné sur 2/3 et évalué sur 1/3
- Le modèle optimal est choisi à partir de la moyenne des performances
- La performance finale est évaluée sur le Test set

CONSEILS

- Préférez la **stratification** selon la variable Y à un tirage aléatoire simple pour le split des échantillons
- Si split en 2 échantillons : 70% (train) / 30% (test)
- K-FOLD : 3, 5 ou 10 selon la taille de l'échantillon

PARTITIONNEMENT DES DONNÉES

VALIDATION CROISÉE SIMPLE : APPRENTISSAGE / TEST

Pour créer un modèle et évaluer la qualité du modèle, nous allons diviser nos données en 2 :

- Un jeu de données « train » qui va nous permettre de créer le modèle (phase d'apprentissage)
- Un jeu de données « test » qui va nous permettre d'évaluer la qualité du modèle (phase de test)

Exemple :

```
import pandas as pd
from sklearn.model_selection import train_test_split
df = pd.read_sas(r'C:/Users/TLENOUVEL/Formation_Python/Sources/titanic.sas7bdat', encoding='utf-8')

# Exemple de recodage de la variable cible
df['Survived'] = df['Survived'].map({'yes':1, 'no':0}).astype('int8')

# Variable cible
y = df['Survived'].copy()

# Covariables
X = df.loc[:, df.columns != 'Survived'].copy()

# 1. Création des échantillons train et test pour X et y
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.3, random_state=42, stratify=y)

# 2. Séparation apprentissage / test en utilisant les index de lignes
train_index, test_index = train_test_split(df.index, test_size = 0.3, random_state=42, stratify=y)

# 3. Séparation apprentissage / test en utilisant les numéros de lignes
train_index, test_index = train_test_split(range(len(df)), test_size = 0.3, random_state=42, stratify=y)
```

PARTITIONNEMENT DES DONNÉES

K-FOLD VALIDATION : EXEMPLE D'APPLICATION

Code Python :

```
'''Stratified K-fold'''

from sklearn.model_selection import StratifiedKFold
k=5
y =df['Survived'] #Extraction du vecteur cible au format series
X = df.copy()
del X['Survived'] # Suppression de la variable cible pour
l'apprentissage

skf = StratifiedKFold(n_splits = k)

Y =pd.DataFrame(df['Survived'], index = df.index) # extraction de la
variable cible au format DataFrame
for train_index, test_index in skf.split(X,y):
    X_train = X.iloc[train_index,:]
    X_test = X.iloc[test_index,:]
    y_train = Y.iloc[train_index,:]
    y_test = Y.iloc[test_index,:]
```

La suite du code dépendra des algorithmes testés et des métriques choisies

01

Introduction à Scikit-Learn

02

Principaux concepts de Scikit-Learn

03

Partitionnement des données

04

Préprocessing

05

Fonctions avancées pour la gestion des variables catégorielles

06

Régressions logistiques (ridge, lasso, elasticnet)

07

Algorithmes à base d'arbres

08

Éléments pour l'interprétation

09

Sélection de variables explicatives

10

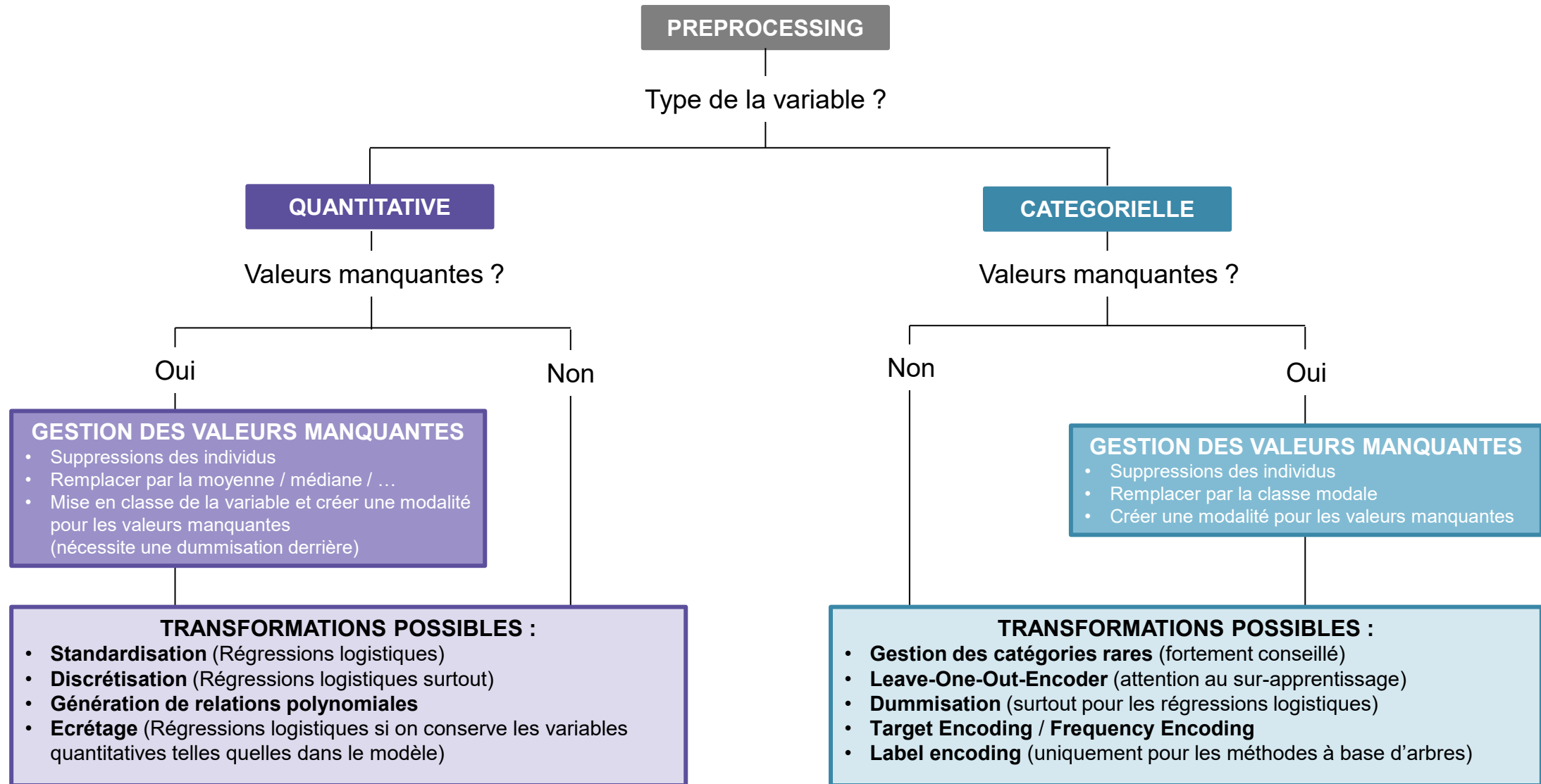
Optimisation des hyperparamètres

11

Métriques d'évaluation de la performance des modèles

FONCTIONS DE PRÉPROCESSING DE SCIKIT-LEARN

TRAITEMENTS SELON LE TYPE DE CHAMPS



FONCTIONS DE PRÉPROCESSING DE SCIKIT-LEARN

DATA TRANSFORMATION

La préparation des données est très importante sur Python car l'ensemble des algorithmes de Machine Learning de Scikit-Learn fonctionnent uniquement avec des variables numériques et sans valeurs manquantes.

```
 from sklearn import preprocessing as pp
```

Variables catégorielles : Transformation en variables numériques

- **Dummmisation**



```
b = pp.OneHotEncoder()  
b.fit(X)  
X=b.transform(X)
```

- **Label Encoder**



```
l = pp.LabelEncoder()  
l.fit(X["Var1"])  
X["Var1"]=b.transform(X["Var1"])
```

Normalisation des données :

Pour le bon fonctionnement de certains algorithmes, les données doivent être standardisées.

```
 n = pp.StandardScaler()  
n = pp.RobustScaler()  
n.fit(X)  
X=n.transform(X)
```

VALEURS MANQUANTES

Variables catégorielles :

- Création d'une nouvelle modalité pour les valeurs manquantes



```
df["Var1_NA"] = df["Var1"].isnull().astype(int)
```

- Imputation par la classe modale



```
i = pp.imputer(strategy = "most_frequent")  
i.fit(X_train)  
X_new = i.transform(X_test)
```

Variables numériques :

- Remplacer par une constante avec fillna de Pandas (efficace pour les arbres)



```
df["Var1"] = df["Var1"].fillna(-999)
```

- Imputation par la médiane / moyenne et créer en addition une variable supplémentaire pour indiquer si la valeur était manquante



```
i = pp.imputer(strategy = "mean")  
i.fit(X_train)  
X_new = i.transform(X_test)
```

FONCTIONS DE PRÉPROCESSING DE SCIKIT-LEARN

GESTION DES VARIABLES NUMÉRIQUES

Attention : Les algorithmes de Scikit-Learn **ne traitent pas automatiquement les valeurs manquantes** (numériques ou catégorielle)

→ **ValueError**: Input contains NaN

Il faut donc les traiter lors de l'étape du preprocessing

Traitements possibles des variables numériques :

- **Imputer les valeurs manquantes** (remplacer par la moyenne, médiane...): `preprocessing.imputer` (cf slide suivante)
- **Standardiser (centrer/réduire)** : `preprocessing.scale(X)` → fortement conseillé pour les modèles linéaires
- **Autres standardisations possibles** : `MinMaxScaler`, `MaxAbsScaler`...
- **Binariser** une variable numérique une variable : `preprocessing.Binarizer(threshold = x)` (Par défaut `x = 0`)
- **Mettre en classes de fréquences égales** (discrétisation basée sur les quantiles) :
`pd.qcut(df.Variable, labels=[« classe1 », ... , « classex »])`
- **Discrétiser par arbre de décision** (approche supervisée. Non traité dans ce support)
- **Générer des variables polynomiales et des interactions** : `preprocessing.PolynomialFeatures(degrees)`
→ **Attention** peut générer beaucoup de variables

FONCTIONS DE PRÉPROCESSING DE SCIKIT-LEARN

GESTION DES VARIABLES QUANTI : IMPUTER LES VALEURS MANQUANTES

Remplacement des valeurs manquantes

```
from sklearn.impute import SimpleImputer
```

```
import numpy as np
```

```
import pandas as pd
```

```
df= pd.DataFrame({'V1' : [1,2,np.nan,10,2], 'V2' : [1,np.nan,3,4,5]})
```

```
imp = SimpleImputer(strategy="mean")
```

```
imp.fit(df)
```

```
df_bis = imp.transform(df)
```

```
#array([[ 1. ,  1. ],
```

```
#      [ 2. ,  3.25],
```

```
#      [ 3.75,  3. ],
```

```
#      [10. ,  4. ],
```

```
#      [ 2. ,  5. ]])
```

```
imp_ter = SimpleImputer(strategy = "median")
```

```
'''Pour le mode : strategy = 'most_frequent' '''
```

```
imp_ter.fit(df)
```

```
df_ter = imp_ter.transform(df)
```

```
#array([[ 1. ,  1. ],
```

```
#      [ 2. ,  3.5],
```

```
#      [ 2. ,  3. ],
```

```
#      [10. ,  4. ],
```

```
#      [ 2. ,  5. ]])
```

FONCTIONS DE PRÉPROCESSING DE SCIKIT-LEARN

GESTION DES VARIABLES QUANTI : STANDARDISATION

Utilisation de la fonction `preprocessing.StandardScaler()`

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler # charger la fonction pour centrer réduire

# Lecture de la table
path_data = 'C:/Formation Python/Datasets/'
df = pd.read_csv(path_data + 'adult.csv', sep = ',')
y = (df['income']=='>50K').astype('int8').copy()
del df['income']

# Partitionnement
train_index, test_index = train_test_split(range(len(df)), test_size = 0.3, random_state=42, stratify=y)

# Standardisation des variables quantitatives
liste_var_cont = [var for var in df.select_dtypes(np.number)] # liste des variables numériques

#Normalisation des variables concernées :
scaler = StandardScaler().fit(df.loc[train_index, liste_var_cont]) # préparation de la transformation à effectuer
df[liste_var_cont] = scaler.transform(df[liste_var_cont]) # application de la transformation

#Pour vérifier que les variables ont bien été normalisées :
df.loc[train_index, liste_var_cont].mean()
df.loc[train_index, liste_var_cont].std()
```

FONCTIONS DE PRÉPROCESSING DE SCIKIT-LEARN

GESTION DES VARIABLES NUMÉRIQUES : GÉNÉRER DES VARIABLES POLYNOMIALES

Utilisation de la fonction « PolynomialFeatures »

```
# Polynomiale features :
from sklearn.preprocessing import PolynomialFeatures
Z = np.arange(6).reshape(3, 2)
#array([[0, 1],
#       [2, 3],
#       [4, 5]])
poly = PolynomialFeatures(2) # degres par défaut = 2
poly.fit_transform(Z)
#array([[ 1.,  0.,  1.,  0.,  0.,  1.],
#       [ 1.,  2.,  3.,  4.,  6.,  9.],
#       [ 1.,  4.,  5., 16., 20., 25.]])
# (X1, X2) to (1, X1, X2, X12, X1X2, X22).

poly = PolynomialFeatures(interaction_only=True)
poly.fit_transform(Z)
#array([[ 1.,  0.,  1.,  0.],
#       [ 1.,  2.,  3.,  6.],
#       [ 1.,  4.,  5., 20.]])

poly = PolynomialFeatures(degree=3, interaction_only=False, include_bias=False)
poly.fit_transform(Z)
#array([[ 0.,  1.,  0.,  0.,  1.,  0.,  0.,  0.,  1.],
#       [ 2.,  3.,  4.,  6.,  9.,  8., 12., 18., 27.],
#       [ 4.,  5., 16., 20., 25., 64., 80., 100., 125.]])
```

~~(1, X₁, X₂, X₃, X₁X₂, X₁X₃, X₂X₃, X₁X₂X₃).~~

FONCTIONS DE PRÉPROCESSING DE SCIKIT-LEARN

GESTION DES VARIABLES CATÉGORIELLES

Les algorithmes de Scikit-Learn **ne savent pas gérer automatiquement les variables nominales**

Plusieurs cas de figures sont possibles :

- Encodage de la variable en caractère (ex : Ouest / Est / Nord / Sud) : **l'algorithme ne fonctionne pas**
- Encodage de la variable en entier ou réel : **l'algorithme considère la variable comme une variable continue**

Dans le cas où la variable est encodée en entier ou réel :

- Cas binaire : 0/1 aucun problème
- Cas multinomial : 0/1/2 l'algorithme considèrera que la modalité 2 est deux fois plus grande que la modalité 1, ce qui n'a aucun sens. 2 solutions possibles :
 - Transformation de la variable nominale en variable continue (technique du Leave-One-Out)
 - Dummisation : `pandas.get_dummies()` (package Pandas)

```
df = pd.DataFrame({'Var_1': ['Paris', 'Londres', 'Manchester', 'Paris'],
                  'Var_2': ['Actif', 'Actif', 'Etudiant', 'Retraité'],
                  'Var_3': [45, 56, 19, 78]})

df = pd.get_dummies(df, drop_first = True)
```

#	Var_3	Var_1_Manchester	Var_1_Paris	Var_2_Etudiant	Var_2_Retraité
#0	45	0.0	1.0	0.0	0.0
#1	56	0.0	0.0	0.0	0.0
#2	19	1.0	0.0	1.0	0.0
#3	78	0.0	1.0	0.0	1.0

FONCTIONS DE PRÉPROCESSING DE SCIKIT-LEARN

ENCODAGE DES VARIABLES CATÉGORIELLES

Binarization / Dummisation / One Hot Encoding:

Var1			
A	1	0	0
B	0	1	0
C	0	0	1



 `Sklearn.preprocessing.Binarizer()`

Label Encoding :

Var1		
A	1	
B	2	
C	3	



Cette méthode encode chaque catégorie comme un entier dans l'ordre croissant

 `Sklearn.preprocessing.LabelEncoder()`

➤ Nous verrons plus tard d'autres types d'encodage :

- Frequency Encoding
- Target Encoding
- Leave One Out Encoding

FONCTIONS DE PRÉPROCESSING DE SCIKIT-LEARN

ENCODAGE DES VARIABLES CATÉGORIELLES : LABEL ENCODING

Objectif : remplacer chaque variable catégorielle par une variable numérique en affectant à chaque catégorie un numéro spécifique

```
import pandas as pd
from sklearn import preprocessing

df = pd.DataFrame({"Var_1": [1, 1, 2, 4, 7], 'Var_2' : ["Paris", "Tokyo", "Tokyo", "Pekin", "Paris"]})
Y = pd.Series(["Paris", "Tokyo", "Tokyo", "Pekin", "Paris"])
```

Import package
+ Création dataframe

```
le = preprocessing.LabelEncoder()
le.fit(Y)
Y_lab = le.transform(Y)
print(Y_lab)      # [0 2 2 1 0]
list(le.inverse_transform(Y_lab)) # ['Paris', 'Tokyo', 'Tokyo',
                                   #'Pekin', 'Paris']
```

Schéma classique :
définition, fit, transform

```
Test = [0, 1, 1, 2]
list(le.inverse_transform(Test)) # ['Paris', 'Pekin', 'Pekin', 'Tokyo']
```

```
Test2 = [0, 1, 1, 2, 3]
list(le.inverse_transform(Test2)) # ValueError: y contains unseen labels: [3]
```

```
Test3 = ["Paris", "Tokyo", "Madrid"]
list(le.transform(Test3))          # ValueError: y contains unseen labels: ['Madrid']
```

Erreurs quasi certaines en
production. Il faut gérer
l'apparition de nouvelles
modalités

FONCTIONS DE PRÉPROCESSING DE SCIKIT-LEARN

ENCODAGE DES VARIABLES CATÉGORIELLES : ONE HOT ENCODING

One Hot Encoder

- Binariser des variables en créant une indicatrice par modalité
- Ne prend en compte que des matrices / dataframes typées « integer » → utiliser le label encoder pour remplacer les variables non numériques en numérique
- A utiliser pour les variables catégorielles

```
import pandas as pd
from sklearn import preprocessing
df = pd.DataFrame({"Var_1": [1, 1, 2, 4, 7], 'Var_2' : ["Paris", "Tokyo", "Tokyo", "Pekin", "Paris"]})
#   Var_1  Var_2
#0      1   Paris
#1      1   Tokyo
#2      2   Tokyo
#3      4   Pekin
#4      7   Paris
le = preprocessing.LabelEncoder()
le.fit(df['Var_2'])
df['Var_2'] = le.transform(df['Var_2'])
#   Var_1  Var_2
#0      1      0
#1      1      2
#2      2      2
#3      4      1
#4      7      0
enc = preprocessing.OneHotEncoder()
enc.fit(df)
df_bis = enc.transform(df).toarray()
#array([[ 1.,  0.,  0.,  0.,  1.,  0.,  0.],
#       [ 1.,  0.,  0.,  0.,  0.,  0.,  1.],
#       [ 0.,  1.,  0.,  0.,  0.,  0.,  1.],
#       [ 0.,  0.,  1.,  0.,  0.,  1.,  0.],
#       [ 0.,  0.,  0.,  1.,  1.,  0.,  0.]])
```

Par défaut traite toutes les variables, le résultat est un array → on perd le nom des colonnes

FONCTIONS DE PRÉPROCESSING DE SCIKIT-LEARN

ENCODAGE DES VARIABLES CATÉGORIELLES : ONE HOT ENCODING

```
'''Choisir les variables catégorielles'''
```

```
from sklearn.compose import ColumnTransformer
```

```
transformer = ColumnTransformer(  
    transformers=[  
        ("OneHot",  
         preprocessing.OneHotEncoder(categories='auto'), # la transformation à appliquer  
         ['Var_2'] # la colonne sur laquelle appliquer la transformation  
        ),  
    ], remainder='passthrough' # ne rien faire sur les autres colonnes  
)
```

```
df_bis = transformer.fit_transform(df)
```

```
#array([[ 1.,  0.,  0.,  1.],  
#       [ 0.,  0.,  1.,  1.],  
#       [ 0.,  0.,  1.,  2.],  
#       [ 0.,  1.,  0.,  4.],  
#       [ 1.,  0.,  0.,  7.]])
```

```
'''En une ligne avec pandas '''
```

```
df = pd.DataFrame({"Var_1": [1, 1, 2, 4, 7], 'Var_2' :  
                  ["Paris", "Tokyo", "Tokyo", "Pekin", "Paris"]})
```

```
df = pd.get_dummies(df, columns = ['Var_2'])
```

```
#   Var_1  Var_2_Paris  Var_2_Pekin  Var_2_Tokyo  
#0      1           1.0           0.0           0.0  
#1      1           0.0           0.0           1.0  
#2      2           0.0           0.0           1.0  
#3      4           0.0           1.0           0.0  
#4      7           1.0           0.0           0.0
```



Préférer utiliser OneHotEncoder()
(cf slide suivante)

FONCTIONS DE PRÉPROCESSING DE SCIKIT-LEARN

ONEHOTENCODER VS GET_DUMMIES

```
import pandas as pd
train_data = pd.DataFrame(data = ['good', 'bad', 'worst', 'good', 'good', 'bad'], columns = ['State'])
test_data = pd.DataFrame(data = ['good', 'bad', 'good', 'good', 'bad', 'excellent', 'perfect'], columns = ['State'])
# test_data contient 2 valeurs que data n'a pas et ne contient pas 'worst'
```

➤ Dummisation avec pandas.get_dummies

```
df_train = pd.get_dummies(train_data)
```

	State_bad	State_good	State_worst
0	0	1	0
1	1	0	0
2	0	0	1
3	0	1	0
4	0	1	0
5	1	0	0

```
df_test = pd.get_dummies(test_data)
```

	State_bad	State_excellent	State_good	State_perfect
0	0	0	1	0
1	1	0	0	0
2	0	0	1	0
3	0	0	1	0
4	1	0	0	0
5	0	1	0	0
6	0	0	0	1

```
set(df_train.columns) < set(df_test.columns) # pour vérifier si les dummies de df_train sont bien présentes dans le test.
# Retourne False
```

FONCTIONS DE PRÉPROCESSING DE SCIKIT-LEARN

ONEHOTENCODER VS GET_DUMMIES

➤ Dummisation avec `sklearn.preprocessing.OneHotEncoder`

```
from sklearn.preprocessing import OneHotEncoder
encoder = OneHotEncoder(handle_unknown="ignore", sparse=False)
```

```
encoder.fit(train_data)
```

```
encoder.transform(train_data)
```

```
array([[0., 1., 0.],
       [1., 0., 0.],
       [0., 0., 1.],
       [0., 1., 0.],
       [0., 1., 0.],
       [1., 0., 0.]])
```

```
encoder.transform(test_data)
```

```
array([[0., 1., 0.],
       [1., 0., 0.],
       [0., 1., 0.],
       [0., 1., 0.],
       [1., 0., 0.],
       [0., 0., 0.],
       [0., 0., 0.]])
```

Afin de créer les dummies pour la table test tout en respectant les catégories présentes dans la table d'apprentissage :

1. Préciser `handle_unknown="ignore"` comme option de `OneHotEncoder()` afin de ne pas gérer les catégories inconnues lors de la transformation.

2. Utiliser la table train comme table de référence pour l'encodage :

```
encoder.transform(train_data)
```

3. Appliquer la transformation à la table test_data :

```
encoder.transform(test_data)
```

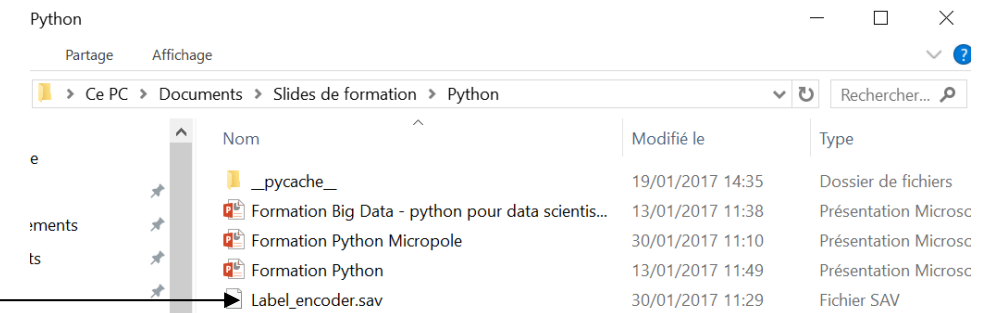
FONCTIONS DE PRÉPROCESSING DE SCIKIT-LEARN

SAUVEGARDER LES TRANSFORMATIONS

Toutes les transformations peuvent être sauvegardées et réutilisées grâce au package pickle

Code Python : Sauvegarde de la transformation

```
'''Exemple avec le label Encoder'''
import pickle
le = preprocessing.LabelEncoder()
le.fit(Y)
filename = r"C:\Users\Documents\Slides de formation\Python\Label_encoder.sav"
pickle.dump(le, open(filename, 'wb'))
```



Téléchargement de la transformation et application :

```
filename_bis = r"C:\Users\Documents\Slides de formation\Python\Label_encoder.sav"

le_bis = pickle.load(open(filename_bis, 'rb' ))
Y_bis = ["Paris", "Paris", "Pekin", "Pekin"]
list(le_bis.transform(Y_bis)) # [0, 0, 1, 1]
```

Fonctionne aussi pour les modèles et pour tout autre objet en Python (matrice, dataframe, ...)
Gain de temps et de flexibilité

FONCTIONS DE PRÉPROCESSING DE SCIKIT-LEARN

PCA

PCA (Principal Composant Analysis) → Réduction de dimension

```
import pandas as pd
from sklearn.decomposition import PCA
```

```
path_data = 'C:/Formation Python/Datasets/'
df = pd.read_csv(path_data + 'diabete.csv', sep = ',')
Y = df['Outcome']
del df['Outcome']
X = df.copy()
```

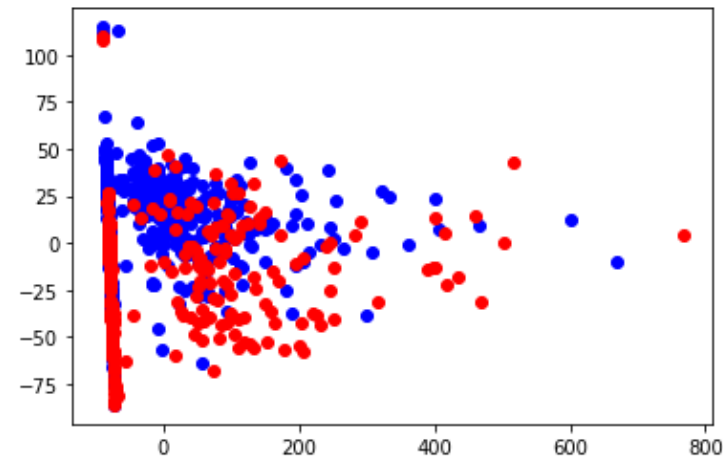
```
# PCA
pca = PCA(n_components= 2)
pca.fit(X)
X_trans = pca.transform(X)
X_trans = pd.DataFrame(X_trans)
X_trans.head(3)
```

#	0	1	Target
#0	-75.714655	-35.950783	1
#1	-82.358268	28.908213	0
#2	-74.630643	-67.906496	1

```
'''Graphique classique'''
import matplotlib.pyplot as plt
X_trans['Target'] = Y
X_trans_0 = X_trans[X_trans['Target']==0]
X_trans_1 = X_trans[X_trans['Target']==1]
plt.plot(X_trans_0[0],X_trans_0[1], 'bo')
plt.plot(X_trans_1[0],X_trans_1[1], 'ro')
```

- PCA ne propose que des analyses non normées, basées sur la matrice de covariance.
- Pour réaliser des analyses normées (cercle de corrélations), il est nécessaire de centrer-réduire au préalable vos données avant d'utiliser PCA.
- Pour ce faire, vous pouvez utiliser Pandas : $X = (X - X.mean()) / X.std()$ ou utiliser la fonction `StandardScaler` de Scikit-learn

On retrouve le schéma classique (fit/ transform)
En une ligne : `X_trans = pca.fit_transform(X)`



FONCTIONS DE PRÉPROCESSING DE SCIKIT-LEARN

ETUDE DES CORRÉLATIONS

```
import pandas as pd
import seaborn as sns
```

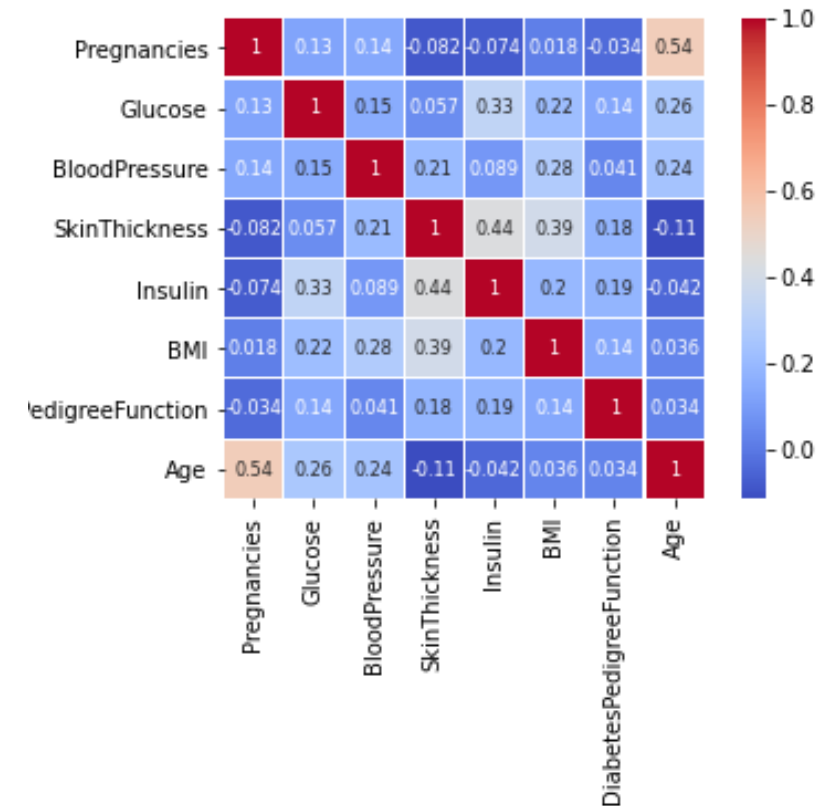
```
path_data = 'C:/Formation Python/Datasets/'
df = pd.read_csv(path_data + 'diabete.csv', sep = ',')
Y = df['Outcome']
del df['Outcome']
X = df.copy()
```

```
# Calcul de la matrice des corrélations avec Pandas
corr_mat = X.corr()
```

```
# Heatmap avec le package seaborn (beaucoup plus simple que matplotlib.pyplot)
sns.heatmap(corr_mat, vmin=corr_mat.values.min(), vmax=1, square=True,
            linewidths=0.1, annot=True, annot_kws={"size":8}, cmap= 'coolwarm')
```

- `X.corr()` assez lent quand on travaille avec beaucoup de données
- Ci-dessous une estimation rapide de la matrice des corrélations avec Numpy si beaucoup d'individus et variables.
- Notez que la première ligne gère l'éventuelle présence de données manquantes et que le résultat est mis en forme dans un dataframe

```
# Estimation matrice de corrélation avec numpy avec remplacement des données manquantes par la moyenne
matrix_corr = np.corrcoef(X.fillna(X.mean()).values, rowvar = False)
matrix_corr = pd.DataFrame(data = matrix_corr, index = X.columns, columns = X.columns)
```



01

Introduction à Scikit-Learn

02

Principaux concepts de Scikit-Learn

03

Partitionnement des données

04

Préprocessing

05

Fonctions avancées pour la gestion
des variables catégorielles

06

Régressions logistiques (ridge, lasso,
elasticnet)

07

Algorithmes à base d'arbres

08

Éléments pour l'interprétation

09

Sélection de variables explicatives

10

Optimisation des hyperparamètres

11

Métriques d'évaluation de la
performance des modèles

FONCTIONS AVANCÉES POUR LA GESTION DES VARIABLES CATÉGORIELLES

FREQUENCY ENCODING

Il s'agit d'utiliser la fréquence des catégories de la variable à encoder comme étiquettes.

Trois étapes pour cela :

- Sélectionner la variable catégorielle à transformer
- Calculer la fréquence de chaque catégorie
- Ajouter le résultat à l'ensemble des données d'entraînement

```
import pandas as pd
df = pd.DataFrame({'Temperature': ['Hot', 'Cold', 'Very Hot', 'Warm',
                                   'Hot', 'Warm', 'Warm', 'Hot', 'Hot', 'Cold'],
                  'Color': ['Red', 'Yellow', 'Blue', 'Blue', 'Red', 'Yellow',
                            'Red', 'Yellow', 'Yellow', 'Yellow'],
                  'Target': [1, 1, 1, 0, 1, 0, 1, 0, 1, 1]})
```

Temperature	Color	Target		Temperature	Color	Target	Freq_encode
Hot	Red	1	→	Hot	Red	1	0.4
Cold	Yellow	1		Cold	Yellow	1	0.2
Very Hot	Blue	1		Very Hot	Blue	1	0.1
Warm	Blue	0		Warm	Blue	0	0.3
Hot	Red	1		Hot	Red	1	0.4
Warm	Yellow	0		Warm	Yellow	0	0.3
Warm	Red	1		Warm	Red	1	0.3
Hot	Yellow	0		Hot	Yellow	0	0.4
Hot	Yellow	1		Hot	Yellow	1	0.4
Cold	Yellow	1		Cold	Yellow	1	0.2

Pour construire l'encodage par fréquence on peut utiliser le code Python suivant :

```
 fe = df.groupby('Temperature').size()/len(df)
df.loc[:, 'Freq_encode'] = df['Temperature'].map(fe)
```

ou

```
 fe = df.Temperature.value_counts()/len(df)
df.loc[:, 'Freq_encode'] = df['Temperature'].map(fe)
```

Il existe une multitude d'autres façons d'encoder les variables catégorielles. Parmi elles, on retrouve notamment :

- Target Encoding (modalités sont remplacées par un mélange de probabilités conditionnelles à la variable cible) 
- Leave One Out Encoding (cf slide suivante) 

FONCTIONS AVANCÉES POUR LA GESTION DES VARIABLES CATÉGORIELLES

LEAVE ONE OUT ENCODING (1/3)

- Variable Split : Train (échantillon d'apprentissage : valeurs Y connues) et Test (valeurs Y inconnues qu'on cherche à prédire)
- Variable VarCat : 2 modalités (A1 / A2)
- Variable Y : variable cible (0 / 1)

Split	VarCat	Y	mean(Y)	Random	LOO_VarCat
Train	A1	0	0.667		
Train	A1	1			
Train	A1	1			
Train	A1	0			
Test	A1	-			
Test	A1	-			
Train	A2	0			

- Etape 1 : Calcul de la première ligne (mean(Y))

$\text{mean}(Y) = \text{mean}(Y \mid \text{Catégorie} == A1)$

Comme pour la méthode de validation Leave-One-Out on exclut la ligne qu'on cherche à calculer

FONCTIONS AVANCÉES POUR LA GESTION DES VARIABLES CATÉGORIELLES

LEAVE ONE OUT ENCODING (2/3)

- Variable Split : Train (échantillon d'apprentissage : valeurs Y connues) et Test (valeurs Y inconnues qu'on cherche à prédire)
- Variable VarCat : 2 modalités (A1 / A2)
- Variable Y : variable cible (0 / 1)

Split	VarCat	Y	mean(Y)	Random	LOO_VarCat
Train	A1	0	0.667		
Train	A1	1	0.333		
Train	A1	1			
Train	A1	0			
Test	A1	-			
Test	A1	-			
Train	A2	0			

Diagram illustrating the calculation of mean(Y) for VarCat A1:

The mean(Y) for VarCat A1 is calculated as $(0 + 1 + 0) / 3 = 0.333$.

- Etape 2 : Calcul des autres lignes (mean(Y))

$$\text{mean}(Y) = \text{mean}(Y \mid \text{Catégorie} == A1)$$

Comme pour la méthode de validation Leave-One-Out on exclut la ligne qu'on cherche à calculer

FONCTIONS AVANCÉES POUR LA GESTION DES VARIABLES CATÉGORIELLES

LEAVE ONE OUT ENCODING (3/3)

- Variable Split : Train (échantillon d'apprentissage : valeurs Y connues) et Test (valeurs Y inconnues qu'on cherche à prédire)
- Variable VarCat : 2 modalités (A1 / A2)
- Variable Y : variable cible (0 / 1)
- **Obtention de la nouvelle variable numérique : LOO_VarCat calculée à partir d'une variable nominale**

Split	VarCat	Y	mean(Y)	Random	LOO_VarCat
Train	A1	0	0.667	1.05	0.70035
Train	A1	1	0.333	0.97	0.32301
Train	A1	1	0.333	0.98	0.32634
Train	A1	0	0.667	1.02	0.68034
Test	A1	-	0.5	1	0.5
Test	A1	-	0.5	1	0.5
Train	A2	0			

Proportion de 1 pour la modalité A1 = 50%

- Etape 3 :
 - Calcul de la variable Random : simulation d'une loi normale de moyenne 1 et d'écart-type 0.05 ou 0.1
 - Calcul de notre nouvelle variable :
 $LOO_VarCat = mean(Y) * Random$
- Etape 4 : Ajout des variables dans le fichier Test
- Etape 5 : répétition de la démarche pour la seconde modalité A2

FONCTIONS AVANCÉES POUR LA GESTION DES VARIABLES CATÉGORIELLES

EXEMPLE D'APPLICATION

```
import pandas as pd
from sklearn.model_selection import train_test_split
from category_encoders.leave_one_out import LeaveOneOutEncoder 
# Lecture de la table
path_data = 'C:/Formation Python/Datasets/'
df = pd.read_csv(path_data + 'adult.csv', sep = ',')

y = (df['income']=='>50K').astype('int8').copy()
del df['income'] # df est maintenant composée uniquement des variables explicatives

# Partitionnement des index en train / test
train_index, test_index = train_test_split(range(len(df)), test_size = 0.3, stratify = y, random_state = 0)

# Identification des variables catégorielles stockées sous forme de chaînes de caractères
liste_var_str = [var for var in df.select_dtypes(object)]

# L00 Encoding des variables de liste_var_str
Loo_enc = LeaveOneOutEncoder()
loo_varnames = ['L00_' + v for v in liste_var_str] # nom des nouvelles variables
# Calcul des valeurs L00 et application à l'échantillon d'apprentissage
df.loc[train_index, loo_varnames] = Loo_enc.fit_transform(df.loc[train_index, liste_var_str], y[train_index]).values
# Application à l'échantillon test
df.loc[test_index, loo_varnames] = Loo_enc.transform(df.loc[test_index, liste_var_str]).values

# Frequency Encoding
fe = df.gender.value_counts()/len(df)
df['FE_gender'] = df['gender'].map(fe)

# Comparaison L00 Encoding VS Frequency Encoding
df.gender.value_counts(); df.FE_gender.value_counts(); pd.crosstab(df.loc[train_index, 'gender'], y[train_index])
```

Male	32650
Female	16192

0.668482	32650
0.331518	16192

income	0	1
gender		
Female	10060	1225
Male	15948	6956

df.L00_gender.value_counts()

0.303716	15948
0.108561	10060
0.303702	9746
0.303672	6956
0.108551	4907
0.108472	1225

01

Introduction à Scikit-Learn

02

Principaux concepts de Scikit-Learn

03

Partitionnement des données

04

Préprocessing

05

Fonctions avancées pour la gestion
des variables catégorielles

06

Régressions logistiques (ridge, lasso,
elasticnet)

07

Algorithmes à base d'arbres

08

Éléments pour l'interprétation

09

Sélection de variables explicatives

10

Optimisation des hyperparamètres

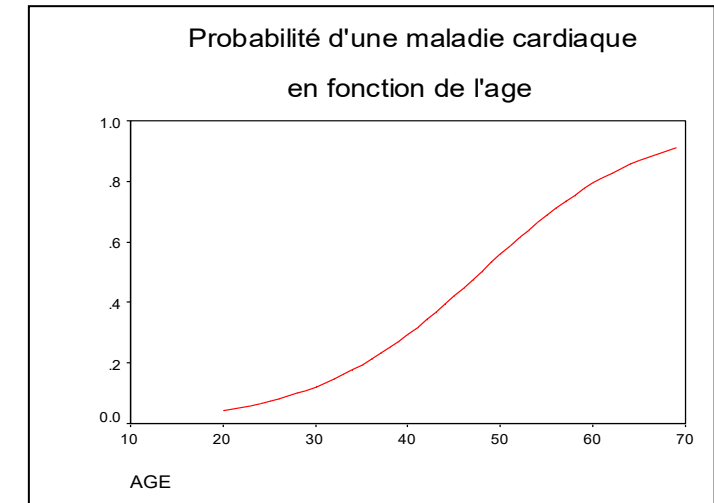
11

Métriques d'évaluation de la
performance des modèles

RÉGRESSIONS LOGISTIQUES

LA RÉGRESSION LOGISTIQUE BINAIRE : MÉTHODE PHASE DU SCORING

- **Y : variable à expliquer binaire**
- **X1, ..., Xp : variables explicatives**
 - **Variables continues et/ou**
 - **Variables nominales**
 - modalités transformées en indicatrices
 - pas de notion de modalité de référence dans Python



- **On modélise la probabilité d'appartenance au groupe cible**

$$\pi(x) = \text{Prob}(Y = 1 / X = x) = \frac{e^{\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p}}{1 + e^{\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p}}$$

- **Fonction de lien : Logit**

$$\text{Log}\left(\frac{\pi(x)}{1 - \pi(x)}\right) = \beta_0 + \beta_1 x + \beta_2 x + \dots + \beta_p x_p$$

RÉGRESSIONS LOGISTIQUES

INTERPRÉTATION DES PARAMÈTRES DU MODÈLE

- **Vérifier que l'on modélise bien la modalité de la variable Y qui nous intéresse** (1 par défaut dans R et Python)
- **Interprétez uniquement les ODDS-RATIO** sauf si les variables ont été standardisées
- **Variables nominales**
 - Un odd-ratio mesure le rapport des chances d'apparition de l'événement $Y=1$ lorsque la variable nominale X vaut m_i par rapport à la situation de référence ($m_{\text{réf}}$).
 - Si Odds-Ratio > 1 : la présence de m_i augmente les chances de l'évènement $Y=1$ par rapport à $m_{\text{réf}}$
 - **Un odd (une cote) est défini par :**
 - $\text{ODD}(X=x) = P(Y=1/X=x) / (1 - P(Y=1/X=x))$
 - **Un odds-ratio se définit par :**
 - $\text{OR}(X = m_i) = \text{ODD}(X=m_i) / \text{ODD}(X=m_{\text{réf}})$
pour X quali ou $m_{\text{réf}}$ est la modalité de référence.
- Dans le package Scikit-Learn, la notion de variable nominale n'existe pas. Chaque modalité (transformée au préalable en indicatrice) est considérée comme une variable continue.

RÉGRESSIONS LOGISTIQUES

INTERPRÉTATION DES PARAMÈTRES

- **Interprétation des coefficients des variables continues :**

- **On interprète l'odds-ratio (OR) lié à la variable continue :**
 - L'OR d'une variable continue est le facteur multiplicatif de la probabilité de l'apparition de l'évènement $Y = 1$ quand ladite variable augmente d'une unité.
- **Exemple : on modélise la variable Ronflement (Oui/Non) à partir de l'âge.**
 - $OR(\text{Age}) = 1.06$ signifie que lorsque l'on vieillit d'un an, la probabilité de ronfler augmente de 6%.
 - Lorsque l'on vieillit de 2 ans, la probabilité de ronfler augmente de $1.06^2 = 1.1236$, soit 12%.
- $OR > 1$: la probabilité de l'évènement augmente lorsque la variable augmente
- $OR < 1$: la probabilité de l'évènement diminue lorsque la variable augmente

- **ATTENTION, les odds ratio ne peuvent être interprétés tels quels si les variables explicatives ont été standardisées.**

RÉGRESSIONS LOGISTIQUES

● Pratiques historiques :

- **Mise en classes des variables continues** (Quantiles, Arbres de décision)
- **Simplification des variables catégorielles** : regroupements de modalités (Métier, ChiMerge), gestion des modalités rares
- **Recherche de nouveaux indicateurs pertinents, interactions** (recherche manuelle, Arbres de décision)
- **Suppression des variables redondantes** (corrélations, V de Cramer)
- **Sélection des variables pertinentes** (Expert, méthodes Stepwise / Forward / Backward)
- **Régression logistique sur variables qualitatives** (nominales et continues mises en classes)
- **Simplification du modèle par la suppression des variables / modalités dont les coefficients sont proches de 0**

● Nouvelles pratiques issues de l'open source :

- **Conserver les variables continues telles quelles si possible**
- **Simplification des variables catégorielles** : regroupements des modalités rares
- **Encodage des variables catégorielles** (frequency encoding, target encoding) pour réduire la dimensionnalité
- **Standardisation des variables explicatives**
- **Gestion de la colinéarité via** :
 - La régression logistique **ridge** (partage des poids sur les variables corrélées)
 - La régression logistique **lasso** (mise à zéro de certains coefficients)
 - La régression logistique **ElasticNet** (ridge + lasso)

RÉGRESSIONS LOGISTIQUES

STANDARDISER LES VARIABLES EXPLICATIVES

- Standardiser les variables revient à toutes les mettre à la même échelle (centrer-réduire)
- Dans Python, on peut notamment utiliser `sklearn.preprocessing.StandardScaler()` pour ce faire (cf slide 173)
- On standardise les variables explicatives d'une régression logistique pour différentes raisons :
 - Éviter que les variables à forte variance aient trop d'influence
 - Comparer l'importance des variables
 - Préférable pour les régressions logistiques pénalisées afin d'obtenir de meilleurs résultats
 - Aider la convergence de certains algorithmes d'optimisation (ex : Newton-Raphson)
 - Certains algorithmes Python demandent à ce que les variables soient standardisées : c'est le cas pour `sklearn.linear_model.SGDClassifier()` par exemple

RÉGRESSIONS LOGISTIQUES

PÉNALISATIONS RIDGE, LASSO, ELASTICNET

➤ Ridge

- On ajoute une contrainte pour minimiser la somme des carrés des coefficients (pénalisation L2)
- Permet de baisser la variance en régularisant les coefficients → permet de contrôler l'amplitude des coefficients
- Permet de pallier le problème d'inversibilité de la matrice $X'X$ → permet de pallier les soucis de multicollinéarité

➤ Lasso

- On ajoute une contrainte pour minimiser la somme des coefficients en valeur absolue (pénalisation L1)
- Intérêts par rapport à Ridge : permet de sélectionner les variables explicatives en annulant certains coefficients
 - Diminue le risque de surapprentissage
 - Facilite l'interprétabilité, améliore les prévisions

➤ Elasticnet

- On ajoute la pénalisation Ridge à la pénalisation Lasso ce qui permet :
 - de rétrécir les coefficients des variables corrélées dans la même direction
 - Pallier le défaut de Lasso (probabilité non nulle d'exclure des variables pertinentes)

RÉGRESSIONS LOGISTIQUES

STRUCTURE DE CODE POUR LA MODÉLISATION

- La création d'un modèle sous scikit-learn suit le même procédé dans la majorité des cas (exemple avec `LogisticRegression()`):

```
import pandas as pd
#Lecture de la table
path_data = 'C:/Formation Python/Datasets/'
df = pd.read_csv(path_data + 'adult.csv', sep = ',')
y = (df['income']=='>50K').astype('int8').copy()
del df['income'] #df est maintenant composée que de la matrice des X

#Dummisation des variables catégorielles
df = pd.get_dummies(df)

#Séparation apprentissage/test
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(df, y, test_size=0.3,
random_state=0)

from sklearn.linear_model import LogisticRegression
clf = LogisticRegression(random_state=123,solver='liblinear',max_iter=200)
clf = clf.fit(X_train,y_train)

y_pred = clf.predict(X_test) # [0, 0, ... 1, ...]
prob = clf.predict_proba(X_test)
#[[0.70325198 0.29674802]
# [0.70793944 0.29206056]
# [0.54816666 0.45183334]
# ...
# [0.80442867 0.19557133]
# [0.82394388 0.17605612]
# [0.81365684 0.18634316]]
```

1. Import du package `sklearn.linear_model`
2. On définit le modèle
3. On l'entraîne sur l'échantillon train
4. On applique le modèle :
 - Le résultat est un array
 - On peut vouloir les probabilités (la première colonne correspond à $P(Y = 0)$)

- Avoir les probabilités nous permet notamment de modifier le cut-off (par défaut à 0.5) et de tracer la courbe ROC

RÉGRESSIONS LOGISTIQUES

FONCTIONS DE SCIKIT-LEARN (1/2)



➤ Il est préférable d'avoir standardisé les variables explicatives au préalable (notamment pour le solveur `saga`)

➤ `Sklearn.linear_model.LogisticRegression()`



Paramètres principaux	Valeurs possibles	Description
<code>penalty</code>	'l1', 'l2', 'elasticnet', 'none' Par défaut : 'l2'	Norme utilisée pour la pénalisation
<code>C</code>	Décimale entre 0 et 1. Par défaut : 1.0	Inverse du paramètre de régularisation. Une grande valeur de C implique une forte régularisation.
<code>fit_intercept</code>	True ou False Par défaut : True	Activation/désactivation de l'utilisation de l'intercept
<code>random_state</code>	Entier Par défaut : None	Graine aléatoire
<code>solver</code>	'newton-cg', 'lbfgs', 'liblinear', 'sag', 'saga' Par défaut : 'lbfgs'	Algorithme à utiliser pour l'optimisation. Attention : 'liblinear' : ne fonctionne pas sans régularisation 'newton-cg', 'sag', 'saga', 'lbfgs' : ne supportent pas la régularisation L1 'saga' : seul qui supportent la régularisation Elastic-net
<code>l1_ratio</code>	Décimale entre 0 et 1. Par défaut : None	Paramètre de combinaison pour l'ElasticNet, avec $0 \leq l1_ratio \leq 1$. Pour <code>l1_ratio = 0</code> , la pénalité est une pénalité L2. Pour <code>l1_ratio = 1</code> c'est une pénalité de L1. Pour $0 < l1_ratio < 1$, la pénalité est une combinaison de L1 et L2
<code>max_iter</code>	Entier positif Par défaut : 100	Nombre maximum d'itérations de l'algorithme pour l'optimisation

➤ Exemple

```
from sklearn.linear_model import LogisticRegression
```

```
lr = LogisticRegression(penalty='none', fit_intercept=False, random_state=123, max_iter=200) #sans régularisation
lr_ridge = LogisticRegression(penalty='l2', fit_intercept=False, random_state=123, max_iter=200) # Ridge
lr_lasso = LogisticRegression(penalty='l1', fit_intercept=False, random_state=123, max_iter=200) # Lasso
lr_elasticnet = LogisticRegression(penalty='elasticnet', fit_intercept=False, random_state=123, max_iter=200) # Elastic net
```

RÉGRESSIONS LOGISTIQUES

FONCTIONS DE SCIKIT-LEARN (2/2)

⚠ ➔ **SGDClassifier() nécessite d'avoir standardisé les variables explicatives au préalable**

➔ `Sklearn.linear_model.SGDClassifier()` 

Paramètres principaux	Valeurs possibles	Description
loss	Pour effectuer une régression logistique : 'log'	Fonction de perte à utiliser
penalty	'l1', 'l2', 'elasticnet', 'none' Par défaut : 'l2'	Norme utilisée pour la pénalisation
fit_intercept	True ou False Par défaut : True	Activation/désactivation de l'utilisation de l'intercept
random_state	Entier Par défaut : None	Graine aléatoire
l1_ratio	Décimale entre 0 et 1. Par défaut : 0.15	Paramètre de combinaison pour l'ElasticNet, avec $0 \leq l1_ratio \leq 1$. Pour $l1_ratio = 0$, la pénalité est une pénalité L2. Pour $l1_ratio = 1$ c'est une pénalité de L1. Pour $0 < l1_ratio < 1$, la pénalité est une combinaison de L1 et L2
max_iter	Entier positif Par défaut : 1000	Nombre maximum d'itérations de l'algorithme pour l'optimisation

➔ Exemple

```
from sklearn.linear_model import SGDClassifier
```

```
lr = SGDClassifier(loss='log',penalty='none',fit_intercept=False,random_state=123,max_iter=200) #sans régularisation
lr_ridge = SGDClassifier(loss='log',penalty='l1',fit_intercept=False,random_state=123,max_iter=200) # Ridge
lr_lasso = SGDClassifier(loss='log',penalty='l2',fit_intercept=False,random_state=123,max_iter=200) # Lasso
lr_elasticnet = SGDClassifier(loss='log',penalty='elasticnet',fit_intercept=False,random_state=123,max_iter=200) # Elasticnet
```

RÉGRESSIONS LOGISTIQUES

FONCTION DU PACKAGE STATSMODELS

➤ Statsmodels est un module statistique :

- permet d'estimer de nombreux modèles
- réaliser un grand nombre de tests statistiques
- Fonctions pour l'exploration des données

➤ `Statsmodels.api.Logit()` pour la construction de régressions logistiques



```
import statsmodels.api as sm
```

```
#Chargement d'un jeu de données
```

```
spector_data = sm.datasets.spector.load_pandas()
```

```
spector_data.exog = sm.add_constant(spector_data.exog)
```

```
logit_mod = sm.Logit(spector_data.endog, spector_data.exog)
```

```
logit_res = logit_mod.fit()
```

```
print(logit_res.summary())
```

Logit Regression Results						
=====						
Dep. Variable:	GRADE	No. Observations:	32			
Model:	Logit	Df Residuals:	28			
Method:	MLE	Df Model:	3			
Date:	Wed, 20 Jan 2021	Pseudo R-squ.:	0.3740			
Time:	15:22:17	Log-Likelihood:	-12.890			
converged:	True	LL-Null:	-20.592			
Covariance Type:	nonrobust	LLR p-value:	0.001502			
=====						
	coef	std err	z	P> z	[0.025	0.975]

const	-13.0213	4.931	-2.641	0.008	-22.687	-3.356
GPA	2.8261	1.263	2.238	0.025	0.351	5.301
TUCE	0.0952	0.142	0.672	0.501	-0.182	0.373
PSI	2.3787	1.065	2.234	0.025	0.292	4.465
=====						

RÉGRESSIONS LOGISTIQUES

RÉSUMÉ

DÉFINITION GÉNÉRALE

- Basée sur une combinaison linéaire de variables explicatives
- Utilisée pour la classification binaire ($Y=0$ ou $Y=1$)
- Trouver une séparation linéaire entre 2 groupes

Exemple de grille de régression logistique :

Variable	Estimation	P-values
Intercept	1.5	0.016
Var1	-0.3	0.013
Var2	0.2	0.012

Estimation des probabilités :

$$\text{Score} = 1.5 - 0.3 \times \text{Var1} + 0.2 \times \text{Var2}$$
$$P(Y = 1) = \frac{1}{1 + \exp^{-\text{Score}}}$$

Impact de chaque variable sur $Y = 1$:

VAR1  VAR2 

CONSTRUCTION ET RÉSULTAT

Training :

- Résoudre un problème d'optimisation avec différents algorithmes
- Possibilité de régularisation :
 - L1 : ajouter une contrainte sur la somme des $|\beta|$ (Lasso)
 - L2 : ajouter une contrainte sur la somme des β^2 (Ridge)
 - L1 + L2 : combinaison de L1 et L2 (ElasticNet)

Avantages :

- Rapide et simple
- Interprétable sur des petits nombres de variables
- Stable et robuste

Inconvénients :

- Ne capture pas les effets non linéaire et les interactions

Exemple de fonction en Python :



```
from sklearn.linear_model import LogisticRegression  
  
cl = LogisticRegression()  
  
cl.fit(X,Y)
```

01

Introduction à Scikit-Learn

02

Principaux concepts de Scikit-Learn

03

Partitionnement des données

04

Préprocessing

05

Fonctions avancées pour la gestion
des variables catégorielles

06

Régressions logistiques (ridge, lasso,
elasticnet)

07

Algorithmes à base d'arbres

08

Éléments pour l'interprétation

09

Sélection de variables explicatives

10

Optimisation des hyperparamètres

11

Métriques d'évaluation de la
performance des modèles

ALGORITHMES À BASE D'ARBRES

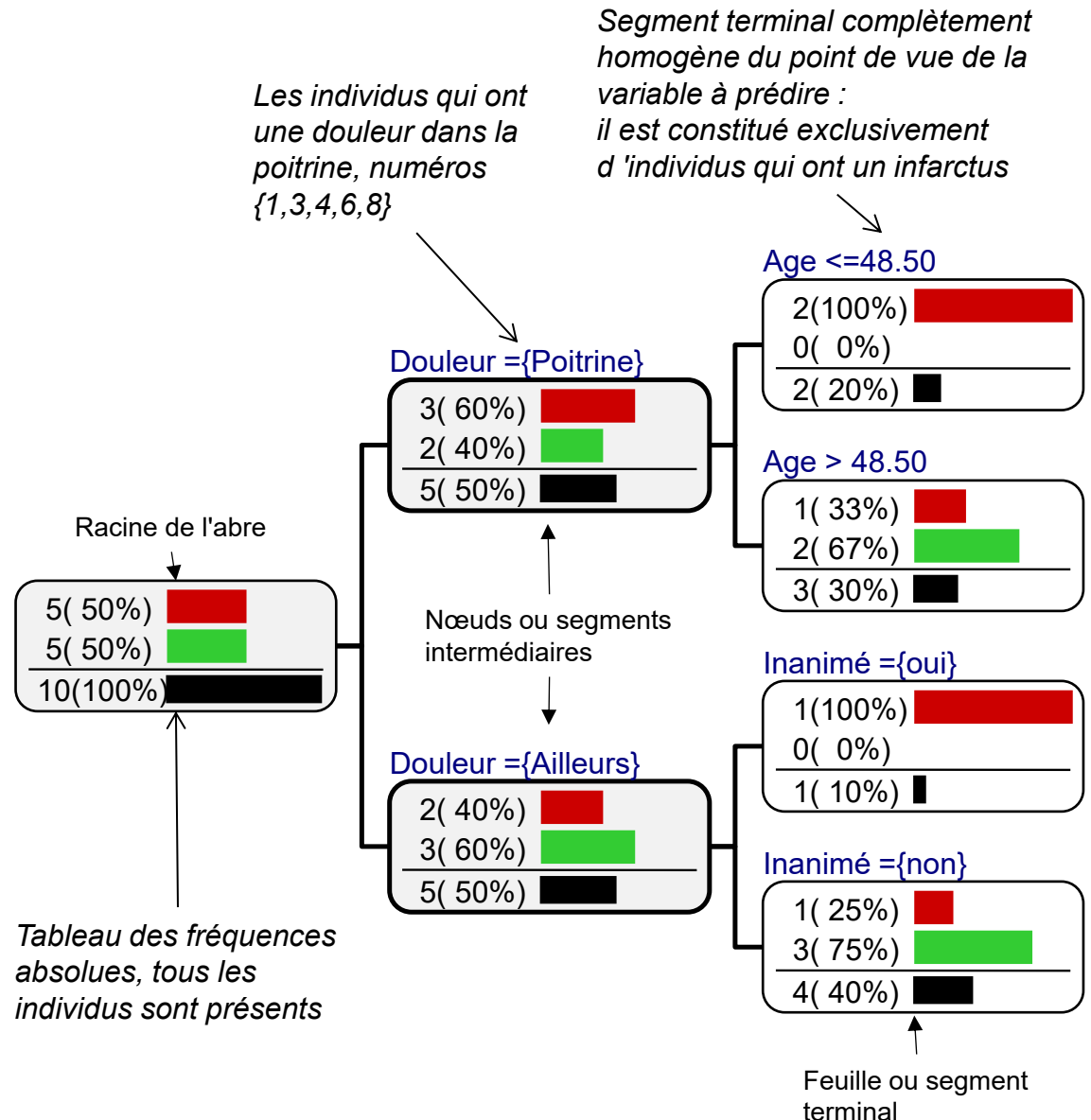
LES ARBRES DE DÉCISION

Numéro	Infarctus	Douleur	Age	Inanimé
1	oui	poitrine	45	oui
2	oui	ailleurs	25	oui
3	oui	poitrine	35	non
4	oui	poitrine	70	oui
5	oui	ailleurs	34	non
6	non	poitrine	60	non
7	non	ailleurs	67	non
8	non	poitrine	52	oui
9	non	ailleurs	58	non
10	non	ailleurs	34	non



Fonctionnement :

- De gauche à droite, on segmente pas-à-pas la population pour obtenir les segments les plus purs possible du point de vue de Y
- A chaque segmentation, on teste toutes les variables X et on choisit la « meilleure »
- Pour les variables continues, on teste toutes les coupures possibles et on choisit la meilleure
- On s'arrête lorsque l'on atteint l'un des critères d'arrêt (taille minimale, segmentation non significative)
- Chaque feuille se voit affecter la catégorie la plus représentée et/ou la probabilité associée à la modalité cible.



ALGORITHMES À BASE D'ARBRES

EXEMPLE – ARBRE DE DÉCISION

➤ `sklearn.tree.DecisionTreeClassifier()` ? i

Paramètres principaux	Valeurs possibles	Description
<code>splitter</code>	{“best”, “random”} Par défaut : “best”	Stratégie utilisée pour choisir la division à chaque nœud. Les stratégies prises en charge sont « best » pour choisir la meilleure division et « random » pour choisir la meilleure division aléatoire.
<code>random_state</code>	int, RandomState instance ou None Par défaut : None	Contrôle le caractère aléatoire de l'estimateur. Les fonctionnalités sont toujours permutées aléatoirement à chaque division, même si le paramètre <code>splitter</code> est défini sur « best »
<code>max_depth</code>	int Par défaut : None	Profondeur d'arbre maximale. Risque d'over-fit si trop grand, et d'under-fit si trop petit
<code>criterion</code>	{“gini”, “entropy”} Par défaut : “gini”	Fonction qui mesure la qualité de division. Les critères pris en charge sont « gini » pour l'impureté de Gini et « entropy » pour le critère d'entropie
<code>min_samples_split</code>	int ou float Par défaut : 2	Le nombre minimum d'éléments requis pour diviser un nœud interne S'il s'agit d'un int, alors <code>min_samples_split</code> correspond au nombre minimum S'il s'agit d'un float, alors <code>min_samples_split</code> est une fraction et <code>ceil(min_samples_split * n_samples)</code> est le nombre minimum d'éléments
<code>max_features</code>	{“auto”, “sqrt”, “log2”}, int ou float Par défaut : “auto”	Nombre de variables à prendre en compte S'il s'agit d'un int, alors <code>max_features</code> correspond au nombre maximum de variables S'il s'agit d'un float alors <code>max_features</code> est une fraction Si « auto » ou « sqrt », alors <code>max_features=sqrt(n_features)</code> If « log2 », alors <code>max_features=log2(n_features)</code> Si None, alors <code>max_features = n_features</code>

➤ Exemple

```
from sklearn.tree import DecisionTreeClassifier
clf = DecisionTreeClassifier(criterion='entropy', splitter='random', min_samples_split=4, random_state=0)
```

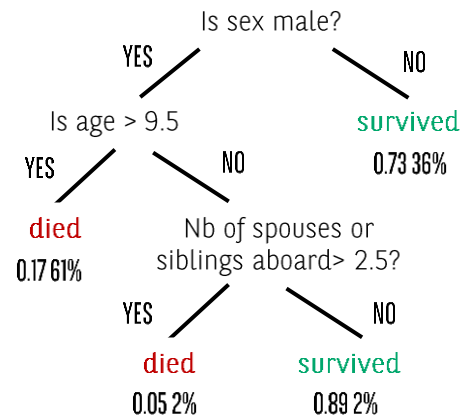
ALGORITHMES À BASE D'ARBRES

LES ARBRES DE DÉCISION

DÉFINITION GÉNÉRALE

- Très utilisé en Data Mining
- Méthode graphique et visuelle avec l'illustration de l'ensemble des règles de décision
- Se compose de :
 - Nœuds : split la population en 2 groupes
 - Feuilles : plus de split, prédiction (décision finale)

Exemple :



¹Tree showing survival of passengers on the "Titanic"

- **Principaux paramètres :**
 - La profondeur maximale
 - Le critère de split

CONSTRUCTION ET RÉSULTAT

Training :

- Pour chaque nœud, mise en compétition de l'ensemble des variables/splits
- Choix de la meilleure variable avec le meilleur split selon un critère (Gini / Entropie)
- Arrêt de l'algorithme quand l'une des conditions est remplie (paramètres)

Avantages :

- Rapide et simple
- Peut capturer les interactions
- Compréhension et interprétation facile (arbre peu profond)

Inconvénients :

- Forte possibilité de surapprentissage
- En général, moins performant que les autres algorithmes
- Nécessite des effectifs importants

En Python :



```
from sklearn.tree import DecisionTreeClassifier

cl = DecisionTreeClassifier()

cl.fit(X,Y)
```

ALGORITHMES À BASE D'ARBRES

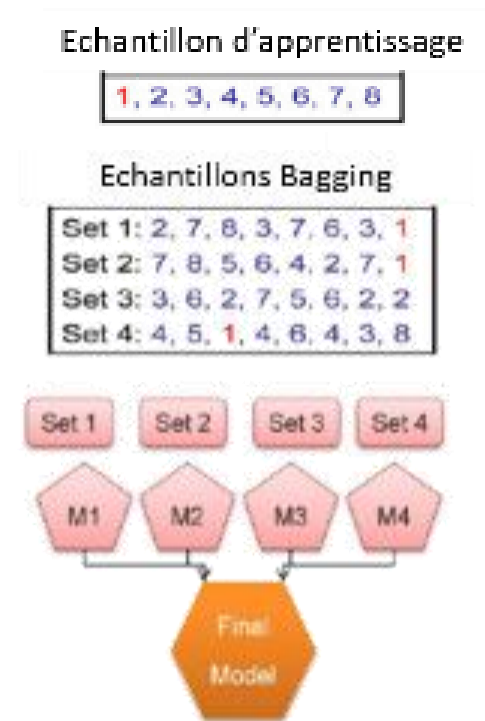
NOUVELLES APPROCHES ENSEMBLISTES

- **APPROCHE ENSEMBLISTE** : faire coopérer plusieurs modèles prédictifs (parfois des milliers) et agréger les différentes prédictions obtenues pour calculer la prédiction finale (Vote/moyenne simple ou pondéré(e)).
- **Pour simplifier, il existe 4 types d'approches** :
 - **BLENDING** : Construire différents types de modèles (Arbre, Régression logistique, Réseau de neurones) sur les mêmes données d'apprentissage
 - **BAGGING** : Construire des modèles de même nature (même algo d'apprentissage) sur différentes versions de données d'apprentissage (tirage aléatoire avec remise)
 - **BOOSTING** : Construire des modèles successivement sur différentes versions de données d'apprentissage en augmentant le poids des individus mal classés
 - **GRADIENT BOOSTING** : Construire des modèles successivement (même algo d'apprentissage) en modélisant à chaque étape les erreurs de prédiction de l'étape précédente. Possibilité de sélectionner aléatoirement les individus (tirage aléatoire avec remise) et les variables à chaque étape (tirage aléatoire sans remise)
- En général, **chaque modèle individuel prédit moins bien qu'un modèle « classique »**, mais l'agrégation des prédictions de tous les modèles permet de **réduire le biais et la variance de l'erreur** (si le paramétrage est cohérent...)
- **En réalité, il n'est pas rare de voir ces approches entremêlées, voire réalisées séquentiellement sur plusieurs niveaux**

ALGORITHMES À BASE D'ARBRES

LE BAGGING – BOOSTRAP AGGREGATING (BREIMAN, 1996)

1. **Choix d'un algorithme** (Arbre, RégLog, RégLinéaire...)
2. **Choix du nombre de modèles** à construire : B
3. **Construction de B échantillons par tirage aléatoire avec remise (bootstrap)** à partir de l'échantillon initial.
4. **Construction de B modèles** : un par échantillon
5. **Prédiction finale** obtenue en calculant :
 - **Si Y continue** :
 - → Moyenne des prédictions des B modèles.
 - **Si Y nominale** :
 - **Modalité la plus fréquente** dans les conclusions des B modèles
 - **Probabilité associée à la modalité la plus fréquente** :
 - (nb de modèles qui classent l'individu dans la modalité la plus fréquente / B)
 - Moyenne des probabilité d'appartenance à la modalité la plus fréquente dans les différents modèles. Possible uniquement si l'algorithme choisi renvoie une probabilité.
 - On peut aussi calculer la probabilité pour les autres modalités de Y



Remarque : Au-delà d'un certain nombre de modèles, il n'y a plus d'améliorations notables. Mais il n'y a pas de dégradation non plus (pas d'overfitting)

ALGORITHMES À BASE D'ARBRES

NOUVELLES APPROCHES ENSEMBLISTES

- **Intérêt de la coopération** : Faire coopérer des modèles n'a d'intérêt que si les modèles ne classent/prédisent pas tous de la même manière (si le vote est systématiquement unanime, autant n'avoir qu'un seul modèle).
- **Biais et variance** : Biais bagging = biais du modèle sous-jacent. Bagging réduit avant tout la variance. Il faut donc que les modèles sous-jacents aient un biais faible, capturant la complexité des relations entre y et les X .
- **Apprenants faibles** : Bagging ne sait pas tirer parti des apprenants faibles (weak classifier – cf. boosting). Il faut que les modèles sous-jacents soient de bonne qualité.
- **Sur-apprentissage (overfitting)** : Augmenter B n'aboutit pas au sur-apprentissage. En pratique, une centaine suffit, mais on peut l'ajuster à l'étude.
- **Arbres de décision** : Bagging peut s'appliquer à tout type de modèle. Mais les arbres sont particulièrement avantageux parce qu'on peut réduire le biais individuel en créant des arbres profonds (non élagués), la variance (individuelle de chaque arbre) est alors compensée par le mécanisme de coopération en classement.

ALGORITHMES À BASE D'ARBRES

RANDOM FOREST (BREIMAN, 2001) ET EXTRA TREES (GEURTS, ERNST ET WEHENKEL, 2006)

- Random Forest et Extra Trees sont basés sur le **même raisonnement que le bagging d'arbres de décision : B arbres de décision entraînés sur des échantillons d'observations tirés aléatoirement avec remise.**
- **Une différence majeure :** introduction d'un **aléas supplémentaire sur le choix des variables explicatives et leur utilisation**
- **Algorithme :**
 1. **Choix du nombre d'arbres de décision à construire :** B
 2. **Choix du nombre de variables explicatives à utiliser parmi les p disponibles :** m
 3. **Construction des B tables d'apprentissage :** pour chaque arbre, les individus et les variables sont sélectionnées aléatoirement. Tirage aléatoire avec remise pour les individus, tirage aléatoire de m variables explicatives parmi p.
 4. **Création de chaque arbre de décision :**
 - **Random Forest:**
 1. Pour chaque variable sélectionnée, optimisation du critère de découpage (Gini, entropie de Shannon)
 2. Sélection de la variable présentant le meilleur découpage (idem arbres)
 - **Extra-Trees :**
 1. Pour chaque variable, le critère de découpage est choisi aléatoirement
 2. La variable ayant le meilleur résultat par rapport au critère de split est sélectionnée
 3. Cette technique permet de réduire la variance mais augmente légèrement le biais
 5. **Calcul de la prédiction finale (CF Bagging)**

ALGORITHMES À BASE D'ARBRES

SCHÉMA D'UN RANDOM FOREST / EXTRA-TREES

Phase d'apprentissage

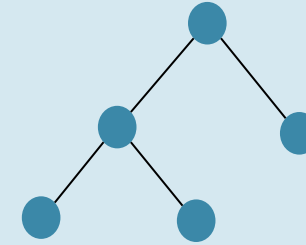
	V1	V2	V3	V4	V5	V6	V7
1							
2							
3							
4							
5							
6							
7							
8							
9							

Table d'apprentissage

Echantillonnage N°1

	V1	V4	V5	V7
3				
3				
5				
8				
9				

Création d'un Arbre de décision (Arbre n°1)

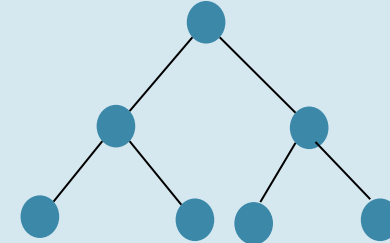


Ces opérations sont effectuées autant de fois, qu'il y a d'arbres dans la forêt (paramètre B)

Echantillonnage N°B

	V1	V2	V5	V6
1				
4				
5				
7				
9				

Création d'un Arbre de décision (Arbre N° B)



Application du modèle (cas binaire)

	V1	V2	V3	V4	V5	V6	V7
10							

Observation à modéliser

Modélisations avec les arbres de décision

Résultats arbre 1 :
 $P(Y=1) = 0.8$

Résultats arbre 2 :
 $P(Y=1) = 0.7$

...

Résultats arbre B :
 $P(Y=1) = 0.6$

Calcul de la moyenne

Résultat final:
 $P(Y=1) = 0.7$
Prédiction :
 $Y=1$

ALGORITHMES À BASE D'ARBRES

RANDOM FOREST ET EXTRA TREES : AVANTAGES ET INCONVÉNIENTS

■ Avantages

- **Bonnes performances en prédiction** : Paramétrage simple (B et m). Meilleures que celles du bagging en général
- **Pas de problème d'overfitting** (on peut augmenter B)
- **Mesure de l'importance des variables**
- **Evaluation de l'erreur intégrée (OOB : out of bag)** : pour chaque modèle, l'erreur est calculée à partir des individus qui n'ont pas été sélectionnés
- **Parallélisation des traitements**

■ Inconvénients

- **Problème si nombre de variables pertinentes très faible**, dans l'absolu et relativement au nombre total de variables
- Comme pour le bagging en général, le **déploiement** de ce type de modèle est **compliqué surtout s'il est nécessaire de le traduire dans un autre langage...**

ALGORITHMES À BASE D'ARBRES

EXEMPLE - RANDOMFOREST

➤ `sklearn.ensemble.RandomForestClassifier()` ? i

Paramètres principaux	Valeurs possibles	Description
<code>n_jobs</code>	int Par défaut : None	Nombre de threads parallèles utilisés pour exécuter XGBoost
<code>n_estimators</code>	int Par défaut : 100	Nombre d'arbres à entraîner
<code>max_depth</code>	int Par défaut : None	Profondeur d'arbre maximale. Risque d'over-fit si trop grand, et d'under-fit si trop petit
<code>criterion</code>	{“gini”, “entropy”} Par défaut : “gini”	Fonction qui mesure la qualité de division. Les critères pris en charge sont « gini » pour l'impureté de Gini et « entropy » pour le critère d'entropie
<code>min_samples_split</code>	int or float Par défaut : 2	Le nombre minimum d'éléments requis pour diviser un nœud interne S'il s'agit d'un int, alors <code>min_samples_split</code> correspond au nombre minimum S'il s'agit d'un float, alors <code>min_samples_split</code> est une fraction et <code>ceil(min_samples_split * n_samples)</code> est le nombre minimum d'éléments
<code>max_features</code>	{“auto”, “sqrt”, “log2”}, int ou float Par défaut : “auto”	Nombre de variables à prendre en compte S'il s'agit d'un int, alors <code>max_features</code> correspond au nombre maximum de variables S'il s'agit d'un float alors <code>max_features</code> est une fraction Si « auto » ou « sqrt », alors <code>max_features=sqrt(n_features)</code> If « log2 », alors <code>max_features=log2(n_features)</code> Si None, alors <code>max_features = n_features</code>
<code>bootstrap</code>	bool Par défaut : True	Indique l'utilisation ou non d'échantillons bootstrap lors de la création d'arbres. Si False, l'ensemble des données est utilisé pour construire chaque arbre

➤ Exemple

```
from sklearn.ensemble import RandomForestClassifier
clf = RandomForestClassifier(n_estimators = 500, max_depth=2)
```

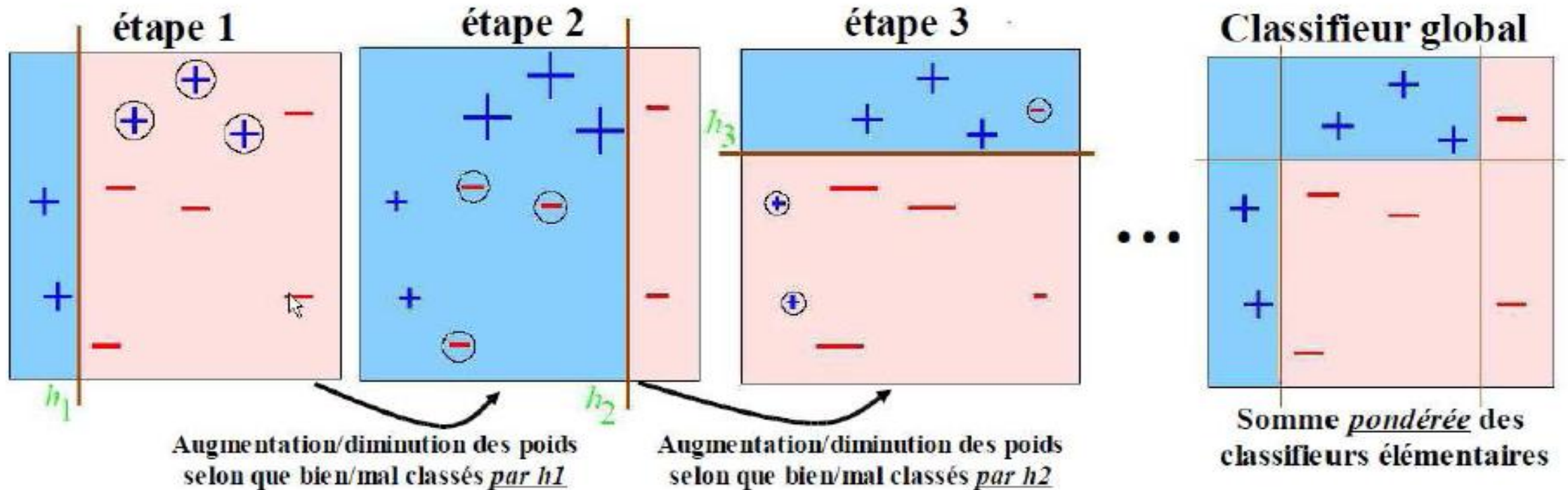
ALGORITHMES À BASE D'ARBRES

LE BOOSTING (FREUND-SHAPIRE, 1996)

- **IDÉE SOUS-JACENTE** : Toujours apprendre sur différentes versions des données. Mais mieux diriger l'apprentissage en se focalisant sur les individus mal classés à l'étape précédente.
- **Apprenants faibles (weak learner)** : Modèle qui fait juste un peu mieux que le hasard.
- **Boosting** : Combiner des weak learners de manière appropriée permet de produire un modèle performant, nettement meilleur que chaque modèle pris individuellement.
- **Pondération des individus** : A l'étape $(b+1)$, l'idée est de donner une pondération plus élevée aux individus mal prédits par le modèle b . La construction des modèles est séquentielle.
- **Pondération des modèles** : Système de vote pondéré (selon la performance) en classement.
- **Biais et variance** : En orientant l'apprentissage à chaque étape, boosting agit sur le biais ; en les combinant, il agit sur la variance.
- **Sur-apprentissage (overfitting)** : Augmenter B n'aboutit pas au sur-apprentissage
- **Arbres de décision** : Boosting peut s'appliquer à tout type de modèle. Mais les arbres sont avantageux parce qu'on peut moduler les propriétés du modèle (plus ou moins de profondeur)

ALGORITHMES À BASE D'ARBRES

LE BOOSTING (FREUND-SHAPIRE, 1996) : ILLUSTRATION



ALGORITHMES À BASE D'ARBRES

LE BOOSTING (FREUND-SHAPIRE, 1996) : ALGORITHME

1. Choix d'un algorithme (Arbre, RégLog, RégLinéaire...)

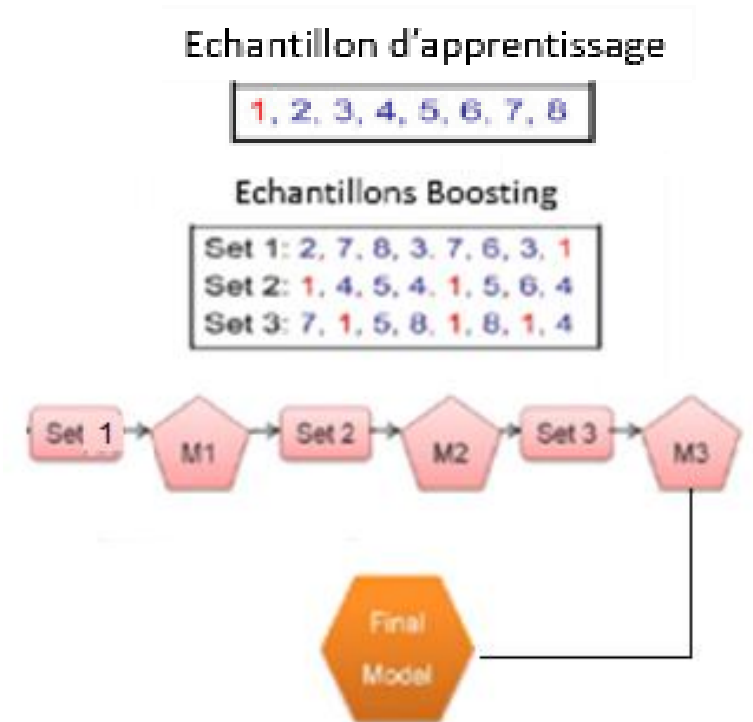
2. Choix du nombre de modèles à construire : B

3. Modélisation :

- **Processus itératif** basé sur une succession de modèles construits sur des individus tirés aléatoirement avec remise et l'introduction d'une pondération donnant plus de poids aux individus mal classés/scorés par le modèle construit lors de l'itération précédente.
- L'algorithme peut s'arrêter avant d'avoir construit les B modèles si le taux d'erreur devient nul ou s'il atteint un niveau trop élevé

4. Prédiction finale obtenue en calculant :

- **Si Y continue :**
 - **Moyenne des prédictions** des B modèles, pondérée selon la performance de prédiction de chaque modèle.
- **Si Y nominale :** même fonctionnement que pour le bagging avec la pondération des prédictions de chaque modèle selon la performance globale du modèle.



ALGORITHMES À BASE D'ARBRES

ARBRES BOOSTÉS – GRADIENT BOOSTING TREES

- La **descente du gradient** est une **technique itérative** qui permet d'approcher la solution d'un problème d'optimisation. En apprentissage supervisé, la construction du modèle revient souvent à déterminer les paramètres (du modèle) qui permettent d'optimiser (max ou min) une fonction objectif
- **IDÉE SOUS-JACENTE : modéliser séquentiellement les résidus du modèle précédent** tant que l'on arrive à optimiser la fonction objectif.
- **Algorithme**
 1. **Arbre n°1** : construit comme en data mining
 2. **Calcul de l'erreur de prédiction** pour chaque individu
 3. **Arbre n°2** : construit en modélisant non plus la variable cible Y mais l'erreur de prédiction
 4. **Mise à jour des prédictions** : somme des scores / prédictions des arbres précédents pour chaque répondant
 5. **Calcul des erreurs individuelles de prédictions et de l'erreur globale**
 6. **et +** : On continue de construire de nouveaux arbres tant qu'on arrive à faire baisser l'erreur de prédiction globale en répétant les étapes 3 à 5

ALGORITHMES À BASE D'ARBRES

ARBRES BOOSTÉS – ILLUSTRATION

● Illustration avec un exemple volontairement simplifié



- Contexte :

- Données issues d'une enquête online, réalisée sur un panel de plusieurs centaines de milliers de personnes.
- Avec cette enquête, nous connaissons l'âge, le genre, le niveau d'équipement et les loisirs des répondants
- Parmi les questions posées : Aimez-vous les jeux vidéo ?
Donnez une note de 0 (non, pas du tout) à 10 (oui, beaucoup).

- Objectif de l'étude : Modéliser l'attrait pour les jeux vidéo à partir des informations disponibles

- Pour notre compréhension, nous allons suivre plus particulièrement ces 5 personnes.

ALGORITHMES À BASE D'ARBRES

ARBRES BOOSTÉS – ILLUSTRATION

Structure des données à analyser :

- **En ligne** : les répondants
- **En colonne** : les réponses aux différentes questions et les caractéristiques des répondants

			X: Les variables potentiellement explicatives					Y
ID	Date saisie	Prénom	Age	Genre	Détention tablette	Détention PC	Fréquence Utilisation PC	Aimez-vous les jeux vidéo ?
1	29/06/2013	Kevin	8	M	1	1	Plusieurs fois par semaine	10
2	11/05/2014	Mathilde	12	F	1	0	Non concerné	8
3	06/10/2011	Gaston	56	M	0	0	Non concerné	2
4	17/08/2012	Isabelle	22	F	1	1	Tous les jours	5
5	23/03/2015	Renée	72	F	1	1	Plusieurs fois par semaine	1
...								

Objectif : expliquer Y à partir de X

ALGORITHMES À BASE D'ARBRES

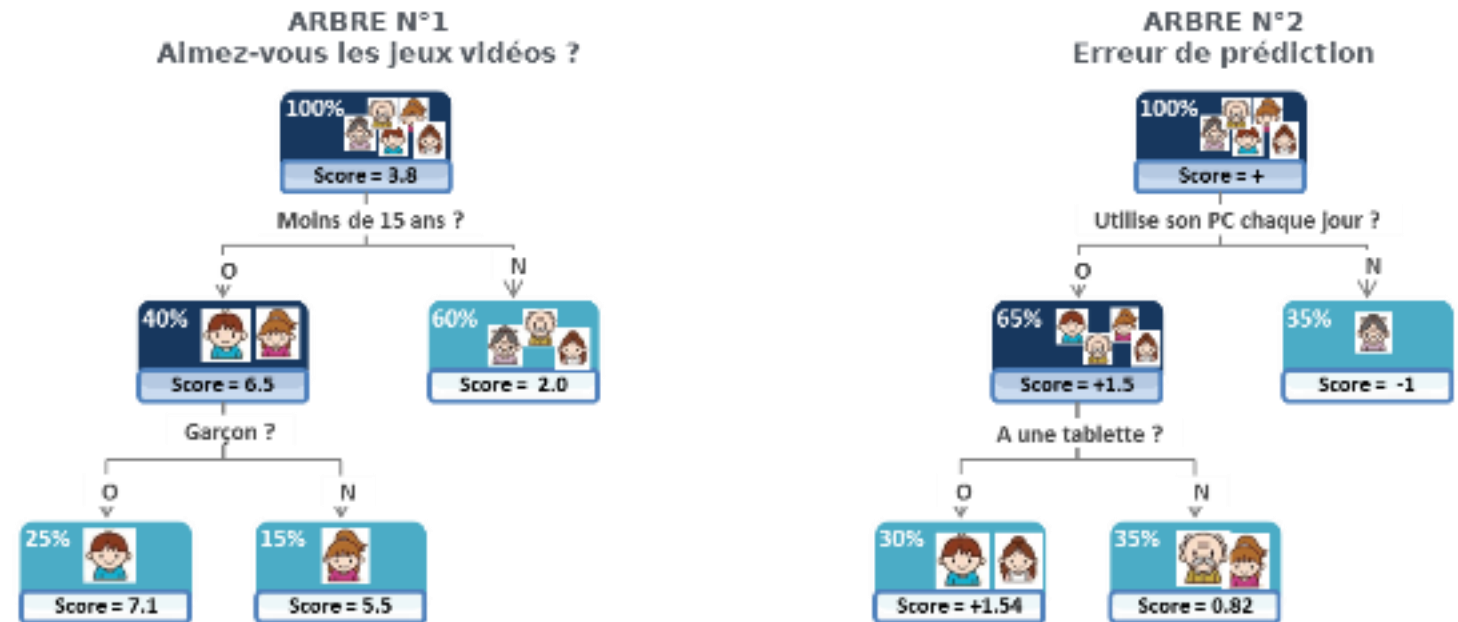
ARBRES BOOSTÉS – ILLUSTRATION

Fonctionnement :

- **Arbre n°1 : construit comme en data mining.**
- **Calcul de l'erreur de prédiction pour chaque répondant.**
- **Arbre n°2 : construit en modélisant non plus « Aimez-vous les jeux vidéos ? » mais l'erreur de prédiction de l'arbre précédent.**
- **Mise à jour des prédictions :** somme des scores des arbres précédents pour chaque répondant.
- **Mise à jour des erreurs de prédictions**
- **On continue** de construire de nouveaux arbres tant qu'on arrive à faire baisser l'erreur de prédiction en répétant les étapes 3 à 5

- On calcule **plusieurs arbres de décision successivement.**

Les arbres sont moins profonds et contiennent moins de segments. Chaque arbre est moins bon que celui du data mining mais les prédictions finales sont meilleures :



- Les prédictions finales (i.e. notes prédites) des répondants sont calculées en sommant les scores obtenus dans chaque arbre :

Note prédite () = 7.1 + 1.54 = 8.64

Note prédite () = 2 - 1 = 1

ALGORITHMES À BASE D'ARBRES

ARBRES BOOSTÉS – ILLUSTRATION

On obtient dans cet exemple de meilleurs résultats qu'avec un arbre unique.
 Dans la pratique, c'est quasiment systématique.
 Les résultats sont même souvent bluffants avec relativement peu d'efforts !

Répondant	Aimez-vous les jeux vidéo ?	Score prédit par l'arbre n°1	Ecart Etape 1	Score prédit par l'arbre n°2	Note prédite par les 2 arbres	Ecart Etape 2
	10	7.1	2.9	+1.54	8.64	1.36
	8	5.5	2.5	+0.82	6.32	1.68
	2	2	0	+0.82	2.82	-0.82
	5	2	3	+1,54	3.54	1.46
	1	2	-1	-1	1	0
...	...	...	...	...	...	...

- **Le modèle est-il apparemment bon ?**
- **Note prédite** = SOMME(scores des différents arbres)
- **Erreur moyenne de prédiction** en valeur absolue (MAE) :
 - Etape 1 : 1.88
 - Etape 2 : 1.06

ALGORITHMES À BASE D'ARBRES

EXEMPLE - XGBOOST

➤ `xgboost.XGBClassifier()` 

Paramètres principaux	Valeurs possibles	Description
<code>n_jobs</code>	int Par défaut : 1	Nombre de threads parallèles utilisés pour exécuter XGBoost
<code>n_estimators</code>	int Par défaut : 100	Nombre d'arbres à entraîner
<code>learning_rate</code>	float Par défaut : 0.1	Contrôle la vitesse à laquelle on converge lors de la descente du gradient fonctionnelle
<code>gamma</code>	float Par défaut : 0.0	Diminution minimale de la valeur de la loss (fonction objectif) pour prendre la décision de partitionner une feuille
<code>max_depth</code>	int Par défaut : 3	Profondeur d'arbre maximale. Risque d'over-fit si trop grand, et d'under-fit si trop petit
<code>colsample_bytree</code>	float Par défaut : 1.0	Pourcentage des colonnes utilisées pour construire chaque arbre
<code>min_child_weight</code>	int Par défaut : 1	Effectif / Poids minimum d'un segment terminal dans un arbre
<code>subsample</code>	float Par défaut : 1.0	Fraction de l'échantillon d'apprentissage utilisée pour construire chaque arbre (compris entre 0 et 1)

➤ Exemple

```
import xgboost as xgb
```

```
xgbc = xgb.XGBClassifier(colsample_bytree=0.4, gamma=0, learning_rate=0.3, max_depth=6, n_estimators=500, n_jobs=6)
```

ALGORITHMES À BASE D'ARBRES

EXEMPLE - LIGHTGBM

➤ `lightgbm.LGBMClassifier()` 

Paramètres principaux	Valeurs possibles	Description
<code>n_jobs</code>	int Par défaut : -1 (<= 0 signifie sans limite)	Nombre de threads parallèles utilisés pour exécuter XGBoost
<code>n_estimators</code>	int Par défaut : 100	Nombre d'arbres à entraîner
<code>learning_rate</code>	float Par défaut : 0.1	Contrôle la vitesse à laquelle on converge lors de la descente du gradient fonctionnelle
<code>max_depth</code>	int Par défaut : -1 (<= 0 signifie sans limite)	Profondeur d'arbre maximale. Risque d'over-fit si trop grand, et d'under-fit si trop petit
<code>colsample_bytree</code>	float Par défaut : 1.0	Pourcentage des colonnes pris pour construire un arbre
<code>min_child_samples</code>	int Par défaut : 1	Effectif / Poids minimum d'un segment terminal dans un arbre
<code>subsample</code>	float Par défaut : 1.0	Fraction de l'échantillon d'apprentissage utilisée pour construire chaque arbre (compris entre 0 et 1)

➤ Exemple

```
import lightgbm as lgb
```

```
lgbc = lightgbm.LGBMClassifier(max_depth=8, learning_rate=0.4, n_estimators=800, colsample_bytree=1.0, n_jobs=6)
```

ALGORITHMES À BASE D'ARBRES

STACKING ET BLENDING

Induit une forte complexité tant dans la phase de construction que d'industrialisation

STACKING :

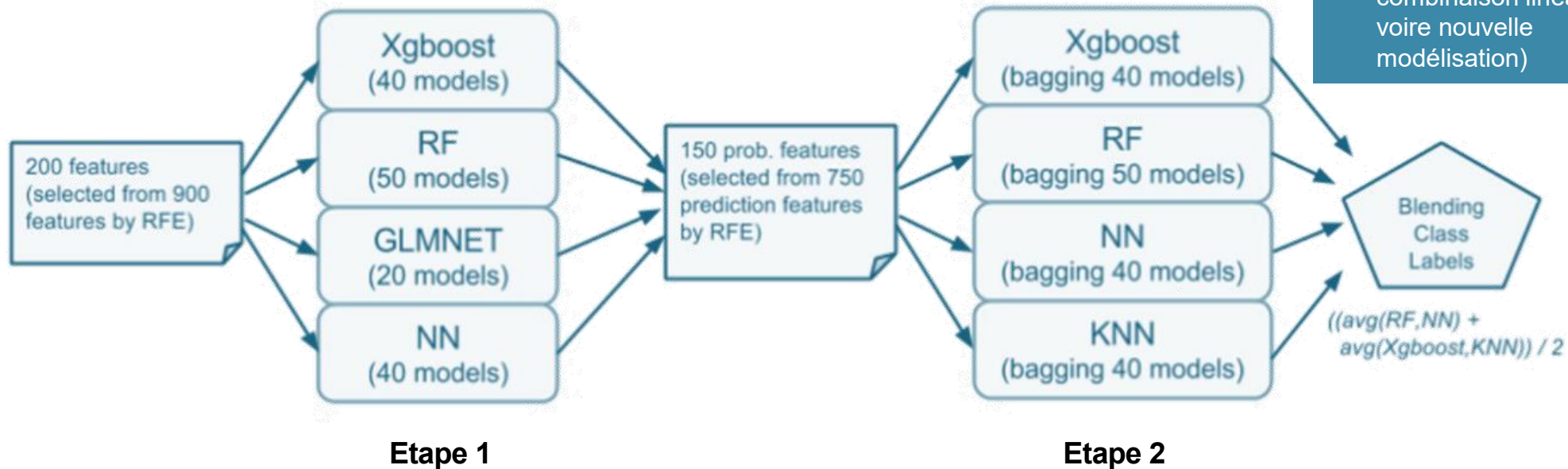
Prédiction à partir de prédictions

- **1ère étape** : on construit plusieurs modèles avec des algorithmes différents
- **2ème étape** : on construit un ou plusieurs modèles à partir des prédictions des modèles précédents (variables explicatives)

BLENDING :

Mélange de modèles

- Phase finale de la modélisation.
- Assemblage de modèles (moyenne, combinaison linéaire, voire nouvelle modélisation)



Xgboost : arbres boostés
RF : Random Forests – Forêts aléatoires
GLMNET : régressions logistiques (lasso, ridge, ...)

NN : Neural Network – Réseaux de Neurones
KNN : K Nearest Neighbors, K plus proches voisins

ALGORITHMES À BASE D'ARBRES

QUELQUES DÉRIVES



NETFLIX n'a jamais implémenté la solution développée par les gagnants de leur concours à 1 Million de dollars pour optimiser leur moteur de recommandations



POURQUOI ?

"This is a truly impressive compilation and culmination of years of work, blending hundreds of predictive models to finally cross the finish line," they say. "We evaluated some of the new methods offline but the additional accuracy gains that we measured did not seem to justify the engineering effort needed to bring them into a production environment."

CLASSIFICATION SUPERVISÉE

BILAN

Critère	Logistic Regression	DecisionTree	RandomForest	XGBoost	LightGBM
Complexité	● ○ ○ ○ ○	○ ○ ○ ○ ○	● ● ● ● ○	● ● ● ● ○	● ● ● ● ○
Transparence	● ● ● ● ●	● ● ● ● ●	● ○ ○ ○ ○	○ ○ ○ ○ ○	○ ○ ○ ○ ○
Stabilité	● ● ● ● ●	● ● ○ ○ ○	● ● ● ● ○	● ● ● ○ ○	● ● ● ○ ○
Gain potentiel	● ● ● ○ ○	● ● ○ ○ ○	● ● ● ● ●	● ● ● ● ●	● ● ● ● ●
Rapidité d'exécution	● ● ● ● ●	● ● ● ● ●	● ● ○ ○ ○	● ● ● ○ ○	● ● ● ● ○
Parcimonie	● ● ● ● ●	● ● ● ● ●	● ● ○ ○ ○	● ● ○ ○ ○	● ● ○ ○ ○

01

Introduction à Scikit-Learn

02

Principaux concepts de Scikit-Learn

03

Partitionnement des données

04

Préprocessing

05

Fonctions avancées pour la gestion
des variables catégorielles

06

Régressions logistiques (ridge, lasso,
elasticnet)

07

Algorithmes à base d'arbres

08

Éléments pour l'interprétation

09

Sélection de variables explicatives

10

Optimisation des hyperparamètres

11

Métriques d'évaluation de la
performance des modèles

ÉLÉMENTS POUR L'INTERPRÉTATION

INTRODUCTION

Globalement, les Français sont méfiants vis-à-vis de l'intelligence artificielle. Cette méfiance est renforcée par l'avènement du big data et la multiplication des données disponibles pour l'entraînement d'intelligences artificielles.

En effet, les risques sont potentiellement nombreux :

- utilisation frauduleuse de données personnelles
- présence de biais discriminatoires...

L'IA peut même devenir un outil de surveillance dans certains pays ...

Au sein de l'Union européenne, les pouvoirs publics sont conscients de ces risques et se saisissent du sujet, que ce soit au niveau européen avec la RGDP ou national (avec la CNIL en France).

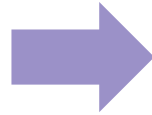
→ Comprendre et être en mesure d'expliquer la prédiction d'un modèle de Machine Learning est donc d'autant plus important

ÉLÉMENTS POUR L'INTERPRÉTATION

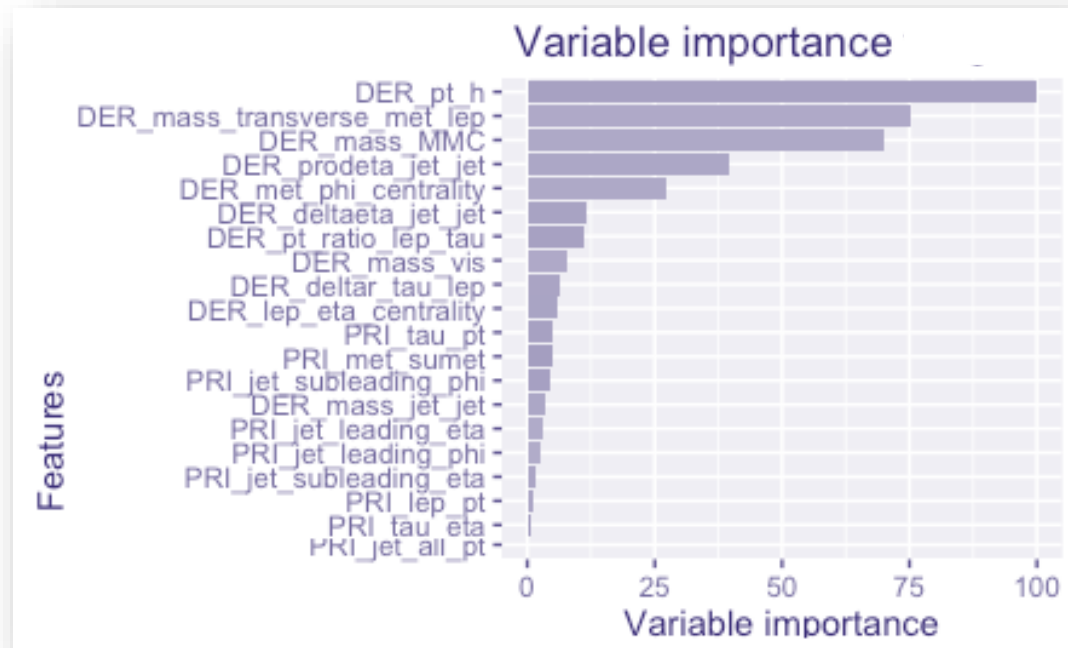
INTERPRÉTATION VERSUS BOITE NOIRE

- En pratique, dans les modèles à base d'arbres, on enchaîne **des centaines voire des milliers d'arbres**.

On perd en lisibilité et donc en compréhension / connaissances



- Le **modèle** devient **illisible** et **ininterprétable**... mais dans certains cas la précision prime sur la compréhension (Risques de défaillance machine, recommandations en ligne, automates)



- Les principales informations disponibles dans Scikit-Learn pour aider à la compréhension concernent l'importance des variables et leur impact dans les différents arbres construits



```
# Build a forest and compute the impurity-based feature
importances
forest = ExtraTreesClassifier(n_estimators=250,
                             random_state=0)

forest.fit(X, y)
importances = forest.feature_importances_
```

ÉLÉMENTS POUR L'INTERPRÉTATION LIBRAIRIE SHAP (1/2)

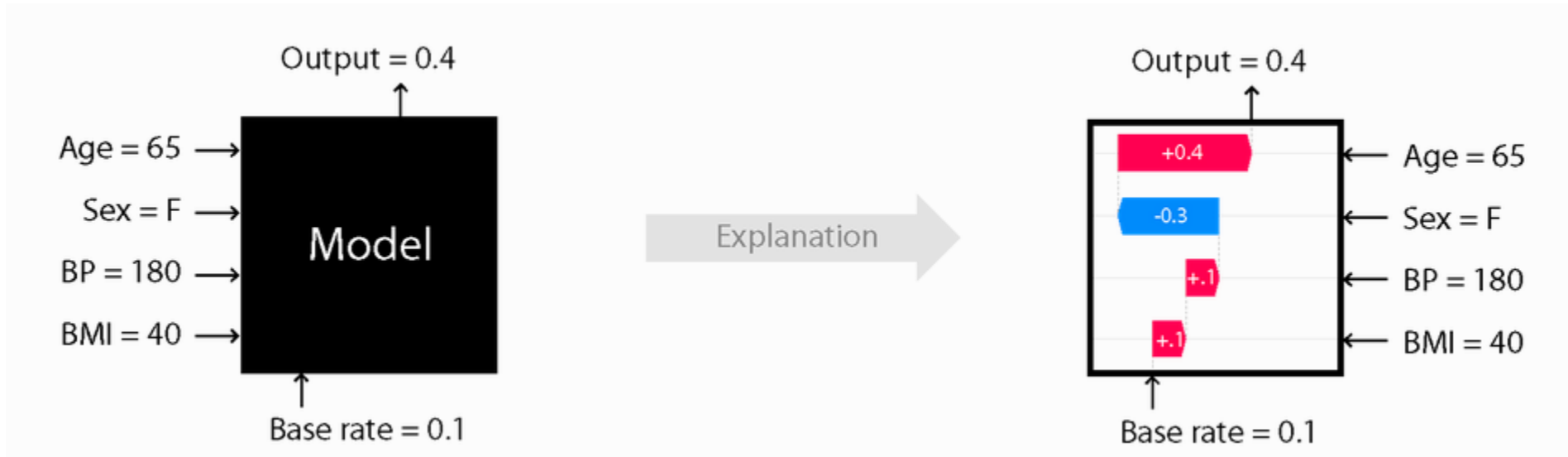


La librairie SHAP utilise les **valeurs de Shapley** pour rendre les modèles de Machine Learning **plus explicables**.

Les valeurs de Shapley sont largement utilisées dans la **théorie des jeux coopératifs**.

Dans la théorie des jeux, les valeurs de Shapley donnent une répartition équitable des gains aux joueurs en fonction de leurs **contributions**.

Dans le cadre de l'explicabilité des modèles de Machine Learning, elles sont utilisées pour déterminer la **contribution de chaque variable à la prédiction finale**



ÉLÉMENTS POUR L'INTERPRÉTATION

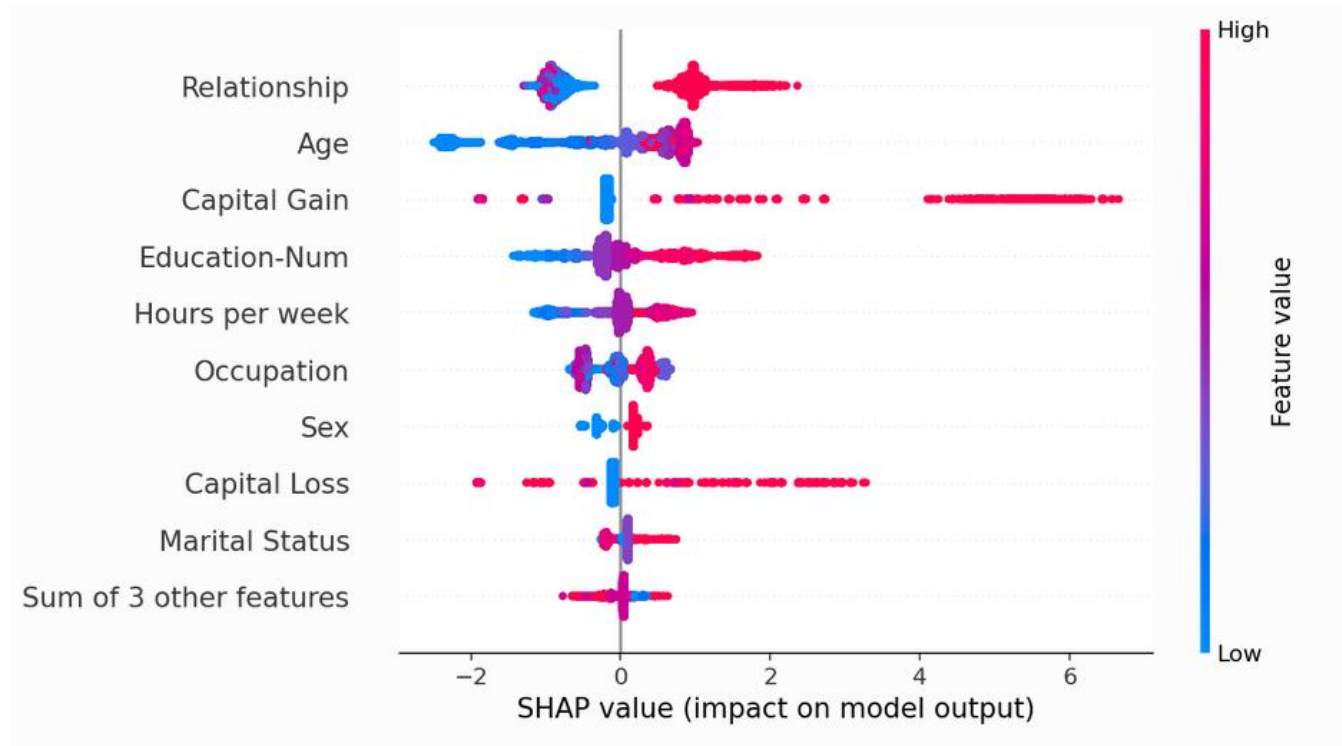
LIBRAIRIE SHAP (2/2)



```
# train XGBoost model
model = xgboost.XGBClassifier(n_estimators=100, max_depth=2).fit(X_adult, y_adult)

# compute SHAP values
explainer = shap.Explainer(model, background_adult)
shap_values = explainer(X_adult)

shap.plots.beeswarm(shap_values)
```



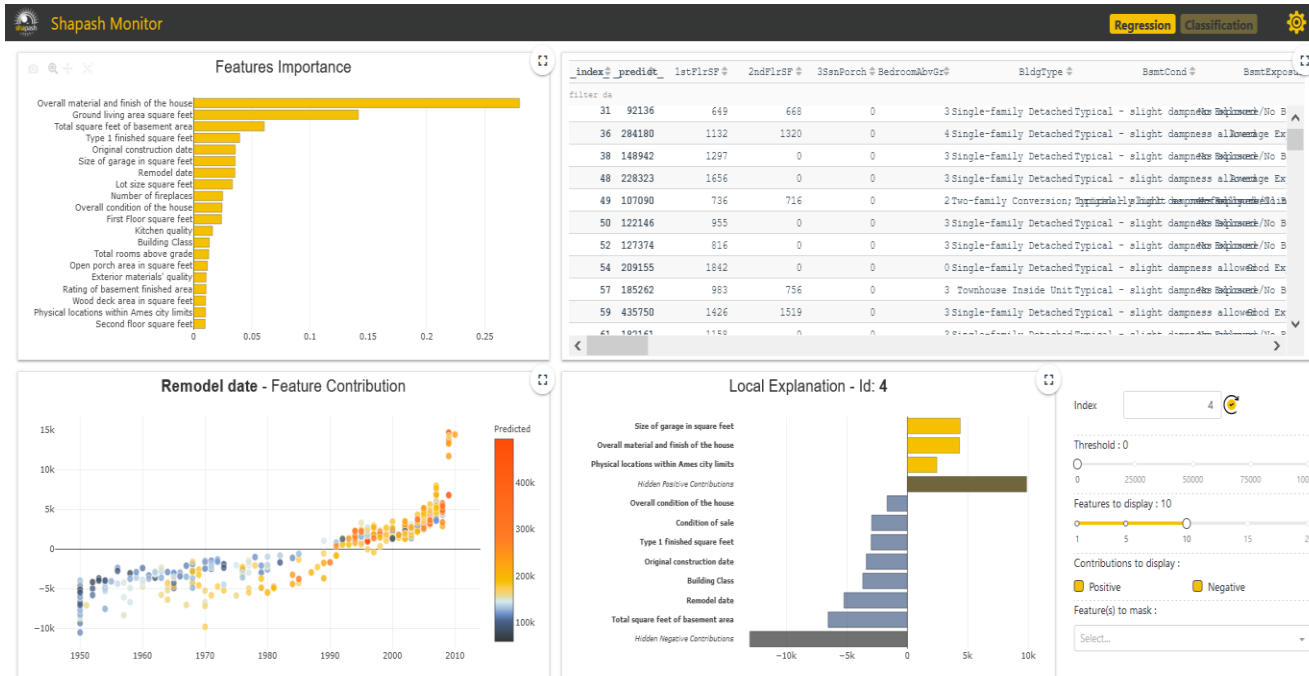
ÉLÉMENTS POUR L'INTERPRÉTATION LIBRAIRIE SHAPASH BY MAIF



Afin de permettre la compréhension des modèles de Machine Learning par un large public, la Maif a développé une librairie disponible en open source : **Shapash**

Cette librairie est une **surcouche d'autres librairies d'explicabilité** comme Shap.

Elle permet de **faciliter l'exploration des différentes fonctions d'un modèle** et dispose d'une interface via une WebApp qui est particulièrement utile pour animer des ateliers métiers et répondre en séance aux **questions sur le fonctionnement du modèle de Machine Learning**



```
from shapash.explainer.smart_explainer import SmartExplainer
```

```
xpl = SmartExplainer(features_dict=house_dict) # optional  
parameter, specifies label for features name
```

```
xpl.compile(  
    x=Xtest,  
    model=regressor,  
    preprocessing=encoder, # Optional  
    y_pred=y_pred # Optional  
)
```

```
app = xpl.run_app()
```

01

Introduction à Scikit-Learn

02

Principaux concepts de Scikit-Learn

03

Partitionnement des données

04

Préprocessing

05

Fonctions avancées pour la gestion des variables catégorielles

06

Régressions logistiques (ridge, lasso, elasticnet)

07

Algorithmes à base d'arbres

08

Éléments pour l'interprétation

09

Sélection de variables explicatives

10

Optimisation des hyperparamètres

11

Métriques d'évaluation de la performance des modèles

SÉLECTION DE VARIABLES EXPLICATIVES

MÉTHODES AUTOMATISÉES (RFE, RFECV)

■ Le choix de variables est très important :

- Trop de variables peut nuire à la qualité du modèle
- Des variables répétitives ou trop liées (ex : forte corrélation) peuvent nuire à la qualité du modèle

■ Chargement des données:

```
from sklearn.linear_model import LogisticRegression
from sklearn.feature_selection import RFE
import pandas as pd

path_data = 'C:/Formation Python/Datasets/'
df = pd.read_csv(path_data + 'diabete.csv', sep = ',')

#Ajouter le nom des colonnes absent dans le fichier d'origine

columns = ['Number of times pregnant',' Plasma glucose concentration',
           'Diastolic blood pressure','Triceps skin fold thickness',
           ' 2-Hour serum insulin (mu U/ml)','Body mass index (weight in kg/(height in m)^2)',
           'Diabetes pedigree function','Age','Target']

df.columns = columns
Y =pd.DataFrame(df['Target'], index = df.index) # extraction de la variable cible au format DataFrame
X = df.copy()
del X['Target']
from sklearn import preprocessing

col = X.columns

X_scaled = preprocessing.scale(X)
X_scaled = pd.DataFrame(X_scaled, columns = col)
```

SÉLECTION DE VARIABLES EXPLICATIVES

MÉTHODES AUTOMATISÉES (RFE, RFECV)

- Python propose plusieurs solutions pour sélectionner les variables en fonction du modèle :

```
lr = LogisticRegression()
selecteur = RFE(estimator=lr, n_features_to_select=5, step=1)

sol = selecteur.fit(X_scaled, Y['Target'])
print(sol.n_features_) # 5 --> Normal on l'a fixé précédement
print(sol.support_) # [ True  True  True False False  True  True False]
print(sol.ranking_) # [1 1 1 4 3 1 1 2]

from sklearn.feature_selection import RFECV
selecteur_cv = RFECV(lr, step=1, cv=5, scoring='accuracy')

sol_cv = selecteur_cv.fit(X_scaled, Y['Target'])
print(sol_cv.n_features_) # 7
print(sol_cv.support_) # [ True  True  True False  True  True  True  True]
print(sol_cv.ranking_) # [1 1 1 2 1 1 1 1]
print(sol_cv.grid_scores_) # [0.74747475 0.75912062 0.76305916 0.76696376 0.77216705
                             # 0.76175197 0.77218403 0.77088532]
```

On fixe le nombre de variables à sélectionner et l'algorithme choisit les plus efficaces selon des critères qui dépendent du modèle

On choisit une métrique à optimiser et un nombre de cross validation par ensemble de variables
→ **Attention** au temps de calcul s'il y a beaucoup de variables

SÉLECTION DE VARIABLES EXPLICATIVES

MÉTHODES AUTOMATISÉES (RFE, RFECV) - APPLICATION

- On applique le choix de variables et on mesure la qualité de notre modèle :

```
X_red = X.iloc[:,sol_cv.support_]
len(X_scaled.columns) #4
acc_lr= []
auc_lr = []
acc_rf = []
auc_rf = []
skf = StratifiedKFold(n_splits = 5)
target = 'Target'

for train_index, test_index in skf.split(X_red,y):
    X_train = X_red.loc[train_index,:]
    X_test = X_red.loc[test_index,:]
    y_train = Y.loc[train_index,:]
    y_test = Y.loc[test_index,:]
    lr = LogisticRegression(random_state=123,solver='liblinear',max_iter=200)
    rf = RandomForestClassifier()
    cross(lr,X_train,y_train,X_test,y_test,acc_lr,auc_lr, target)
    cross(rf,X_train,y_train,X_test,y_test,acc_rf,auc_rf, target)

print(np.mean(acc_lr)) # 0.771 > 0.770 (résultat précédent)
print(np.mean(acc_rf)) # 0.740 < 0.759
print(np.mean(auc_lr)) # 0.833 < 0.832
print(np.mean(auc_rf)) # 0.777 < 0.818
```

```
from sklearn.model_selection import StratifiedKFold
from sklearn.ensemble import RandomForestClassifier
from sklearn import metrics
import numpy as np

def cross(clf,X_train,y_train,X_test,y_test,l_acc,l_auc, target):
    clf.fit(X_train,y_train[target])
    pred = clf.predict(X_test)
    pred_proba = clf.predict_proba(X_test)
    fpr, tpr, threshold = metrics.roc_curve(y_test, pred_proba[:,1])
    roc_auc = metrics.auc(fpr, tpr)
    l_acc.append(metrics.accuracy_score(y_test,pred))
    l_auc.append(roc_auc)
```

Conclusion :

Pour la métrique choisie et le modèle sélectionné nous avons un résultat légèrement supérieur avec un nombre de variable divisé par 2.
Mais pas dans les autres cas

| SÉLECTION DE VARIABLES EXPLICATIVES

MÉTHODES

- **Comment sont sélectionnées les variables :**

- **Modèles linéaires :**

- Calcule les paramètres du modèles (coefficients associés aux variables)
- Les variables avec les coefficients les plus faibles en valeur absolue sont alors supprimées
- → Important que toutes les variables soient centrées et réduites pour que les coefficients ne soient biaisés par un jeu d'échelle.

- **Les algorithmes à base d'arbres de décision :**

- Calcule l'importance des variables en fonction du critère sélectionné pour établit l'arbre (généralement Gini)
- Plus le coefficient est élevé et plus la variable est importante.

SÉLECTION DE VARIABLES EXPLICATIVES

EXEMPLE

- Exemple avec une forêt aléatoire :

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier

clf = RandomForestClassifier()
clf = clf.fit(X, Y['Target'])
info = pd.DataFrame(index = X.columns.tolist())
info["Coeff_importance"] = clf.feature_importances_

# Ecart type (modèles à plusieurs arbres) :
std = np.std([tree.feature_importances_ for tree in clf.estimators_],
             axis=0)
info['Ecart type'] = std
info.sort_values(by="Coeff_importance",ascending = False, inplace = True)

#
```

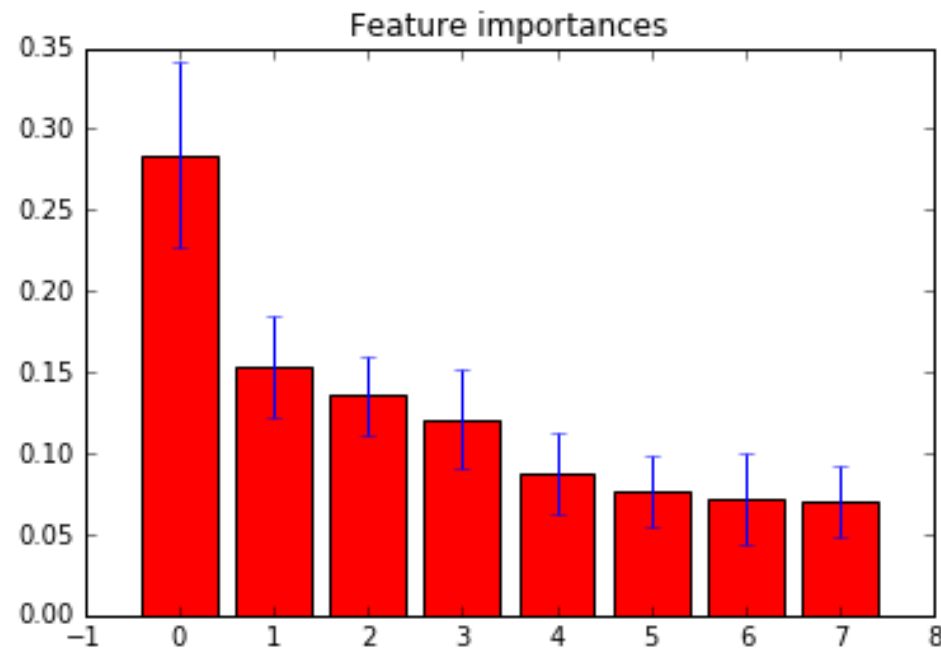
	Coeff_importance	Ecart type
# Plasma glucose concentration	0.284138	0.057054
#Body mass index (weight in kg/(height in m)^2)	0.153848	0.031167
#Diabetes pedigree function	0.135346	0.024858
#Age	0.121143	0.030339
#Diastolic blood pressure	0.087152	0.024849
#Number of times pregnant	0.076997	0.022182
# 2-Hour serum insulin (mu U/ml)	0.071377	0.028540
#Triceps skin fold thickness	0.069999	0.022114

SÉLECTION DE VARIABLES EXPLICATIVES

FEATURE IMPORTANCE

- Exemple avec une forêt aléatoire (graphique) :

```
# Plot the feature importances of the forest
plt.figure()
plt.title("Feature importances")
plt.bar(range(len(info)), info["Coeff_importance"], color="r", yerr=info['Ecart type'],
align="center")
plt.show()
```



SÉLECTION DE VARIABLES EXPLICATIVES

SELECT K-BEST

- Sélectionne k variables qui ont le meilleur le score :
 - Le score est issu d'un test statistique entre la variable cible et une variable explicative
 - Les tests proposés :
 - ANOVA (f_classif par défaut)
 - Mutual_info_classif
 - chi2 (**attention pas fiable**)
 - F-value pour la régression
 - Mutual_info_regression
 - Fonctions similaires :
 - SelectPercentile
 - SelectFpr
 - SelectFdr
 - SelectFwe
 - GenericUnivariateSelect

SÉLECTION DE VARIABLES EXPLICATIVES

SELECT K-BEST (APPLICATION)

- Exemple utilisation :

```
from sklearn.feature_selection import *
anova = SelectKBest(f_classif, k = 'all')
anova.fit_transform(X, Y['Target'])
score = anova.scores_
#array([ 39.31428116, 212.03864104,   3.23399258,   4.12849461,
#        13.56233147,  71.67881257,  23.67113786,  45.28969585])
pval = anova.pvalues_
#array([ 6.02764976e-10,  1.40914192e-42,  7.25187762e-02,
#        4.25119303e-02,  2.47006859e-04,  1.28668938e-16,
#        1.38808834e-06,  3.33255459e-11])

'''On souhaite sélectionner 5 variables'''
anova = SelectKBest(f_classif, k = 5)
X_trans2 = anova.fit_transform(X, Y['Target'])
score = anova.scores_
#array([ 39.31428116, 212.03864104,   3.23399258,   4.12849461,
#        13.56233147,  71.67881257,  23.67113786,  45.28969585])
mask = anova.get_support()
#array([ True,  True, False, False, False,
#        True,  True,  True], dtype=bool)
```


01

Introduction à Scikit-Learn

02

Principaux concepts de Scikit-Learn

03

Partitionnement des données

04

Préprocessing

05

Fonctions avancées pour la gestion des variables catégorielles

06

Régressions logistiques (ridge, lasso, elasticnet)

07

Algorithmes à base d'arbres

08

Éléments pour l'interprétation

09

Sélection de variables explicatives

10

Optimisation des hyperparamètres

11

Métriques d'évaluation de la performance des modèles

OPTIMISATION DES HYPERPARAMÈTRES

MÉTHODES D'OPTIMISATION

➤ Les hyperparamètres sont les paramètres d'ajustements des différents algorithmes de Machine Learning.

➤ Par exemple, dans le cadre d'un Random Forest, on retrouve notamment :

- Le nombre d'arbres
- Critère d'impureté
- La profondeur maximale des arbres
- Etc.

➤ L'optimisation des hyperparamètres est le processus de recherche de la configuration des hyperparamètres qui produit les meilleures performances

➤ 4 grands types de méthodes d'optimisation :

- **Recherche manuelle** : on fait varier manuellement les hyperparamètres
- **Recherche par grille (Grid Search)** : lancement du processus d'entraînement pour une liste de valeurs de nos hyperparamètres de manière automatisée.
 - Exemple : pour une Forêt Aléatoire, essayer un modèle avec 100 arbres, puis avec 200 et enfin avec 500 de manière automatisée en comparant les performances obtenues.
- **Recherche aléatoire (Random Search)** : même principe que pour le Grid Search mais cette fois-ci en choisissant de manière aléatoire la plage de configurations.
- **Optimisation bayésienne** : des valeurs initiales des hyperparamètres sont choisis. Puis, on les fait varier de manière itératif en prenant en compte les résultats des itérations précédentes.

OPTIMISATION DES HYPERPARAMÈTRES

GRID SEARCH

```
import pandas as pd
#Lecture de la table
path_data = 'C:/Formation Python/Datasets/'
df = pd.read_csv(path_data + 'adult.csv', sep = ',')

y = df['income'].copy()
del df['income'] #df est maintenant composée que de la matrice des X

#Dummmisation des variables catégorielles
df = pd.get_dummies(df)

from sklearn.model_selection import train_test_split
#Séparation apprentissage/test
X_train, X_test, y_train, y_test = train_test_split(df, y, test_size=0.3, random_state=123)

from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier()

# Grille de recherche
param_grid = {'bootstrap': [True],
              'max_depth': [6, 10],
              'max_features': ['auto', 'sqrt'],
              'min_samples_leaf': [3, 5],
              'min_samples_split': [4, 6],
              'n_estimators': [50, 100]
             }

from sklearn.model_selection import GridSearchCV
forest_grid_search = GridSearchCV(rf, param_grid, cv=3, scoring='accuracy',
return_train_score=True,n_jobs=-1)

forest_grid_search.fit(X_train, y_train)
forest_grid_search.best_params_
forest_grid_search.best_score_ # 0.8572347831173769
```

param_grid demande à Scikit-Learn d'évaluer 32 combinaisons (1 x 2 x 2 x 2 x 2 x 2).

```
{'bootstrap': True,
 'max_depth': 10,
 'max_features': 'auto',
 'min_samples_leaf': 3,
 'min_samples_split': 4,
 'n_estimators': 100}
```

OPTIMISATION DES HYPERPARAMÈTRES

RANDOM SEARCH

```
param_space = {"bootstrap": [True],
               "max_depth": [6, 8, 10, 12, 14],
               "max_features": ['auto', 'sqrt', 'log2'],
               "min_samples_leaf": [2, 3, 4],
               "min_samples_split": [2, 3, 4, 5],
               "n_estimators": [20, 50, 100, 150, 200, 300]
}

from sklearn.model_selection import RandomizedSearchCV
forest_rand_search = RandomizedSearchCV(rf, param_space, n_iter=30,
                                       scoring="accuracy", cv=3,
                                       n_jobs=-1, random_state=123)

forest_rand_search.fit(X_train, y_train)
forest_grid_search.best_params_
forest_grid_search.best_score_ # 0.8572347831173769
```

Scikit-Learn va sélectionner $n_iter=30$ combinaisons aléatoirement

OPTIMISATION DES HYPERPARAMÈTRES

OPTIMISATION BAYÉSIENNE ?

- L'optimisation bayésienne diffère de la recherche aléatoire et de la recherche de grille car elle améliore la vitesse de recherche en utilisant les performances passées (les deux autres méthodes sont indépendantes des évaluations passées).

```
from skopt.space import Real, Categorical, Integer

search_space = {"bootstrap": Categorical([True, False]),
                "max_depth": Integer(6, 16),
                "max_features": Categorical(['auto', 'sqrt', 'log2']),
                "min_samples_leaf": Integer(2, 10),
                "min_samples_split": Integer(2, 10),
                "n_estimators": Integer(50, 300)
               }

from skopt import BayesSearchCV
forest_bayes_search = BayesSearchCV(rf, search_space, n_iter=30, random_state=123,
                                    scoring="accuracy", n_jobs=-1, cv=3)

forest_bayes_search.fit(X_train, y_train)
forest_bayes_search.best_params_
forest_bayes_search.best_score_ # 0.8626751294275937
```

01

Introduction à Scikit-Learn

02

Principaux concepts de Scikit-Learn

03

Partitionnement des données

04

Préprocessing

05

Fonctions avancées pour la gestion
des variables catégorielles

06

Régressions logistiques
(ridge, lasso, elasticnet)

07

Algorithmes à base d'arbres

08

Éléments pour l'interprétation

09

Sélection de variables explicatives

10

Optimisation des hyperparamètres

11

Métriques d'évaluation de la
performance des modèles

MÉTRIQUES D'ÉVALUATION DE LA QUALITÉ D'UN MODÈLE

- Scikit-learn propose plusieurs métriques pour évaluer la qualité d'un modèle (cette liste n'est pas exhaustive) :
- **Classification :**
 - **Accuracy** – % global de bien classés : `metrics.accuracy_score`
 - **Recall** – Sensibilité – Taux de Vrais Positifs : `metrics.recall_score`
 - **Precision** – % bien classés 1 parmi les observés 1 : `metrics.precision_score`
 - **ROC - AUC** : `metrics.roc_auc_score`
- **Régression :**
 - **R²** : `metrics.r2_score`
 - **Mean Squared Error** : `metrics.mean_squared_error`
 - **Mean Absolute Error** : `metrics.mean_absolute_error`
 - **Explained Variance** : `metrics.variance_score`
- http://scikit-learn.org/stable/modules/model_evaluation.html

Remarque : il est possible de comparer les performances des modèles pour ces métriques

MÉTRIQUES D'ÉVALUATION

PERFORMANCE D'UN MODÈLE DE SCORING

MÉTRIQUES STANDARDS

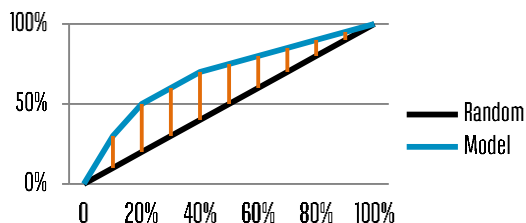
Une métrique évalue les performances d'un modèle prédictif. Le choix de la métrique dépend des objectifs du modèle et de la structure du dataset.

Accuracy :

- Basée sur la matrice de confusion
- Détermine le % de prédictions correctes

		PREDICTED	
		Y = 0	Y = 1
REAL	Y=0	0.7%	0.3%
	Y=1	0.2%	0.8%

AUC / Gini :



- Le meilleur modèle est celui qui enregistre le résultat le plus haut
- Evaluer un bon classement en terme de probabilité estimée

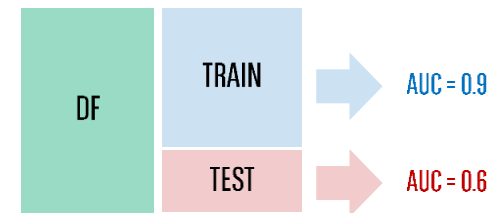
SUR-APPRENTISSAGE

On parle de sur-apprentissage quand un modèle obtient de bonnes performances sur le dataset d'apprentissage (train) mais de mauvaises performances sur un nouveau jeu de données (test).

Par exemple, le sur-apprentissage se produit quand la profondeur d'un arbre de décision est trop grande et que le modèle n'est pas généralisable à d'autres données.

Comment l'évaluer ?

- Subdiviser le Dataset en 2 : Train / Test
- Entraîner le modèle sur le Train
- Tester le modèle sur le Test
- Observer les performances du modèle sur les deux échantillons
- Le sur-apprentissage est mesuré par la différence des performances



Comment le résoudre ?

- Intégrer plus de données / Sélection de variables
- Réduire la complexité du modèle
- Agrégation de plusieurs Classifieurs
- Cross-Validation



```
from sklearn.metrics import roc_auc_score  
  
print(roc_auc_score(T_true, Y_predicted))
```


MÉTRIQUES D'ÉVALUATION

MATRICE DE CONFUSION

Matrice de confusion

		Predicted condition			
		Total population	Predicted Condition positive	Predicted Condition negative	Prevalence $= \frac{\Sigma \text{Condition positive}}{\Sigma \text{Total population}}$
True condition	condition positive	True positive	False Negative (Type II error)	True positive rate (TPR), Sensitivity, Recall, probability of detection $= \frac{\Sigma \text{True positive}}{\Sigma \text{Condition positive}}$	False negative rate (FNR), Miss rate $= \frac{\Sigma \text{False negative}}{\Sigma \text{Condition positive}}$
	condition negative	False Positive (Type I error)	True negative	False positive rate (FPR), Fall-out, probability of false alarm $= \frac{\Sigma \text{False positive}}{\Sigma \text{Condition negative}}$	True negative rate (TNR), Specificity (SPC) $= \frac{\Sigma \text{True negative}}{\Sigma \text{Condition negative}}$
Accuracy (ACC) = $\frac{\Sigma \text{True positive} + \Sigma \text{True negative}}{\Sigma \text{Total population}}$		Positive predictive value (PPV), Precision $= \frac{\Sigma \text{True positive}}{\Sigma \text{Test outcome positive}}$	False omission rate (FOR) = $\frac{\Sigma \text{False negative}}{\Sigma \text{Test outcome negative}}$	Positive likelihood ratio (LR+) = $\frac{\text{TPR}}{\text{FPR}}$	Diagnostic odds ratio (DOR) = $\frac{\text{LR+}}{\text{LR-}}$
		False discovery rate (FDR) $= \frac{\Sigma \text{False positive}}{\Sigma \text{Test outcome positive}}$	Negative predictive value (NPV) $= \frac{\Sigma \text{True negative}}{\Sigma \text{Test outcome negative}}$	Negative likelihood ratio (LR-) = $\frac{\text{FNR}}{\text{TNR}}$	

Précision, accuracy, recall sont des métriques pour évaluer la qualité du modèle

MÉTRIQUES D'ÉVALUATION

AUC ET COURBE ROC

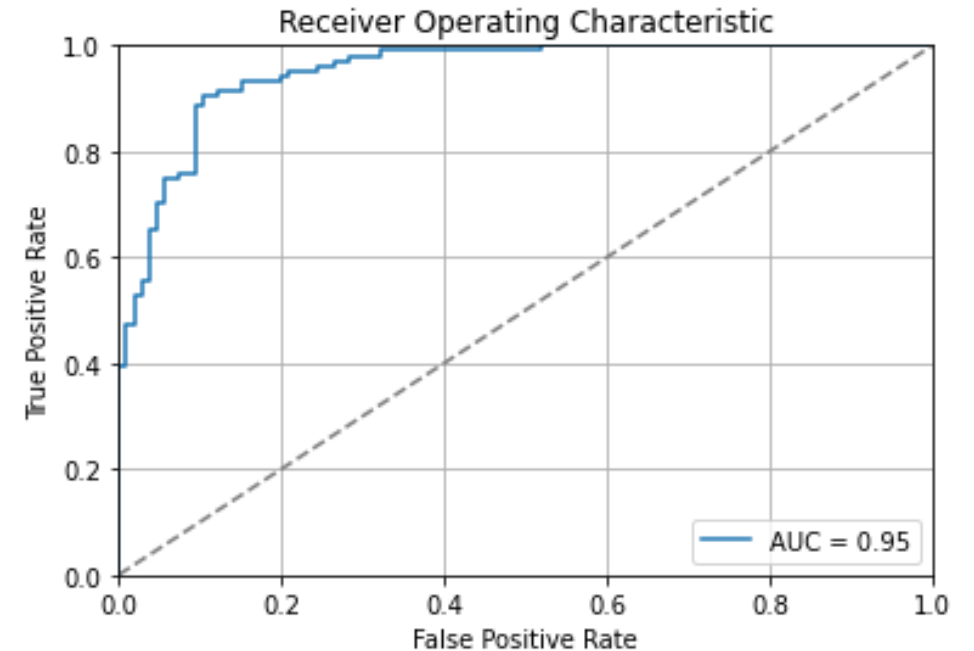
Cas d'une régression logistique :

```
import pandas as pd
```

```
# Création d'un dataset fictif
from sklearn.datasets import make_classification
X, Y = make_classification(n_samples=700)
# Partitionnement des index en train / test
from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size = 0.3, stratify = Y, random_state = 0)
# Régression logistique
from sklearn.linear_model import LogisticRegression
lm = LogisticRegression()
lm.fit(X_train, Y_train)
Y_pred = lm.predict(X_test)
prob = lm.predict_proba(X_test)
print(lm.coef_, lm.intercept_) # Pour imprimer les coefficients
# Metrique AUC
from sklearn import metrics
fpr, tpr, threshold = metrics.roc_curve(Y_test, prob[:,1])
roc_auc = metrics.auc(fpr, tpr)
```

```
# Courbe ROC
import matplotlib.pyplot as plt
plt.title('Receiver Operating Characteristic')
plt.plot(fpr, tpr, 'b', label = 'AUC = %.2f' % roc_auc)
plt.legend(loc = 'lower right')
plt.plot([0, 1], [0, 1], 'r--')
plt.xlim([0, 1])
plt.ylim([0, 1])
plt.ylabel('True Positive Rate')
plt.xlabel('False Positive Rate')
plt.grid(True)
plt.show()
```

Scikit-learn propose plusieurs métriques (slide suivante)



MÉTRIQUES D'ÉVALUATION

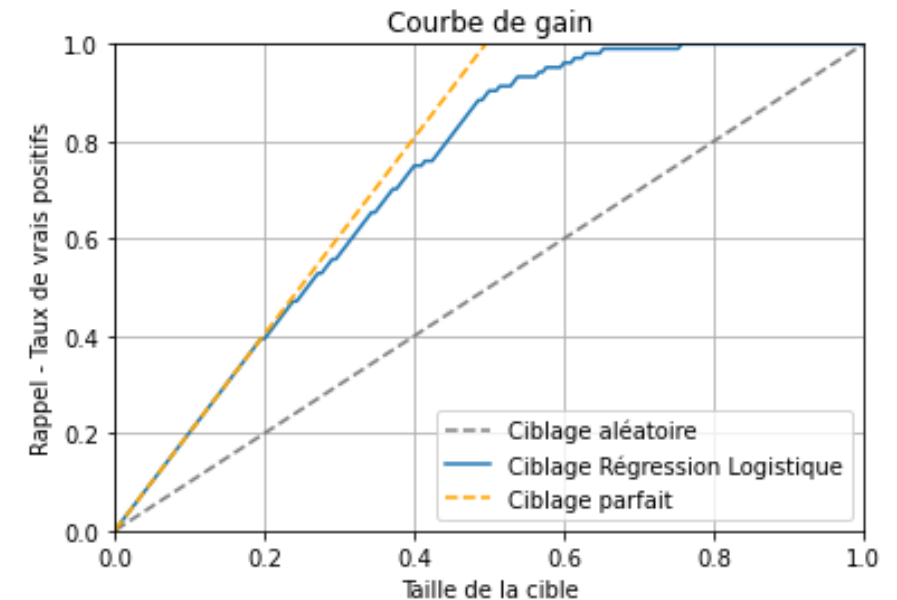
COURBE DE GAIN

- **Création du graphique** (à exécuter à la suite du script du slide précédent) :

```
n = len(Y_test)
n1 = sum(Y_test)
n0 = n-n1
# Taille cumulée de la cible
taille = [(fpr[i]*n0+tptr[i]*n1)/n for i in range(len(fpr))]

# titre principal et libellé des axes
plt.title('Courbe de gain')
plt.xlabel('Taille de la cible')
plt.ylabel('Rappel - Taux de vrais positifs')
# limites en abscisse et ordonnée
plt.xlim(0,1)
plt.ylim(0,1)
# Courbe du ciblage aléatoire
plt.plot(taille,taille,color='grey',linestyle='dashed', label='Ciblage aléatoire')
# Courbe du modèle
plt.plot(taille, tpr, label='Ciblage Régression Logistique',)
# Courbe optimale
plt.plot([0, n1/n],[0,1],color='orange',linestyle='dashed', label='Ciblage parfait',)
# affichage de la légende et du quadrillage
plt.legend()
plt.grid(True)
plt.show()
```

- On peut aller plus loin en créant la **courbe de lift**
- **lift = rappel / hasard** (exemple : $0,4/0,2 = 2$) → permet de mesurer l'apport du modèle (gain d'information par rapport à un tirage au sort)

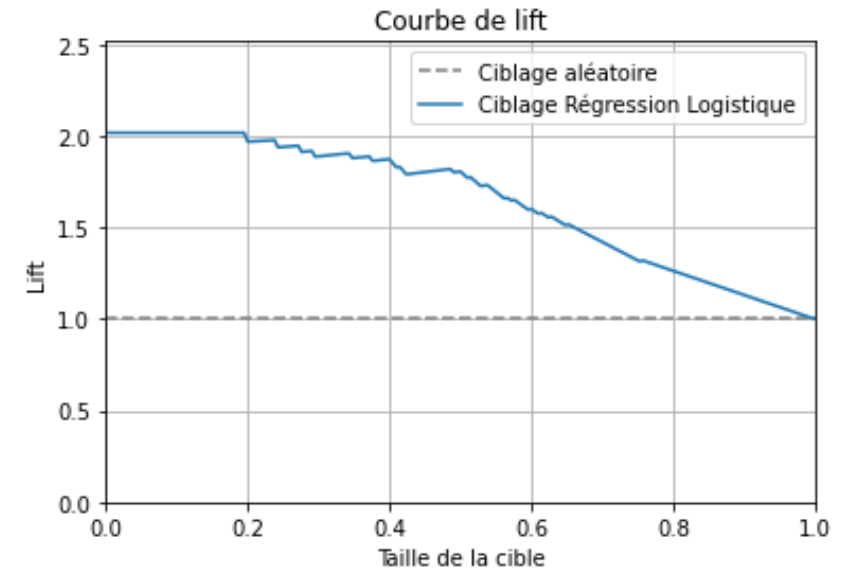


MÉTRIQUES D'ÉVALUATION

COURBE DE LIFT

- **Création de la courbe de lift** (à exécuter à la suite des scripts des deux slides précédents) :

```
plt.title('Courbe de lift')
plt.xlabel('Taille de la cible')
plt.ylabel('Lift')
# Calcul du lift
lift = tpr/taille
# limites en abscisse et ordonnée
plt.xlim(0,1)
plt.ylim(0, max(lift[1:]) + 0.5)
# Ciblage aléatoire
plt.plot([0,1], [1,1], color='grey', linestyle='dashed', label='Ciblage aléatoire')
# Ciblage du modèle
plt.plot(taille, lift, label='Ciblage Régression Logistique')
# affichage de la légende et du quadrillage
plt.legend()
plt.grid(True)
plt.show()
```



MÉTRIQUES D'ÉVALUATION

ÉVALUATION DE LA QUALITÉ D'UN MODÈLE : MÉTRIQUES

Scikit-learn propose plusieurs métriques pour évaluer la qualité d'un modèle ,
quelques exemples :

```
'''importation de metrics - utilisé pour les mesures de performances'''
from sklearn import metrics
#matrice de confusion
#confrontation entre Y obs. sur l'éch. test et la prédiction
cm = metrics.confusion_matrix(Y_test,Y_pred)
print(cm)
#[[139  20]
# [ 31  41]]

'''taux de succès'''
acc = metrics.accuracy_score(Y_test,Y_pred)
print(acc) # 0.779 = (139 + 41)/ (139 + 20 + 31 + 41)
'''taux d'erreur'''
err = 1.0 - acc
print(err) # 0.221
'''sensibilité (ou rappel)'''
se = metrics.recall_score(Y_test,Y_pred)
print(se) # 0.569 = 41 / (31+ 41)
'''précision'''
pr = metrics.precision_score(Y_test,Y_pred)
print(pr) # 0.672 = 41/(20 + 41)
```

Matrice de confusion :

- Ligne : valeurs observées
- Colonne : valeurs prédites

MÉTRIQUES D'ÉVALUATION

CROSS VALIDATION : CHOIX DU MODÈLE

- L'objectif de la cross validation peut être double :
 - Estimer la qualité du modèle selon une métrique
 - Trouver les paramètres optimaux d'un modèle
- Exemple : choix entre 2 modèles Random Forest et régression logistique
- Code Python – Option 1 :

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn import cross_validation

nb_cross = 5
lr = LogisticRegression()
rf = RandomForestClassifier()
lr.fit(X,Y)
rf.fit(X.values,Y)
y = list(Y['Target']) # Question de format

CV_lr = cross_validation.cross_val_score(lr,X_scaled,y,cv=nb_cross,
                                         scoring='accuracy')

print(CV_lr) # [ 0.77272727  0.74025974  0.75816993  0.81699346  0.76470588]
print(CV_lr.mean()) # 0.770571258807

CV_rf = cross_validation.cross_val_score(rf,X_scaled,y,cv=nb_cross,
                                         scoring='accuracy')

print(CV_rf) # [ 0.72727273  0.70779221  0.73856209  0.78431373  0.76470588]
print(CV_rf.mean()) # 0.744529326882
```

- L'avantage de cette solution est la simplicité d'écriture
 - Problèmes:
 1. Les calculs ne sont pas effectués avec le même découpage
 2. On ne choisit pas si les folds sont stratifiés ou pas
 3. Si l'on souhaite plusieurs métriques il faut répéter l'opération (coût de calcul élevé)
- Simple à écrire mais un peu boîte noire

Conclusion : la régression logistique est plus adaptée que le Random Forest

MÉTRIQUES D'ÉVALUATION

CROSS VALIDATION : CHOIX DU MODÈLE

■ Code Python – Option 2 :

```
k=5
skf = StratifiedKFold(y, n_folds = k)
#kf = KFold(n = k)

acc_lr= []
auc_lr = []
acc_rf = []
auc_rf = []
Y = pd.DataFrame(Y) # Question de format
for train_index, test_index in skf:
    X_train = X_scaled.loc[train_index,:]
    X_test = X_scaled.loc[test_index,:]
    y_train = Y.loc[train_index,:]
    y_test = Y.loc[test_index,:]
    lr = LogisticRegression()
    rf = RandomForestClassifier()
    cross(lr,X_train,y_train,X_test,y_test,acc_lr,auc_lr)
    cross(rf,X_train,y_train,X_test,y_test,acc_rf,auc_rf)

print(np.mean(acc_lr)) #0.770
print(np.mean(acc_rf)) # 0.759
print(np.mean(auc_lr)) # 0.832
print(np.mean(auc_rf)) # 0.818
```

```
from sklearn import metrics
from sklearn.cross_validation import StratifiedKFold
#from sklearn.cross_validation import KFold

def cross(clf,X_train,y_train,X_test,y_test,l_acc,l_auc):
    clf.fit(X_train,y_train)
    pred = clf.predict(X_test)
    pred_proba = clf.predict_proba(X_test)
    fpr, tpr, threshold = metrics.roc_curve(y_test, pred_proba[:,1])
    roc_auc = metrics.auc(fpr, tpr)
    l_acc.append(metrics.accuracy_score(y_test,pred))
    l_auc.append(roc_auc)
```

- Code plus long (bien qu'optimisé par une fonction)
- Mais on sait exactement ce qu'il se passe
- Plus rapide en exécution si on souhaite plusieurs métriques

**MERCI
DE VOTRE
ATTENTION !**

**Contactez-
nous**



www.micropole.com



[@groupemicropole](https://twitter.com/groupemicropole)



91/95 rue Carnot,
92300 Levallois-Perret



01 74 18 74 18

MICROPOLE

TANGUY LE NOUVEL
DIRECTEUR DATA SCIENCE

M +33 (0) 6 81 95 84 22
@ tlenouvel@micropole.com

WWW.MICROPOLE.COM

91-95 RUE CARNOT - 92 300 LEVALLOIS-PERRET
T +33 (0)1 74 18 74 18 / **F** +33 (0)1 74 18 74 00

MICROPOLE

FORMATION PYTHON
CONCEPTS DE BASE,
DATA PREPARATION ET MACHINE LEARNING
JANVIER 2021



MICROPOLÉ